

CSCI 598H

Applied Computer Vision for Engineering Systems

Lecture 4-1: Recurrent Neural Networks (RNNs)

Dr. Minghan Li

Assistant Professor

Computer Science Department

Colorado School of Mines

Aug/27/2026



CSCI 598H Course Overview



Weeks 2-4

Weeks 5-11

Weeks 12-16

Module 1

Deep Learning Basics

- Linear Classifier and Optimization
- CNNs and BackPropagation
- **RNNs, Attention, Transformers**

Module 2

Modern Visual Intelligence

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

Module 3

Engineering Systems

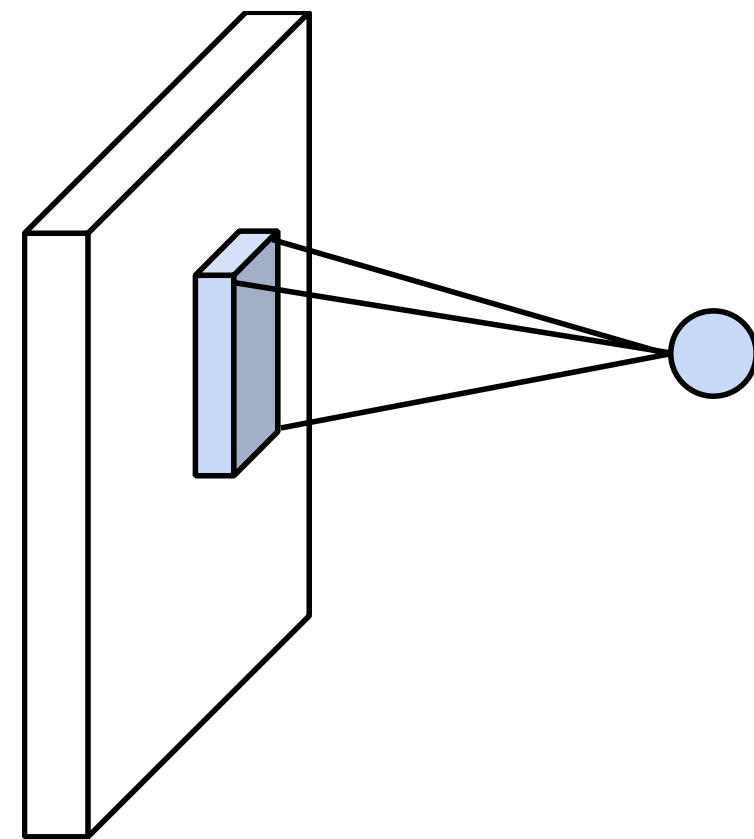
- Robotics
- Earth science
- Medical imaging
- Energy

Recap

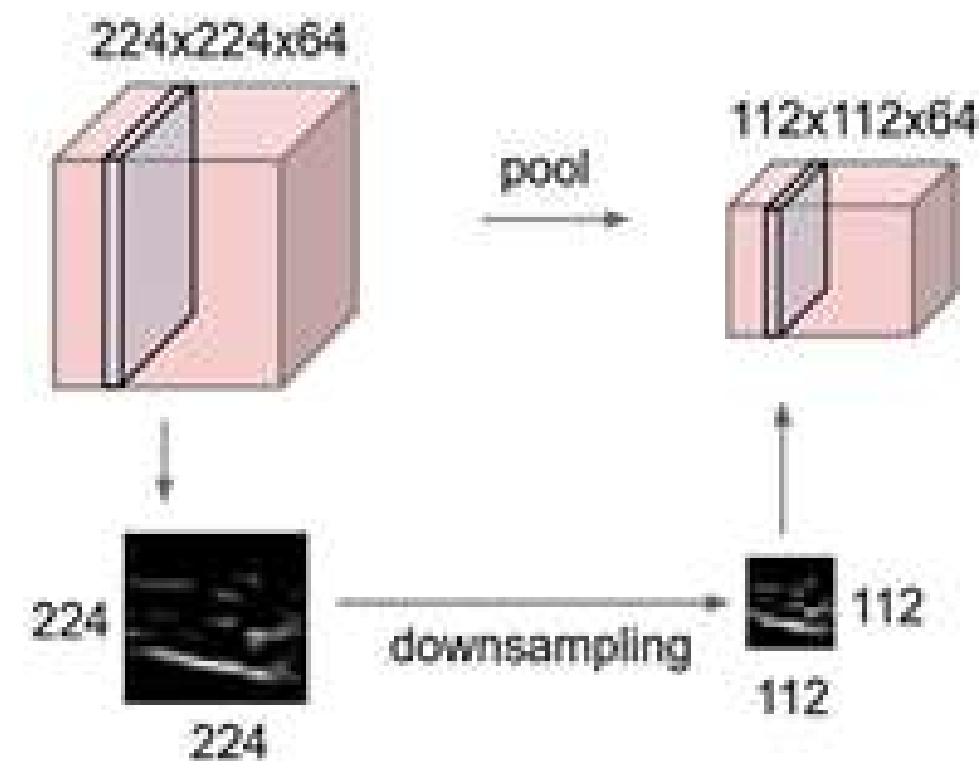
W3: Neural Networks and BackPropagation

Recap: Components of CNNs

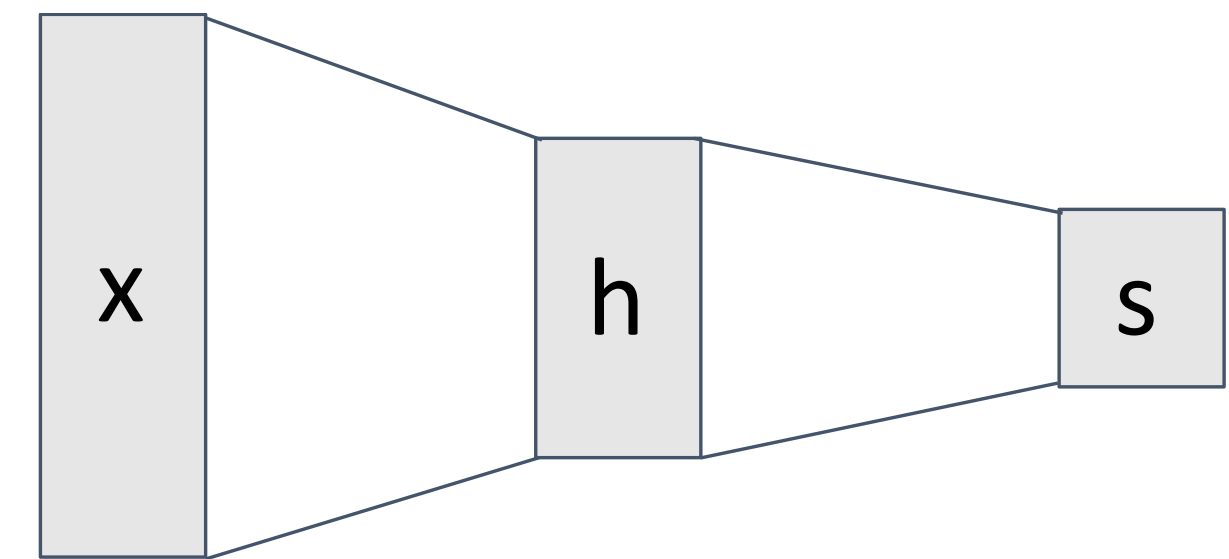
Convolution Layers



Pooling Layers



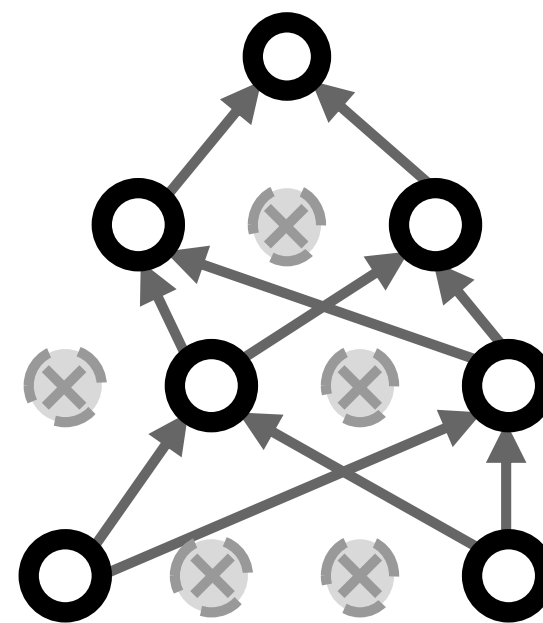
Fully-Connected Layers



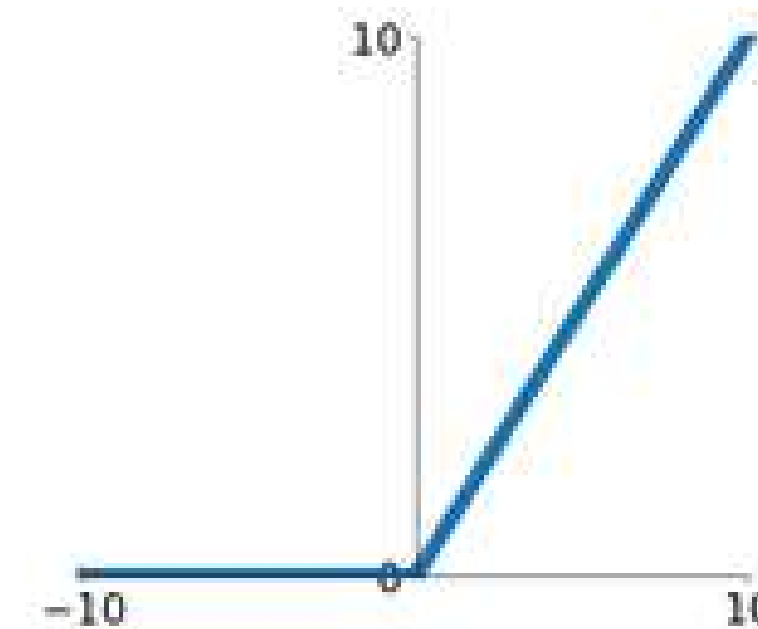
Normalization Layers

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Dropout (sometimes)



Activation Functions



Recap: Convolutional Neural Networks

Convolution and **pooling** operators extract features while respecting 2D image structure

Fully-Connected layers form an MLP at the end to predict scores

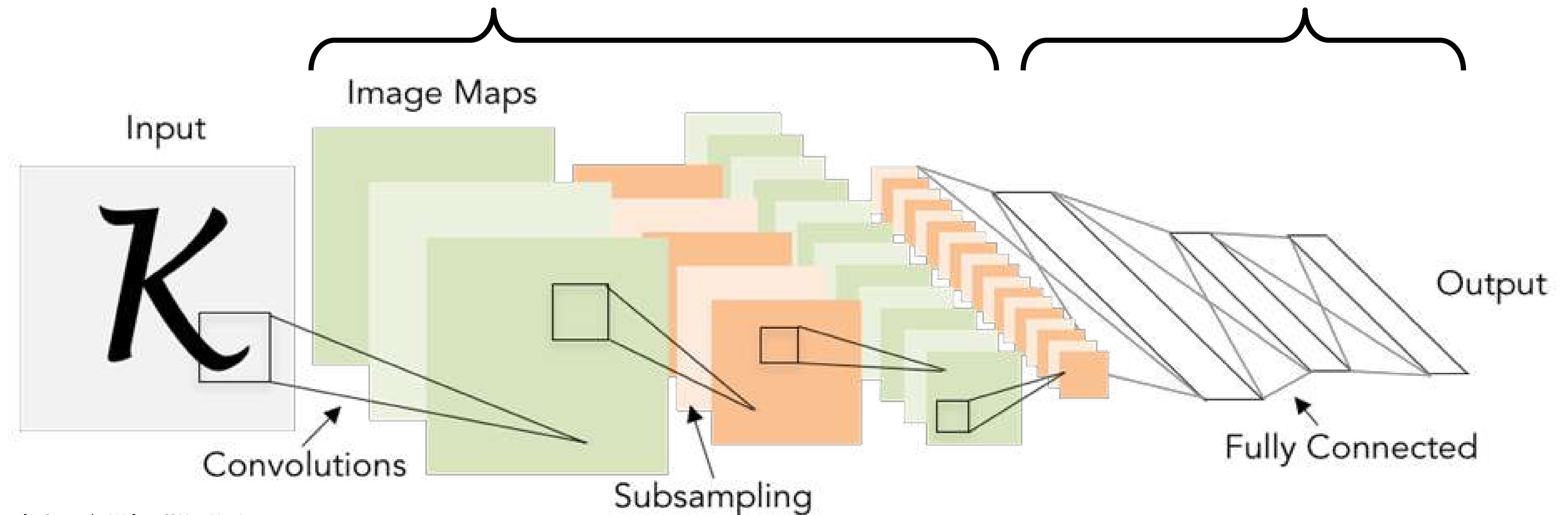


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

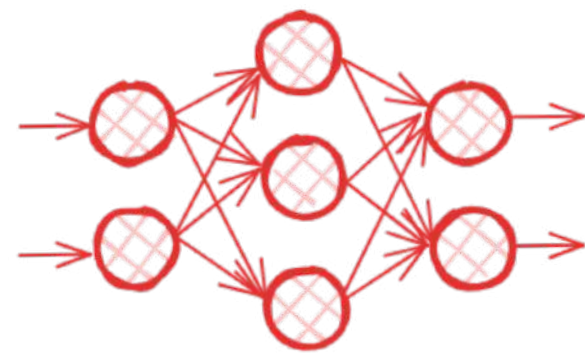
LeNet5 in 1998

Recurrent Neural Networks (RNNs)

Additional materials & code demo:

https://d2l.ai/chapter_recurrent-neural-networks/index.html

One to One “Vanilla” NN



Donkey



$$\hat{y} = f(x)$$

$$x \in R^m$$

$$\hat{y} \in R^n$$

Sequence Data and Modeling

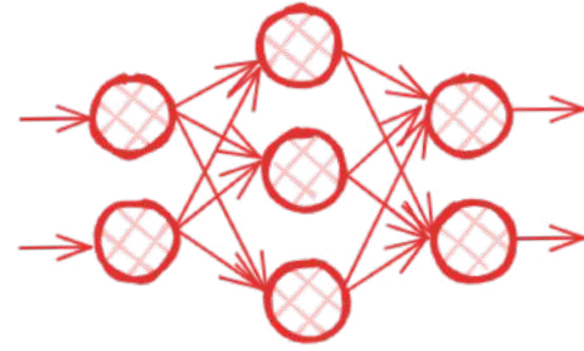
Time



⋮

Handling Individual Time Steps

Time



Donkey

$$\hat{y}_0 = f(x_0)$$

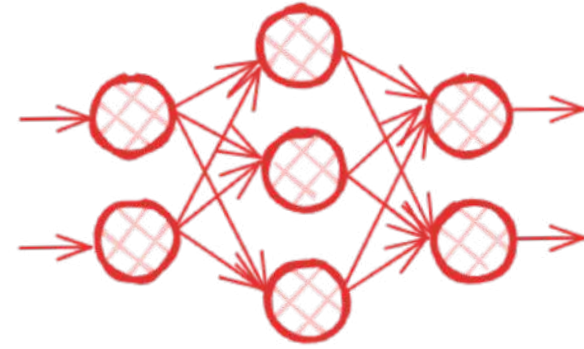


⋮

$$x_t \in R^m$$
$$\hat{y}_t \in R^n$$

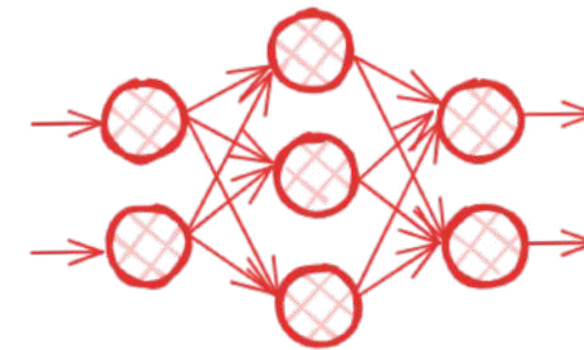
Handling Individual Time Steps

Time



Donkey

$$\hat{y}_0 = f(x_0)$$



Donkey

$$\hat{y}_1 = f(x_1)$$



⋮

$$x_t \in R^m$$

$$\hat{y}_t \in R^n$$

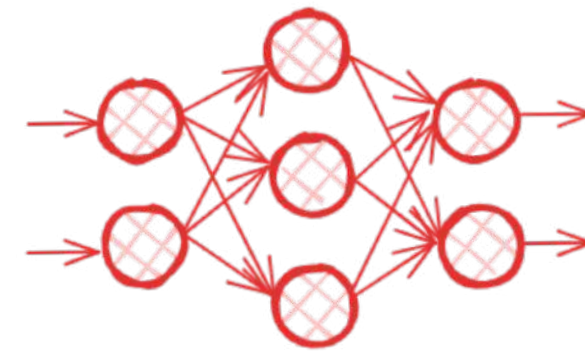
Handling Individual Time Steps

Time



Donkey

$$\hat{y}_0 = f(x_0)$$



Donkey

$$\hat{y}_1 = f(x_1)$$



Donkey

$$\hat{y}_2 = f(x_2)$$

⋮

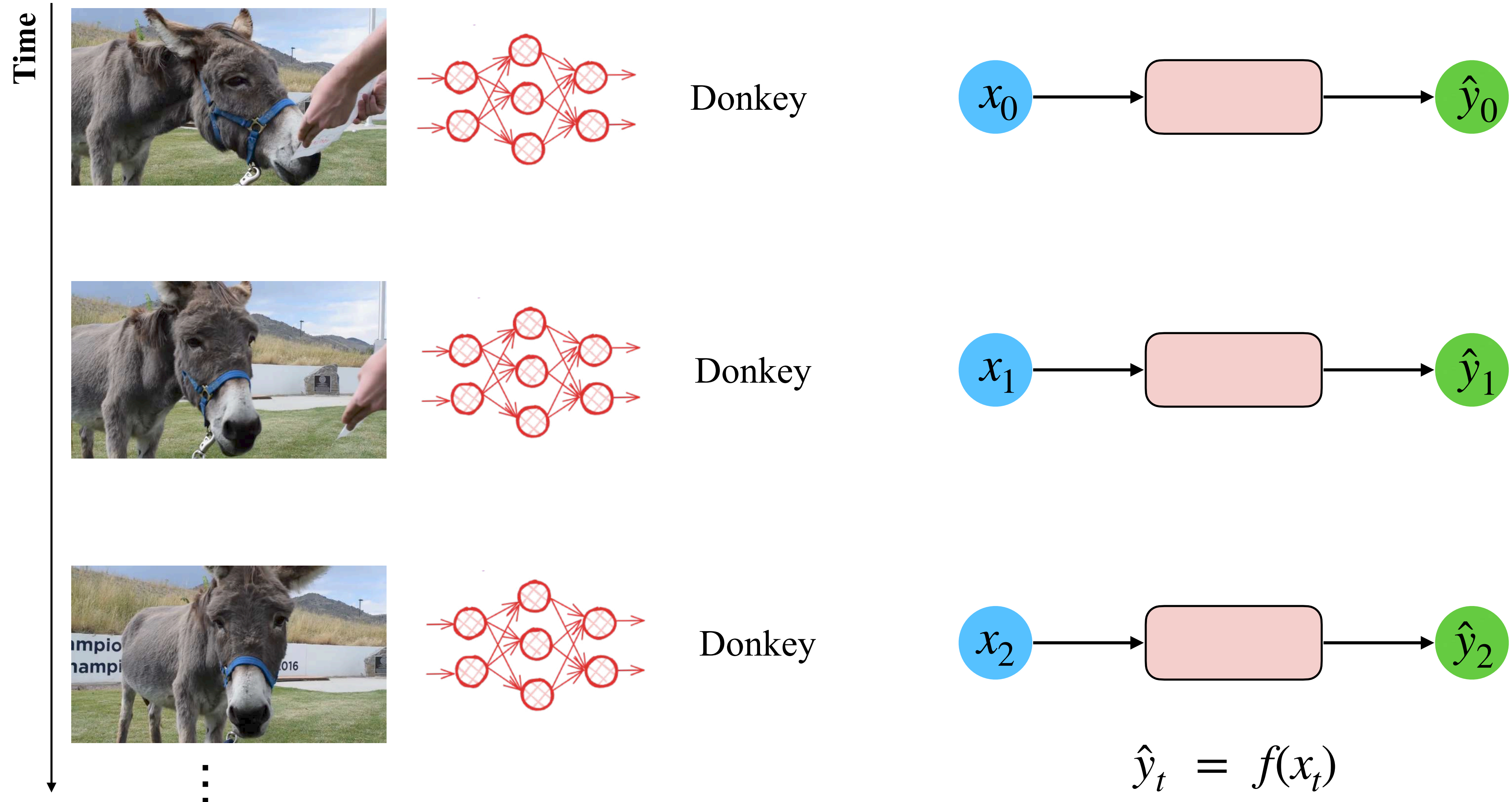
⋮

⋮

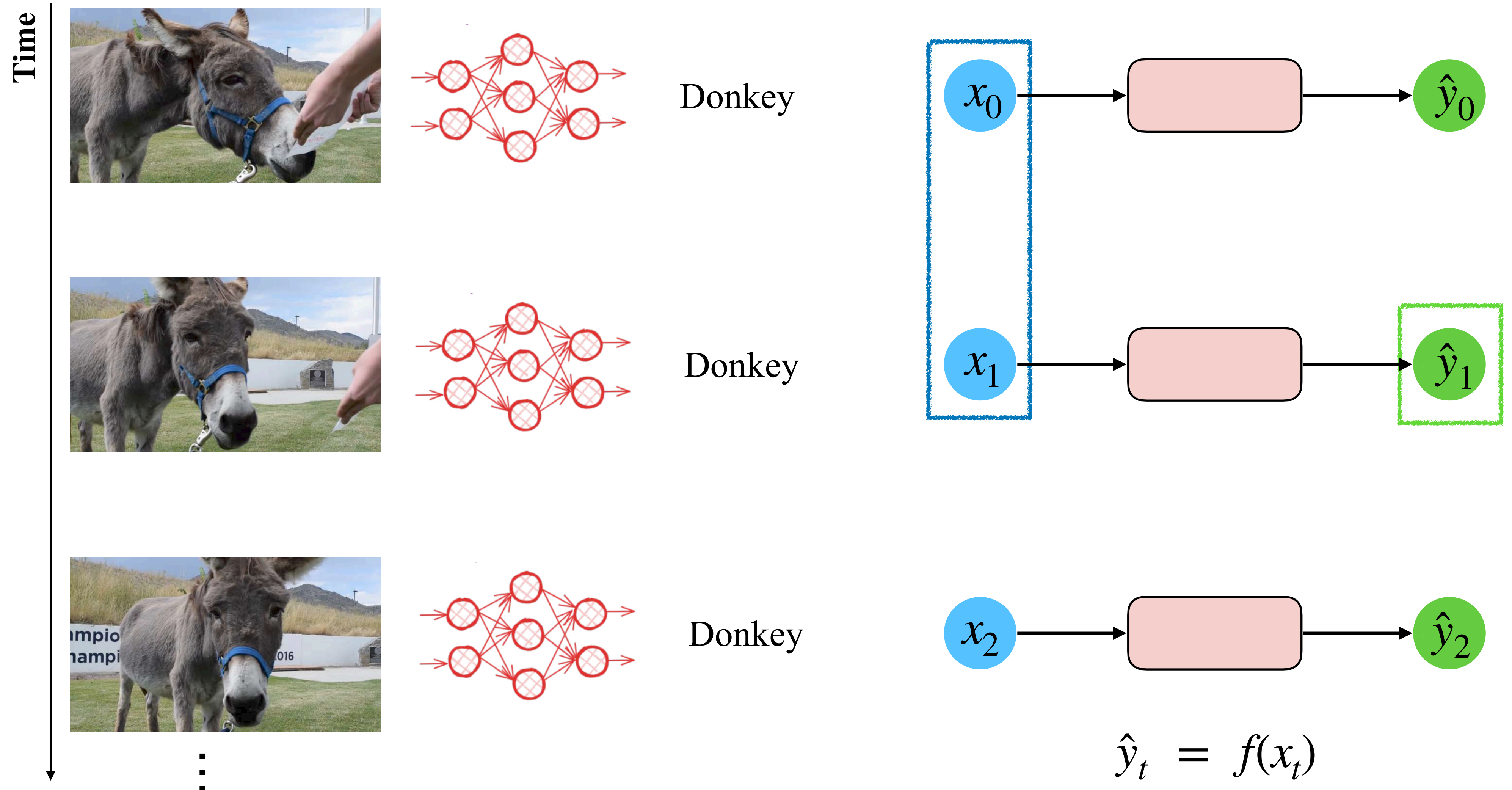
⋮

$$x_t \in R^m$$
$$\hat{y}_t \in R^n$$

Handling Individual Time Steps

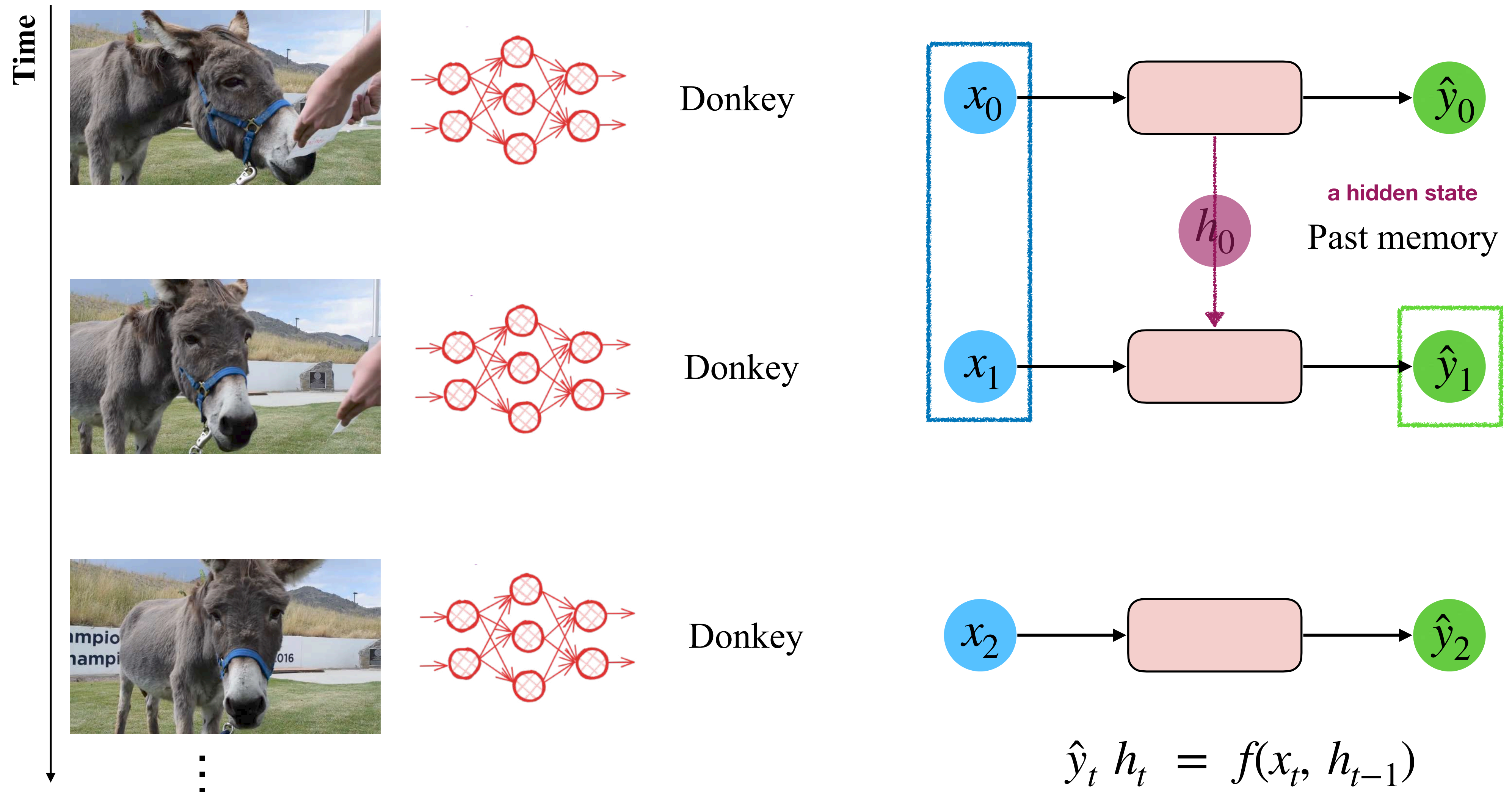


Handling Individual Time Steps

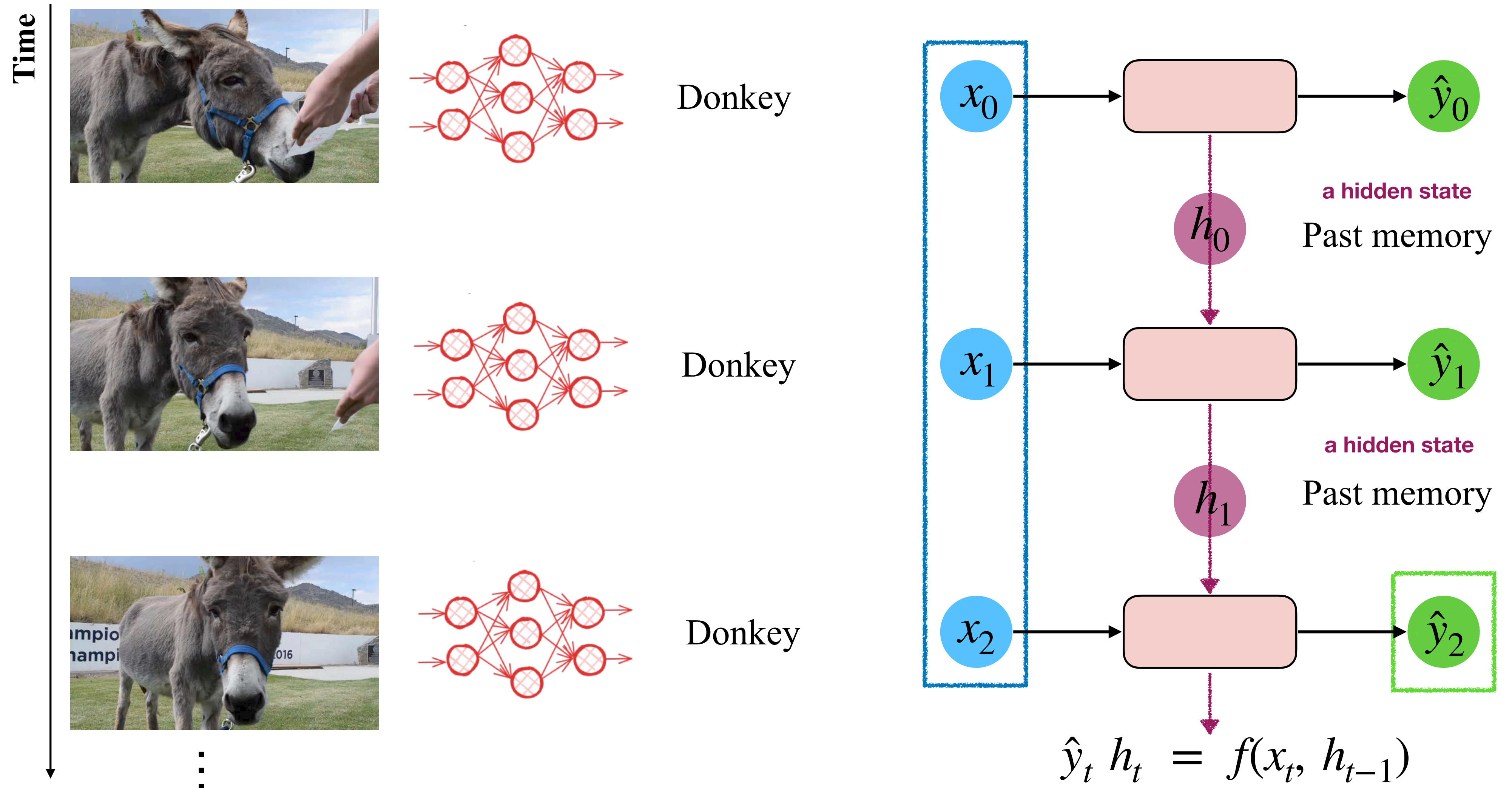


Neurons with Recurrence

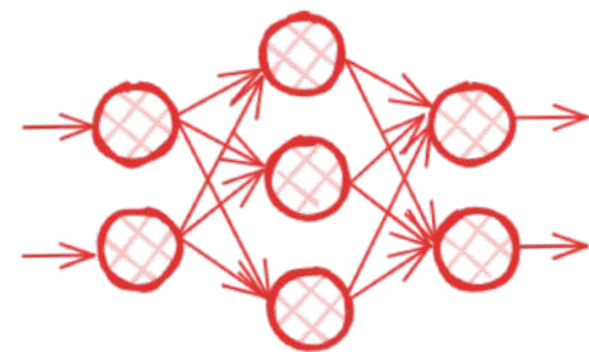
The hidden state acts as **a learned summary of earlier inputs.**



Neurons with Recurrence



“Vanilla” NN V.S. RNN



Donkey



$$\hat{y} = f(x)$$

$$x \in R^m$$

$$\hat{y} \in R^n$$



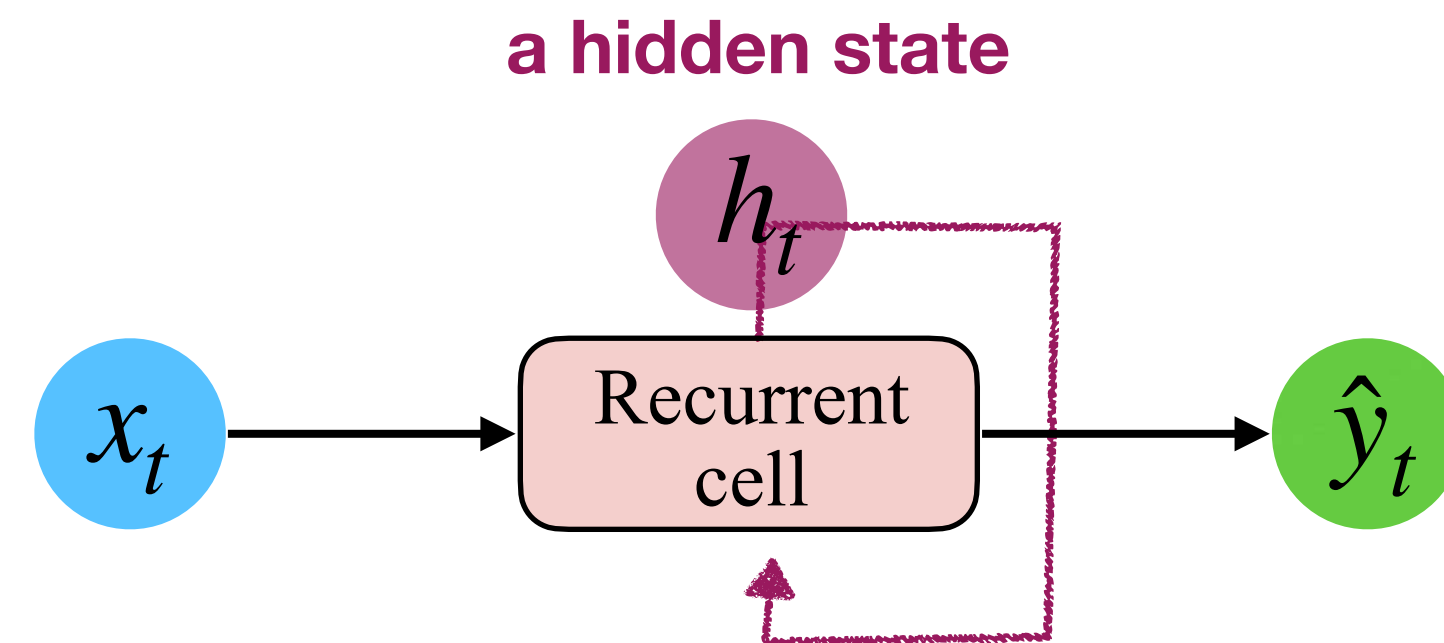
Donkey



Donkey



Donkey



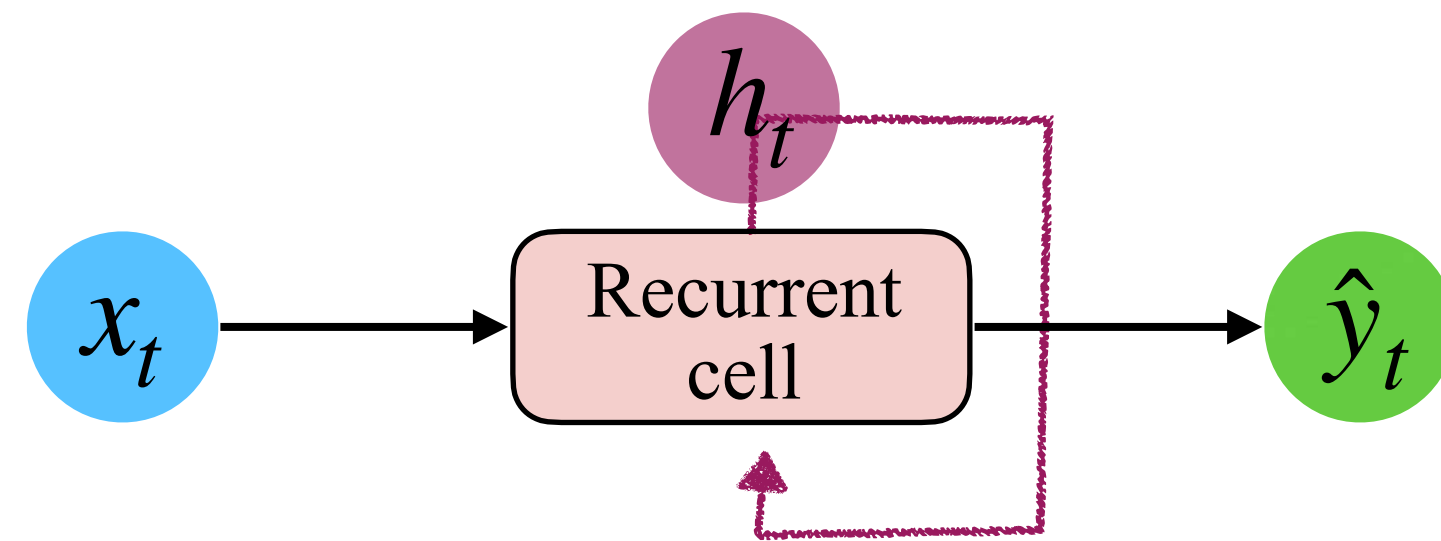
$$\hat{y}_t, h_t = f(x_t, h_{t-1})$$

$$x_t \in R^m$$

$$h_t \in R^h$$

$$\hat{y}_t \in R^n$$

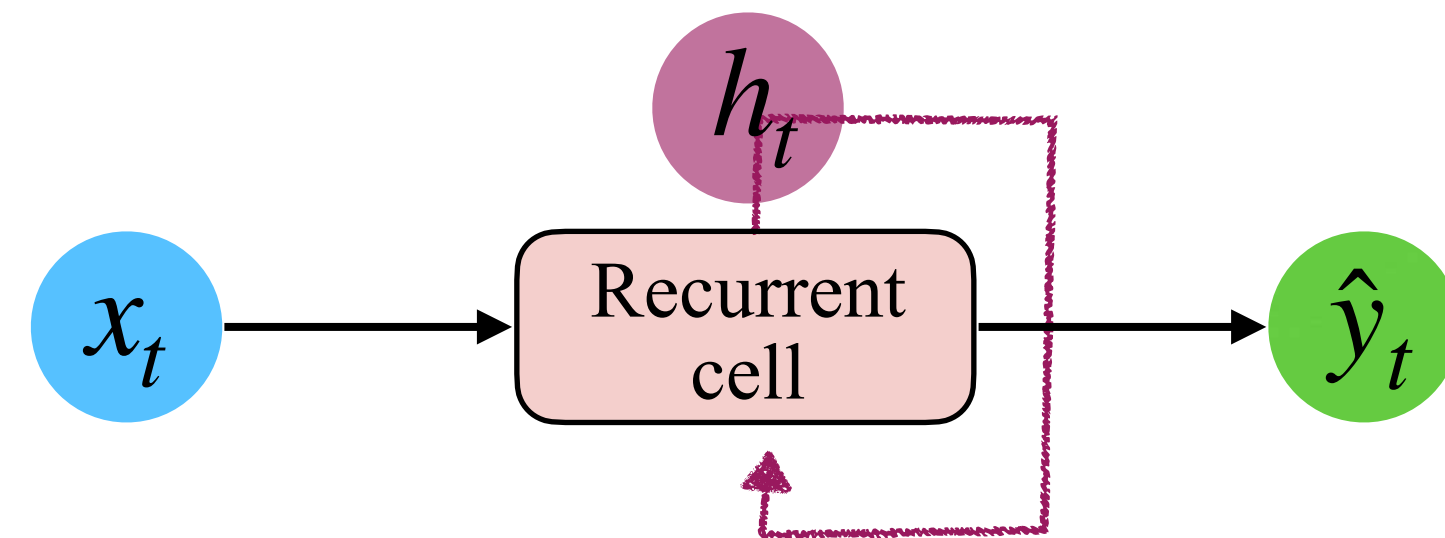
RNN State and Output Update



$$\hat{y}_t, h_t = f_W(x_t, h_{t-1})$$

Output Current state Weight Input Last state

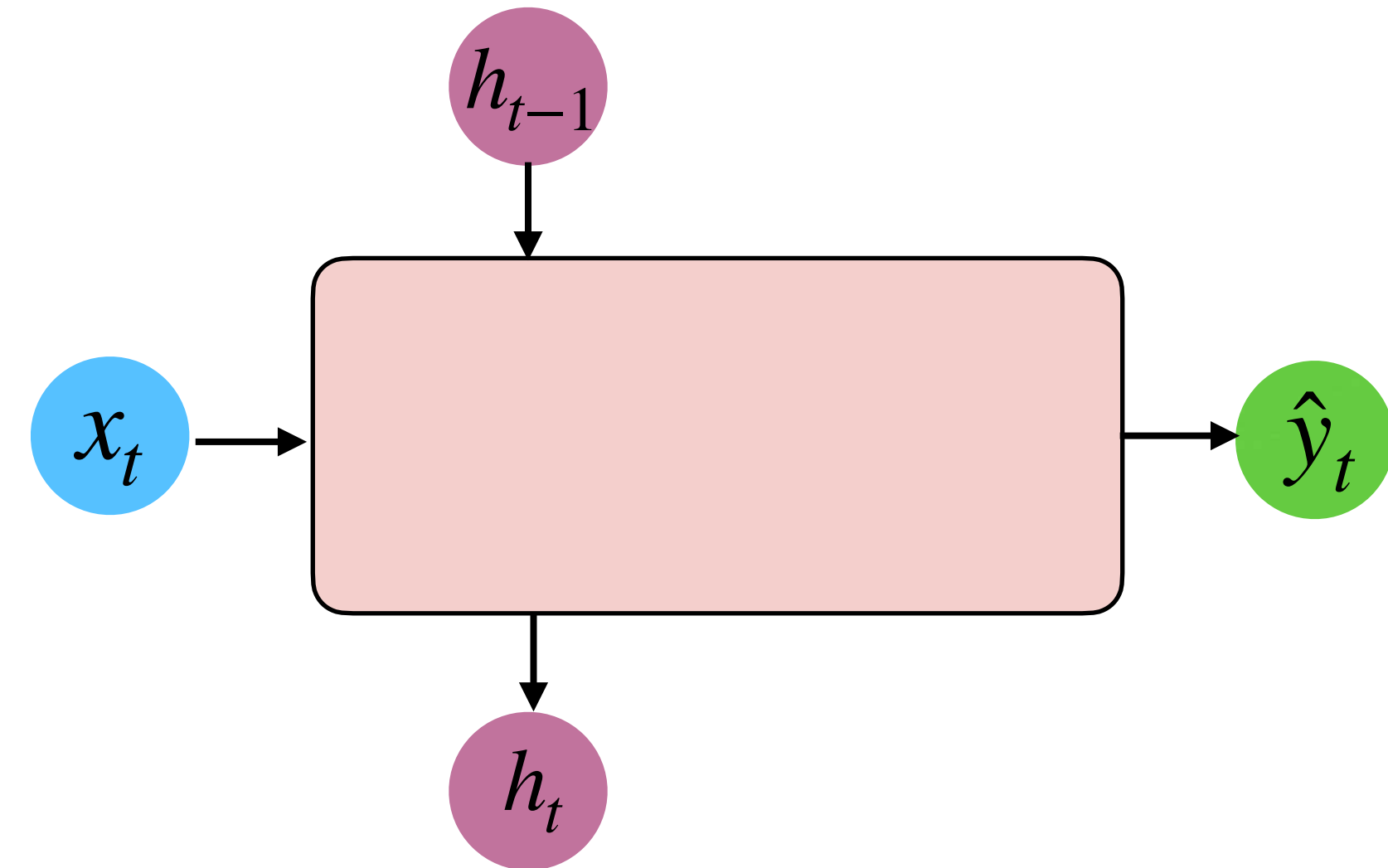
RNN State and Output Update



$$\hat{y}_t, h_t = f_W(x_t, h_{t-1})$$

Output Current state Weight Input Last state

Computation Graph

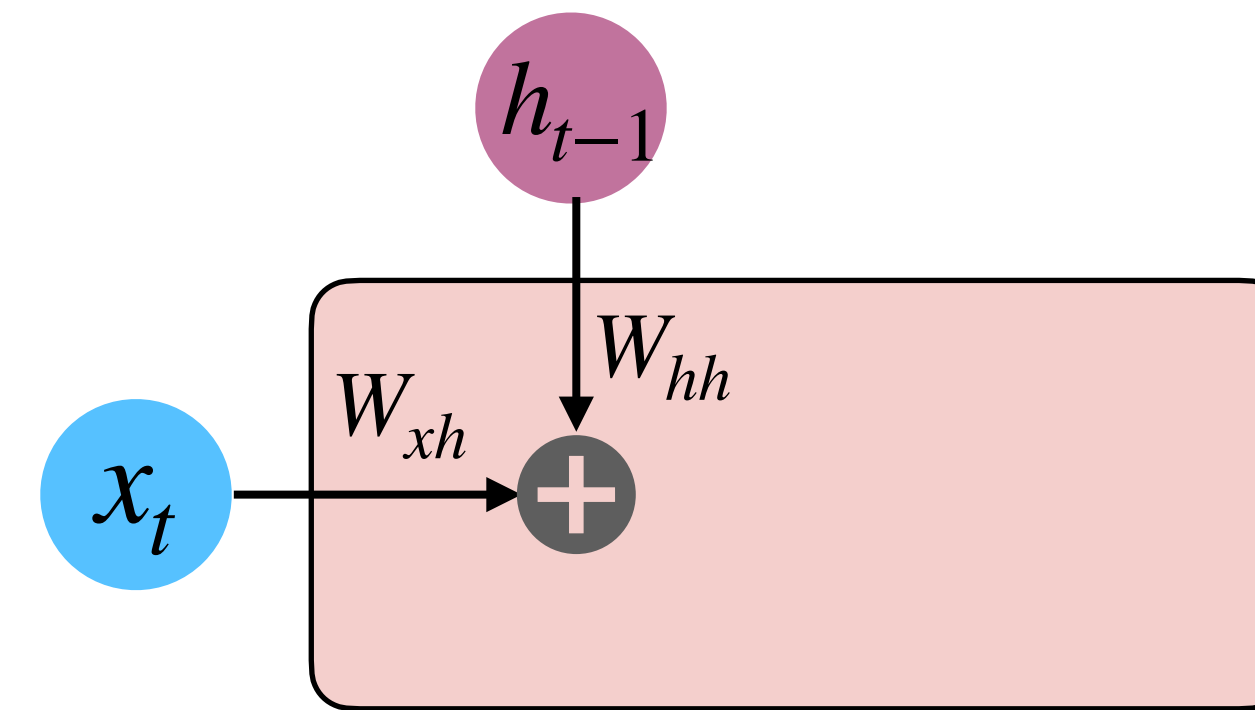
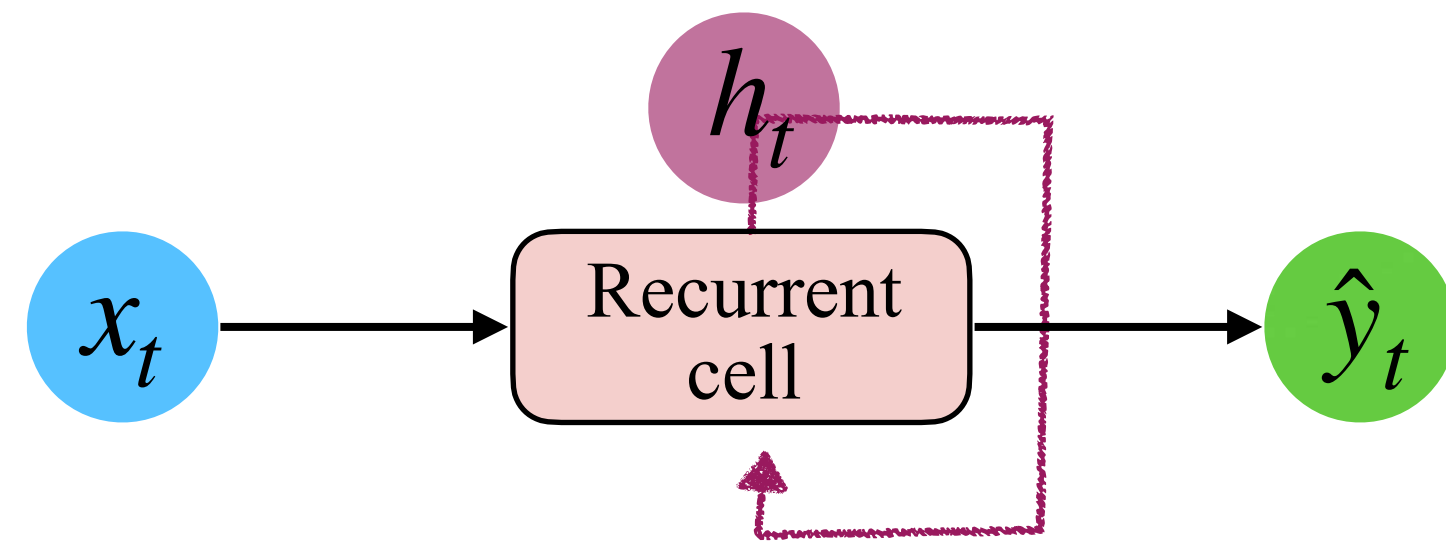


Update Hidden State

Update Output Vector

How we combine the new input with the past information?

RNN State and Output Update



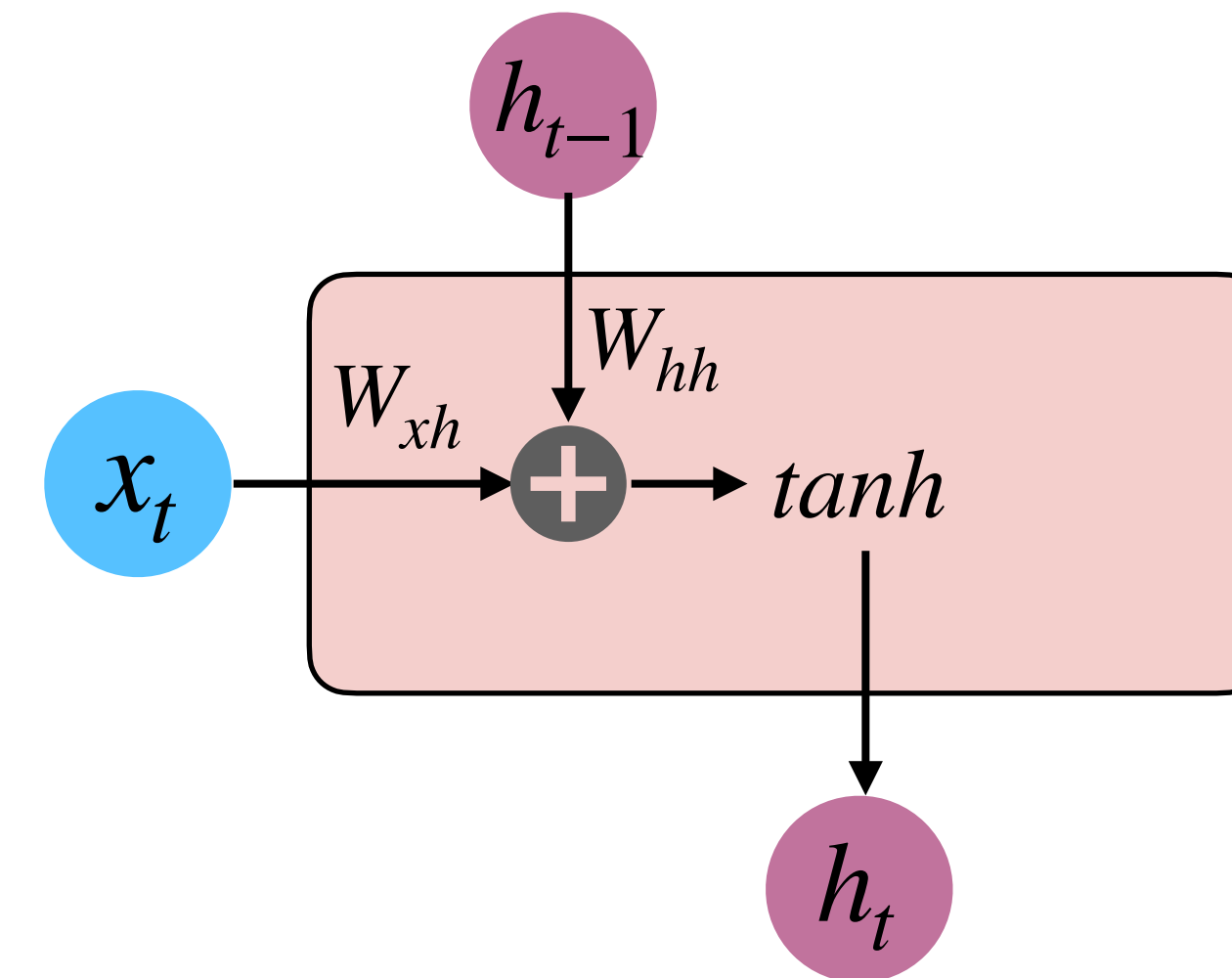
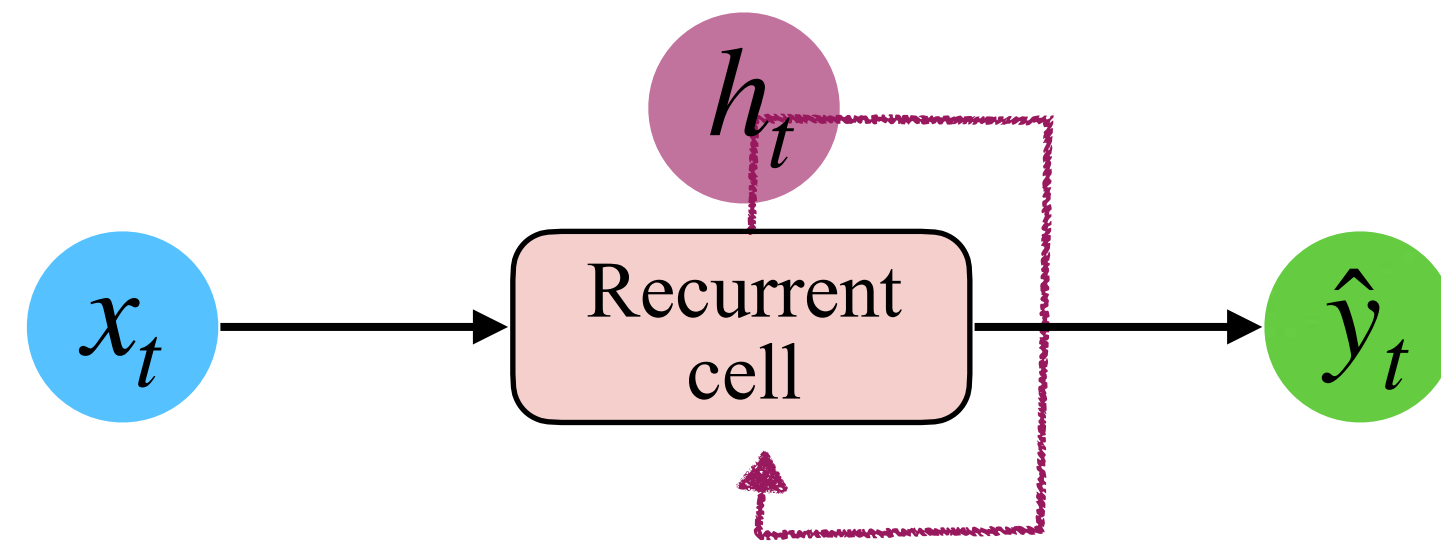
$$\hat{y}_t, h_t = f_W(x_t, h_{t-1})$$

Output Current state Weight Input Last state

$$W_{xh}x_t + W_{hh}h_{t-1}$$

Update Hidden State

RNN State and Output Update



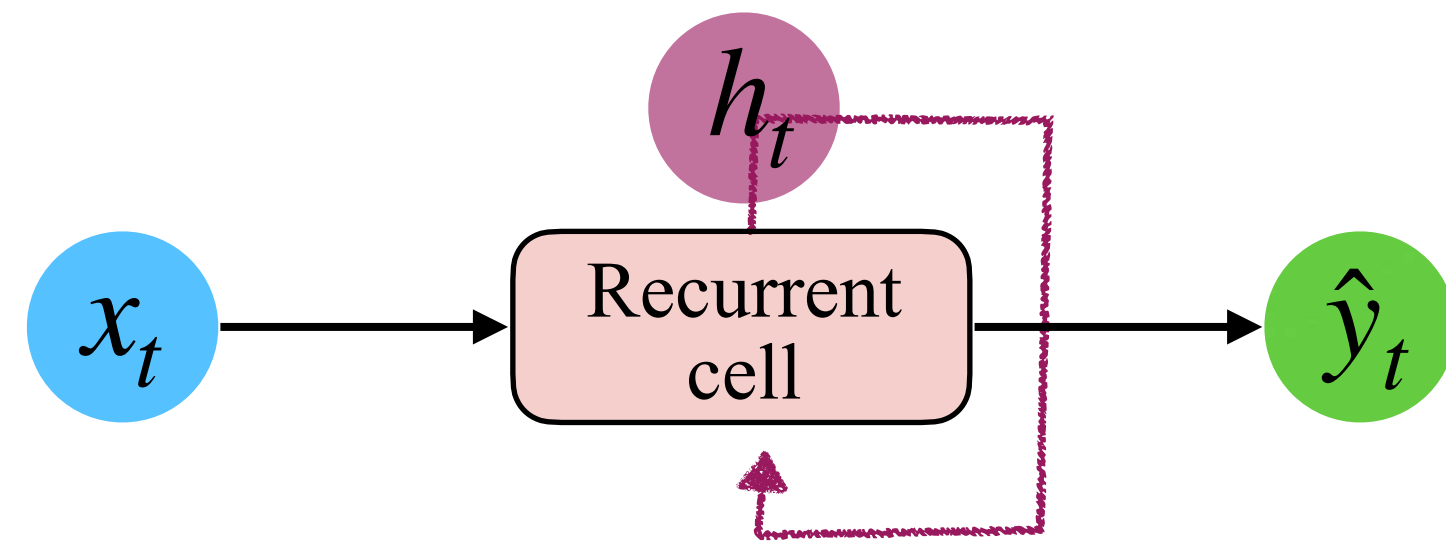
$$\hat{y}_t, h_t = f_W(x_t, h_{t-1})$$

Output Current state Weight Input Last state

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$$

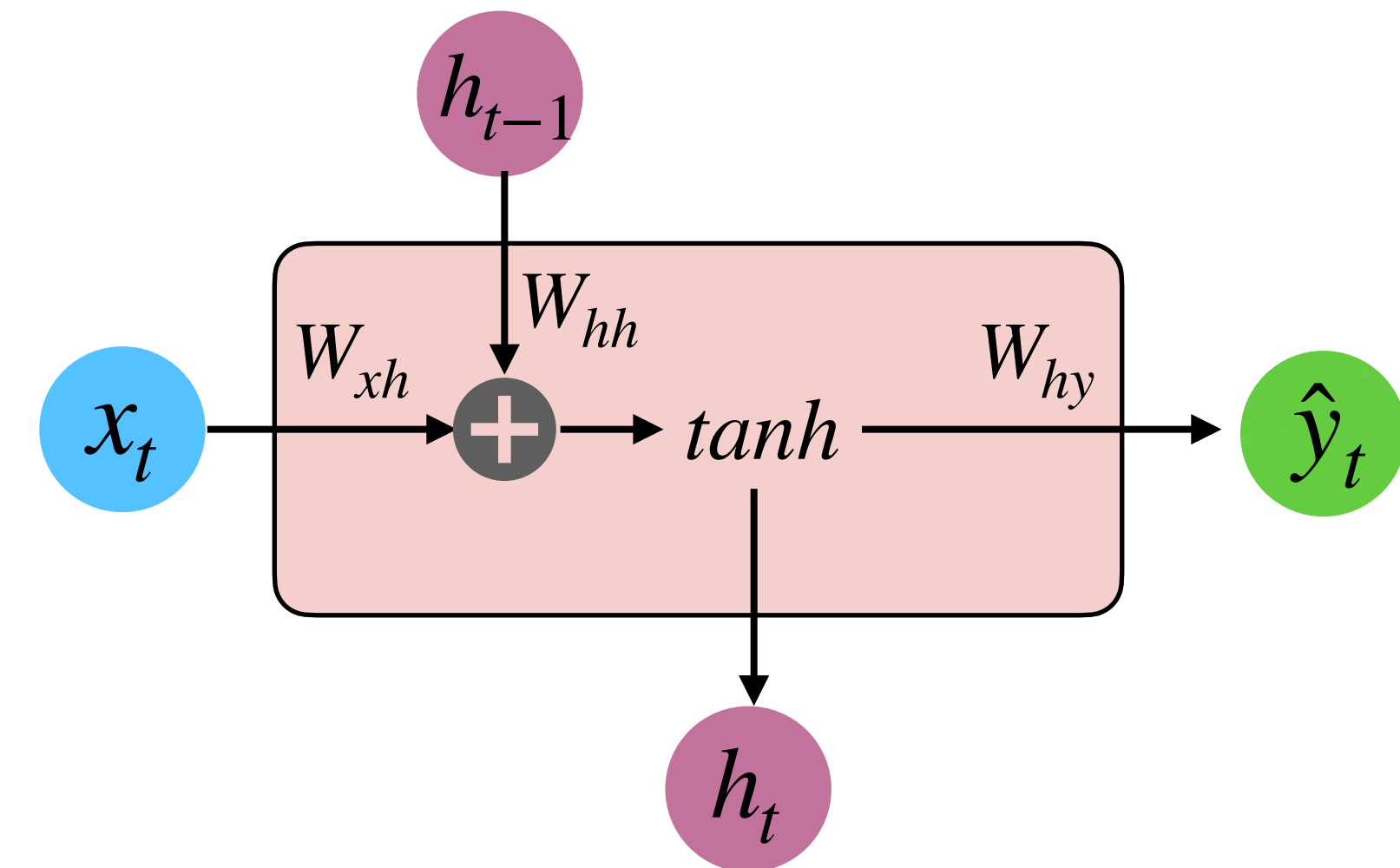
Update Hidden State

RNN State and Output Update



$$\hat{y}_t, h_t = f_W(x_t, h_{t-1})$$

Output Current state Weight Input Last state



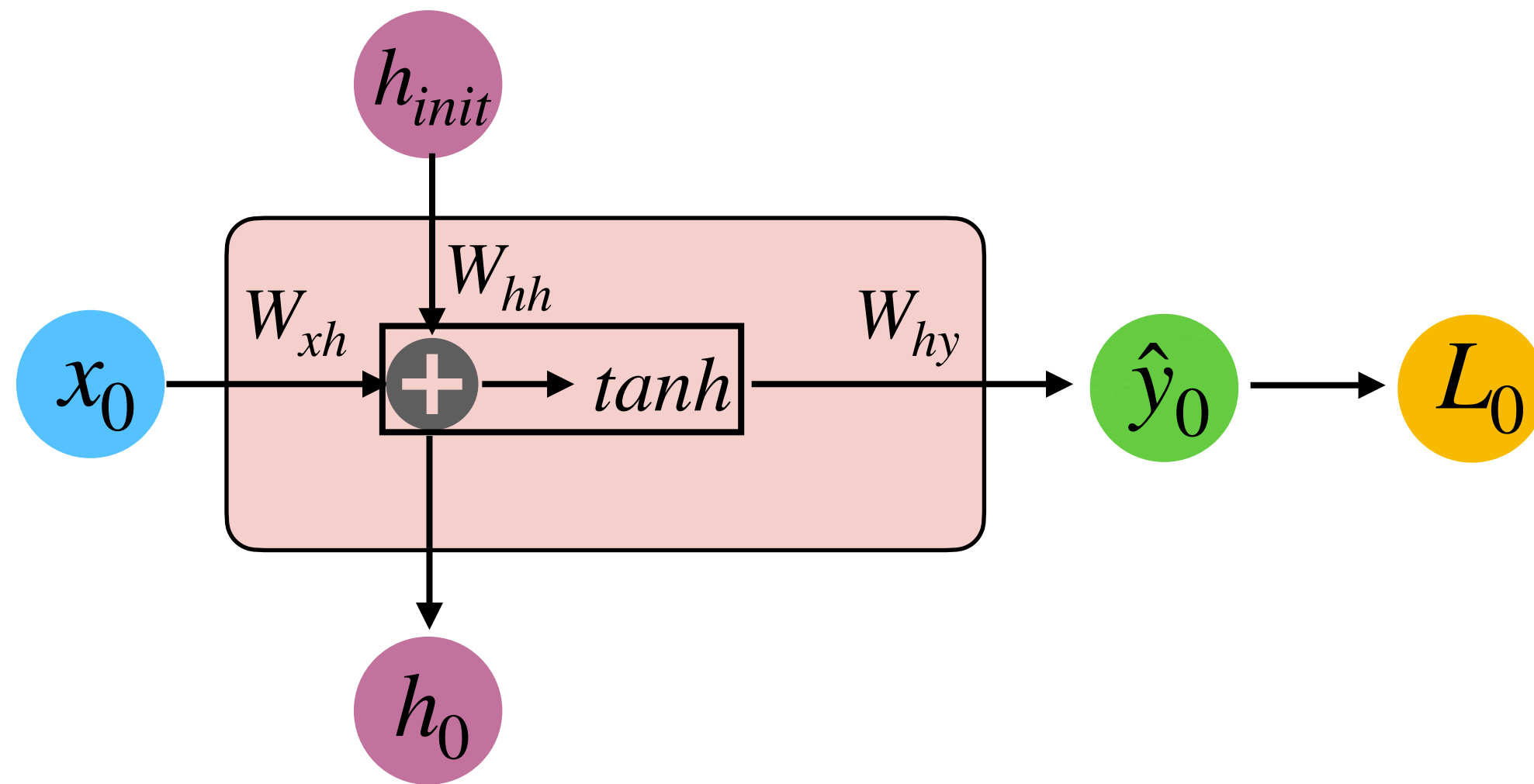
$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$$

Update Hidden State

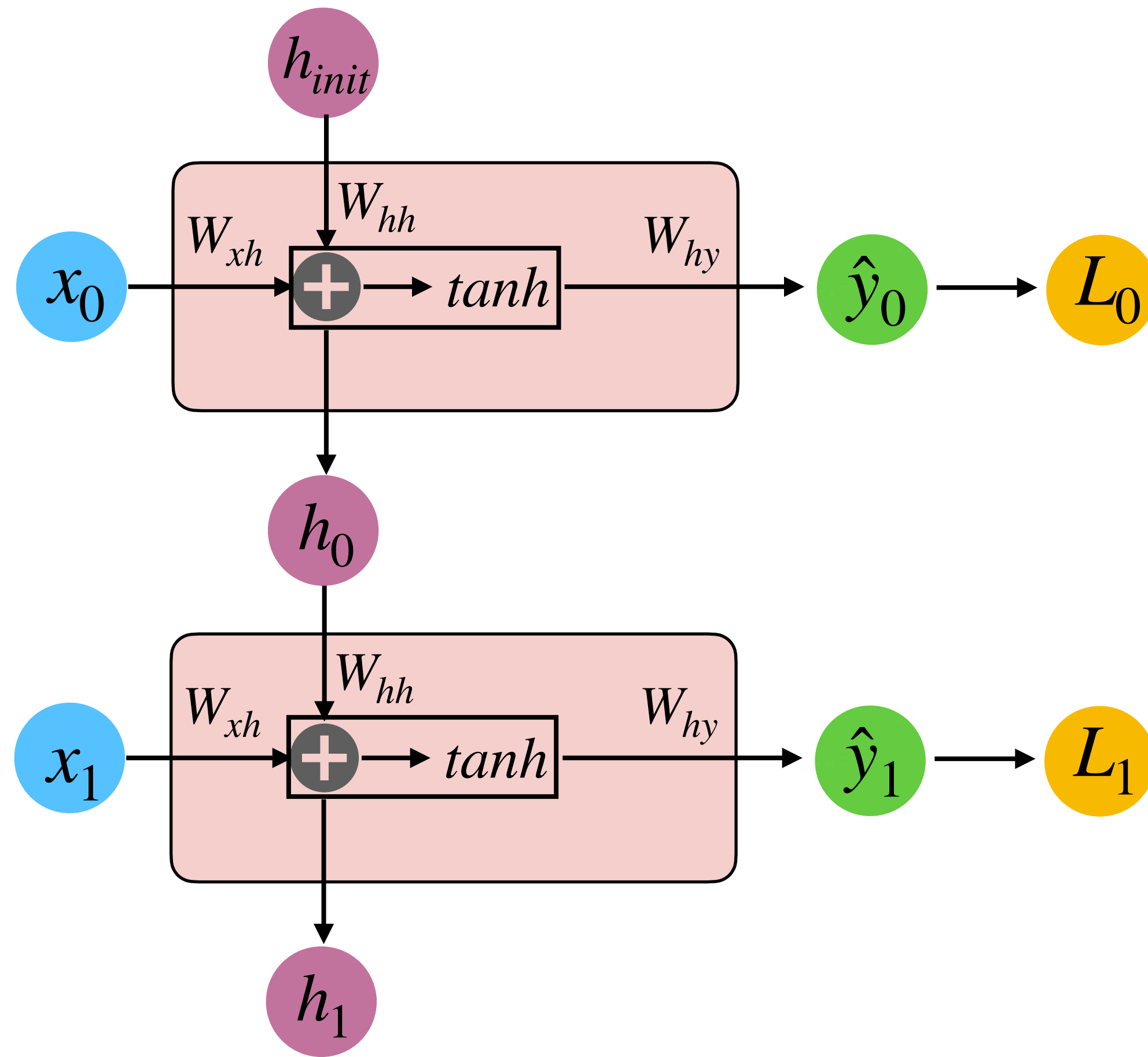
$$\hat{y}_t = W_{hy}^T h_t$$

Update Output Vector

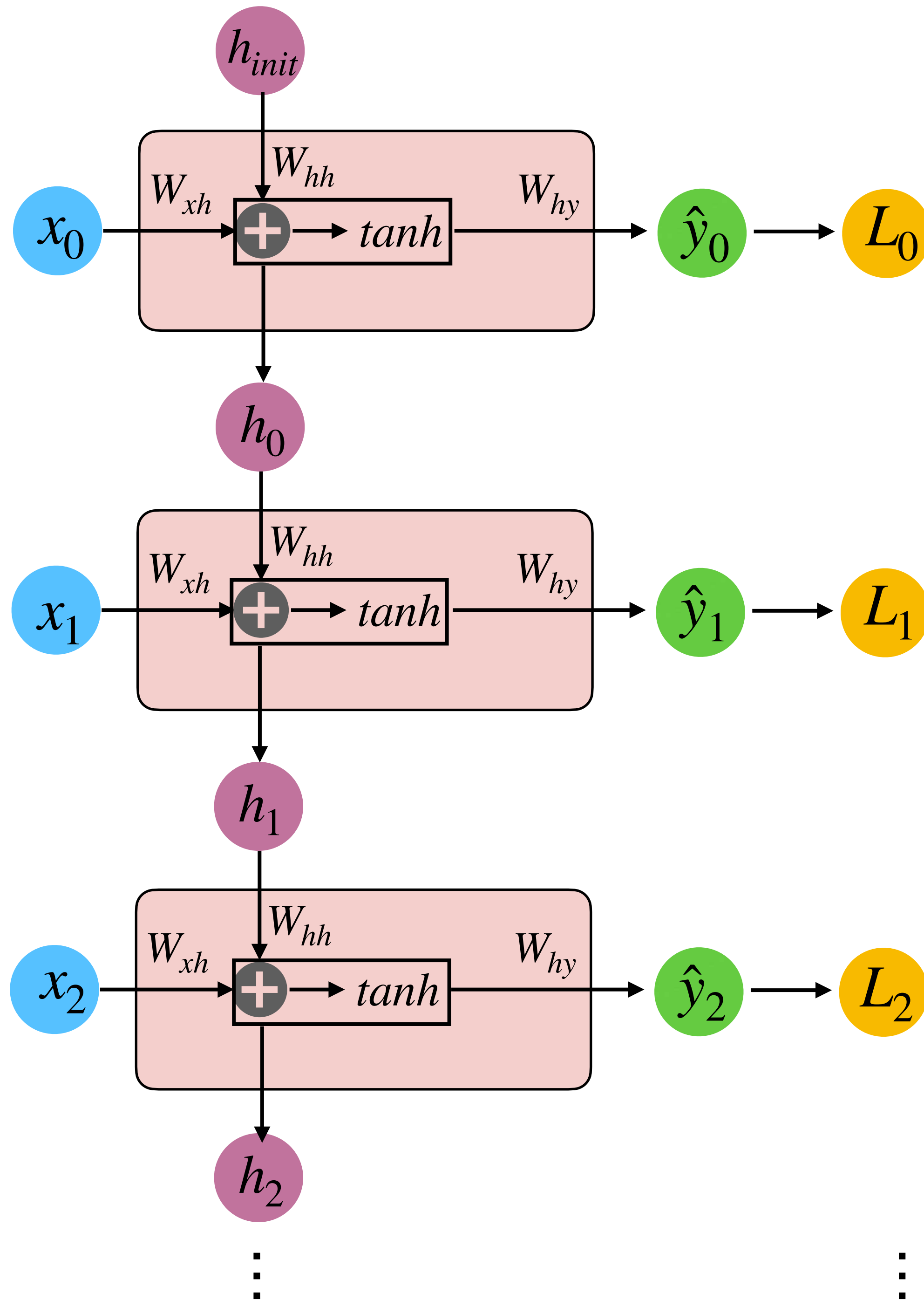
RNNs: Computational Graph Across Time



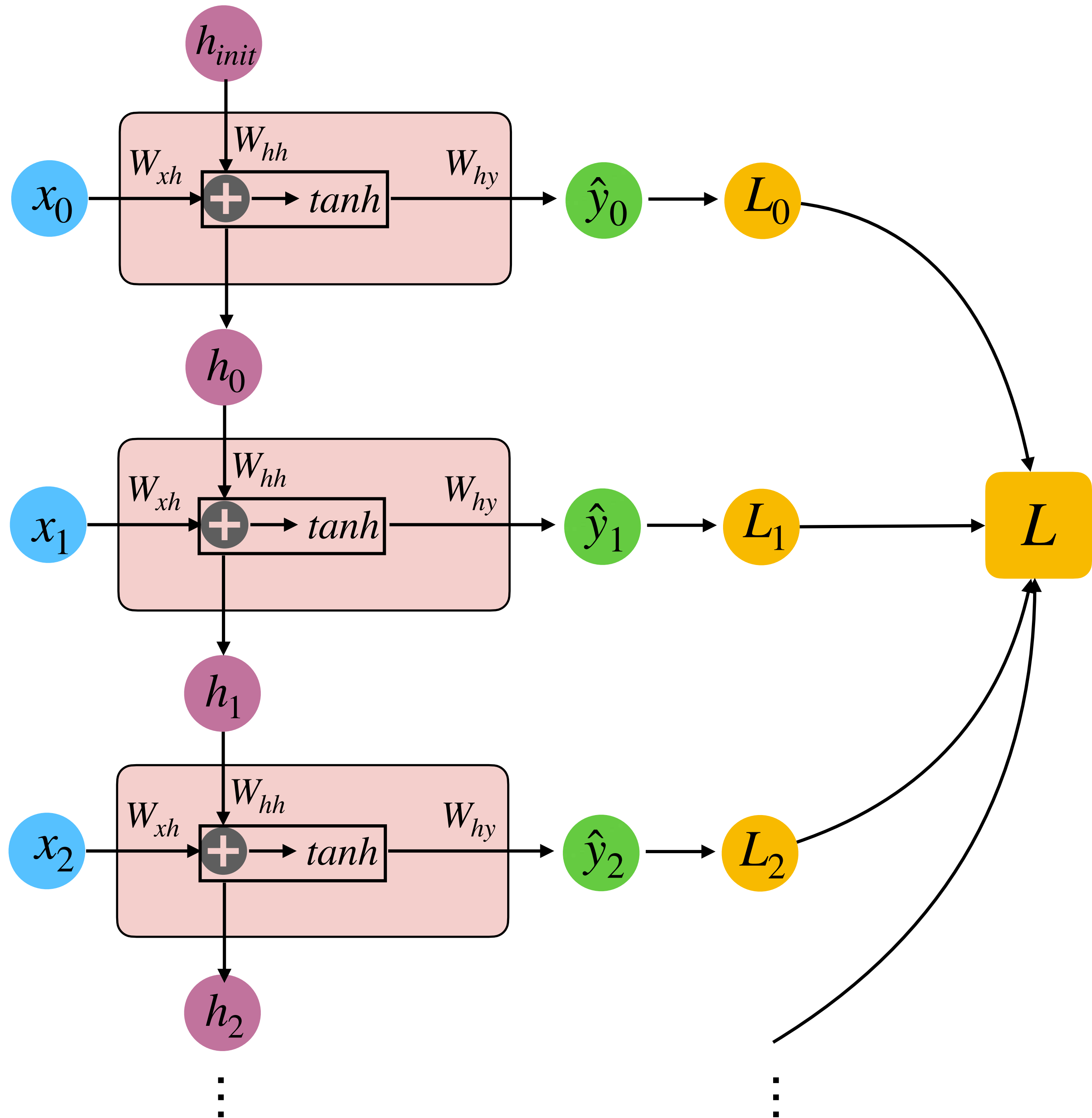
RNNs: Computational Graph Across Time



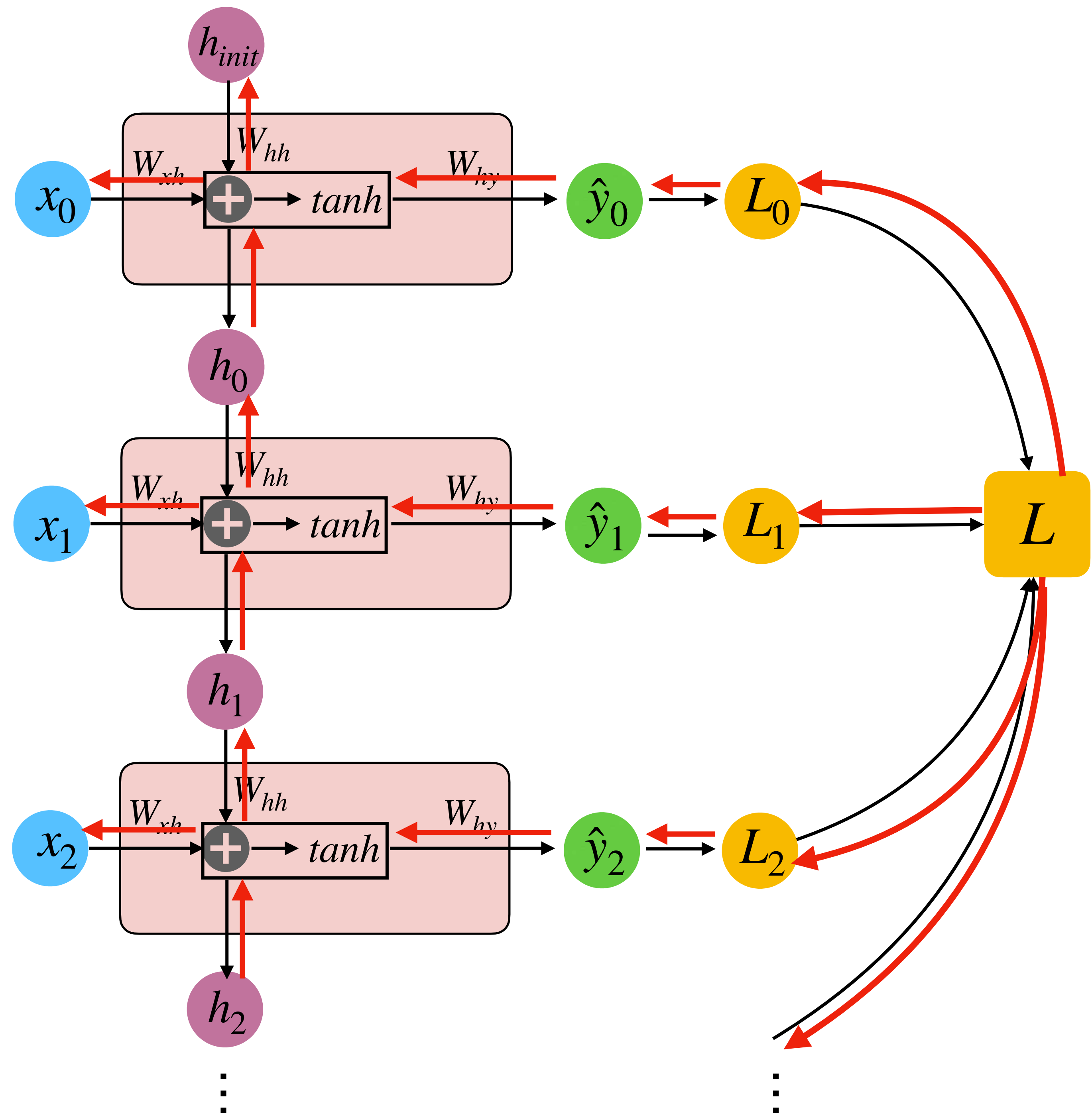
RNNs: Computational Graph Across Time



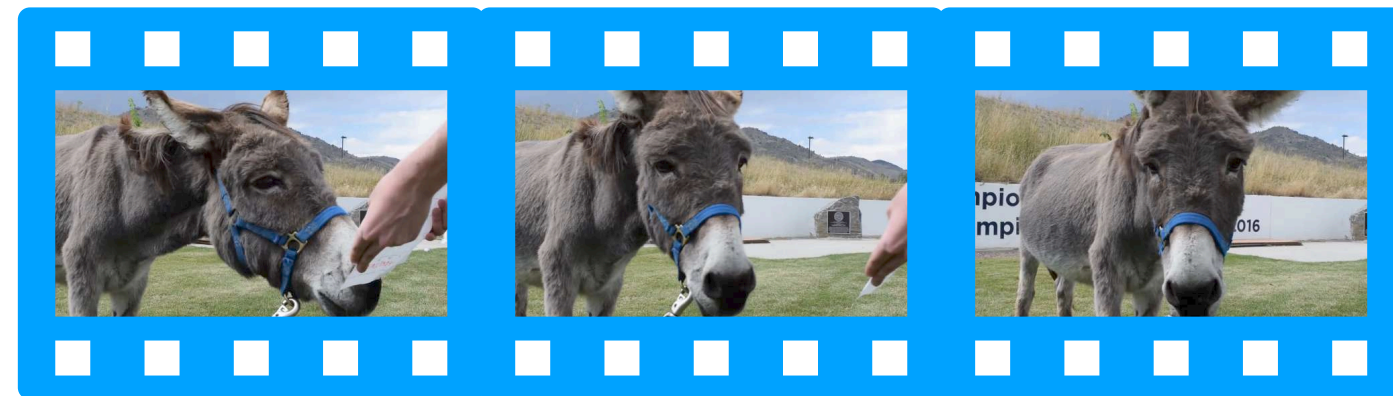
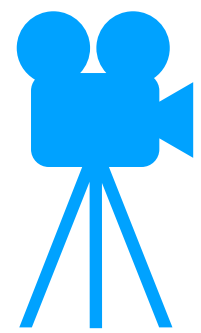
RNNs: Computational Graph Across Time



RNNs: Backpropagation Through Time



Common types of sequential data



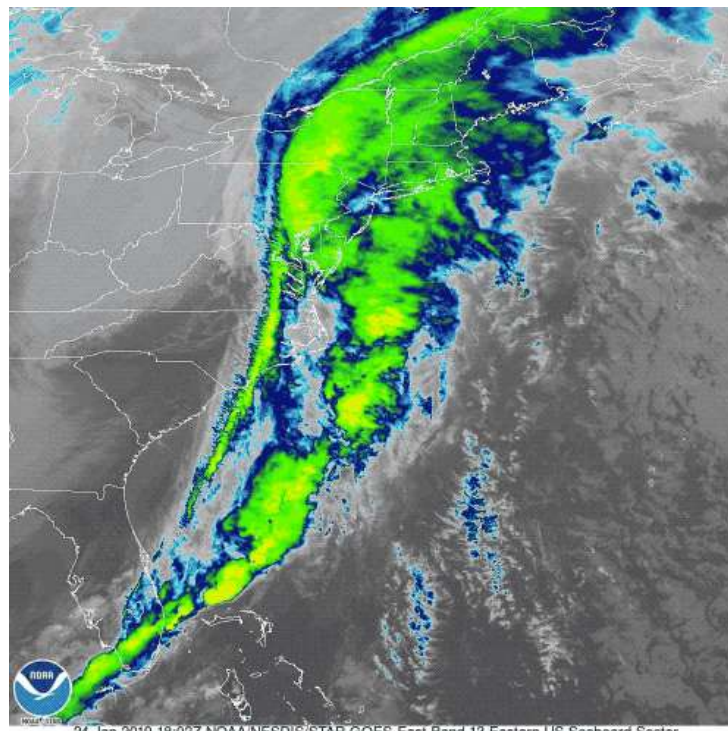
Video: *sequences of words, tokens*

I love this class!

Language/text
sequences of words, tokens



Audio/speech
sequences of waveform samples

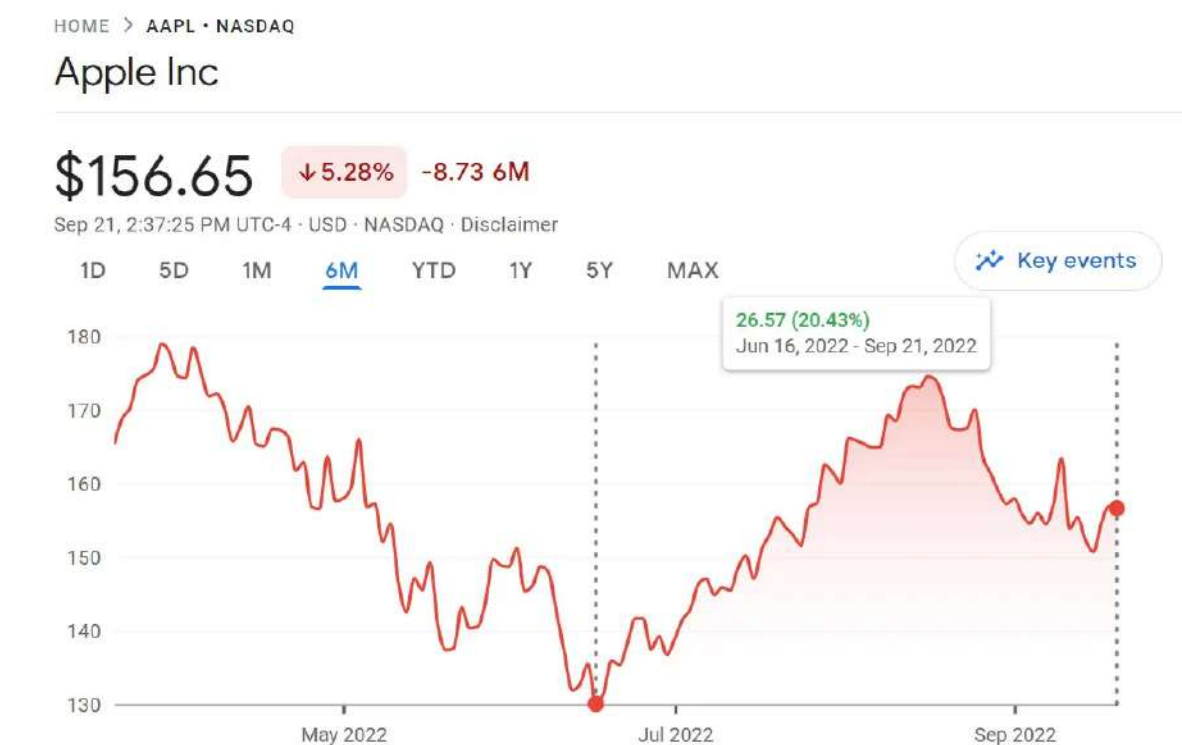


Weather and climate data:
temperature, precipitation, wind

<https://www.sciencefriday.com/segments/100-years-of-your-daily-weather-forecast/>



DNA sequence
sequences of nucleotide bases



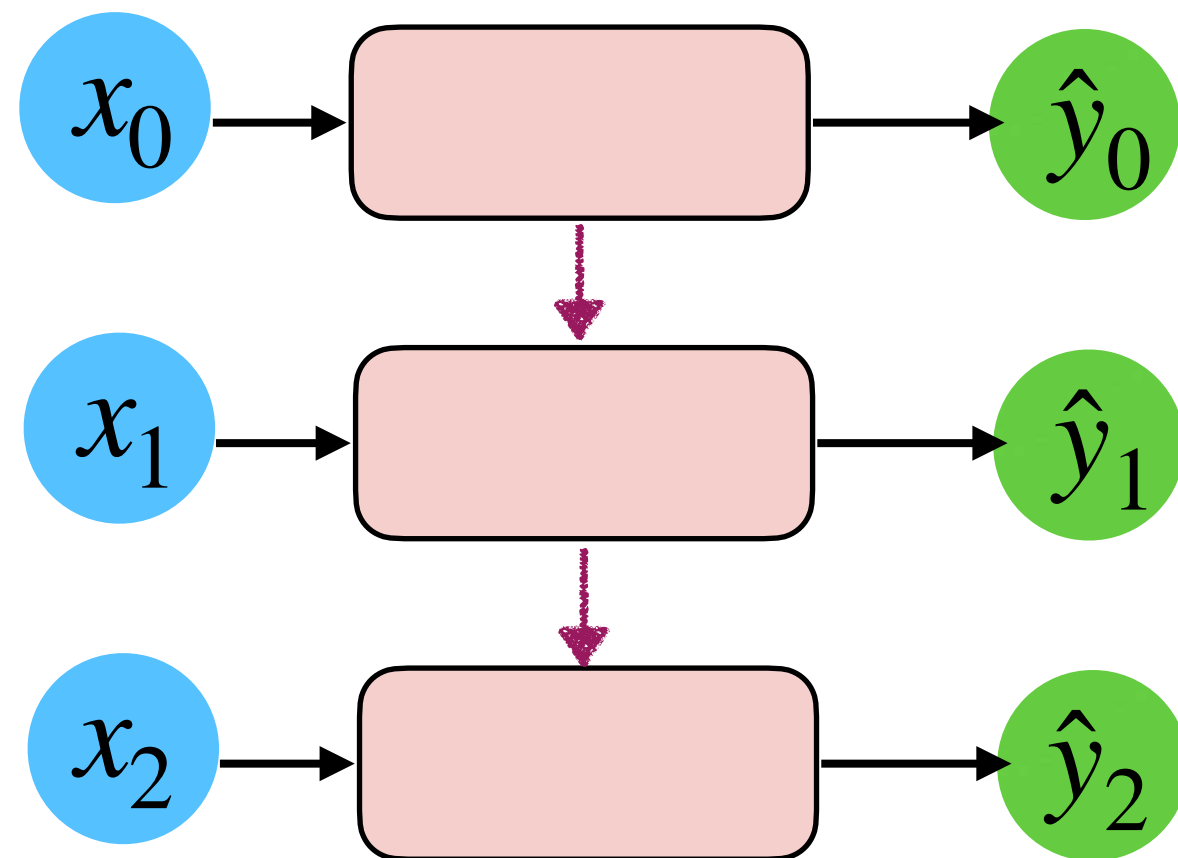
Finance
sequences of prices

The order matters!

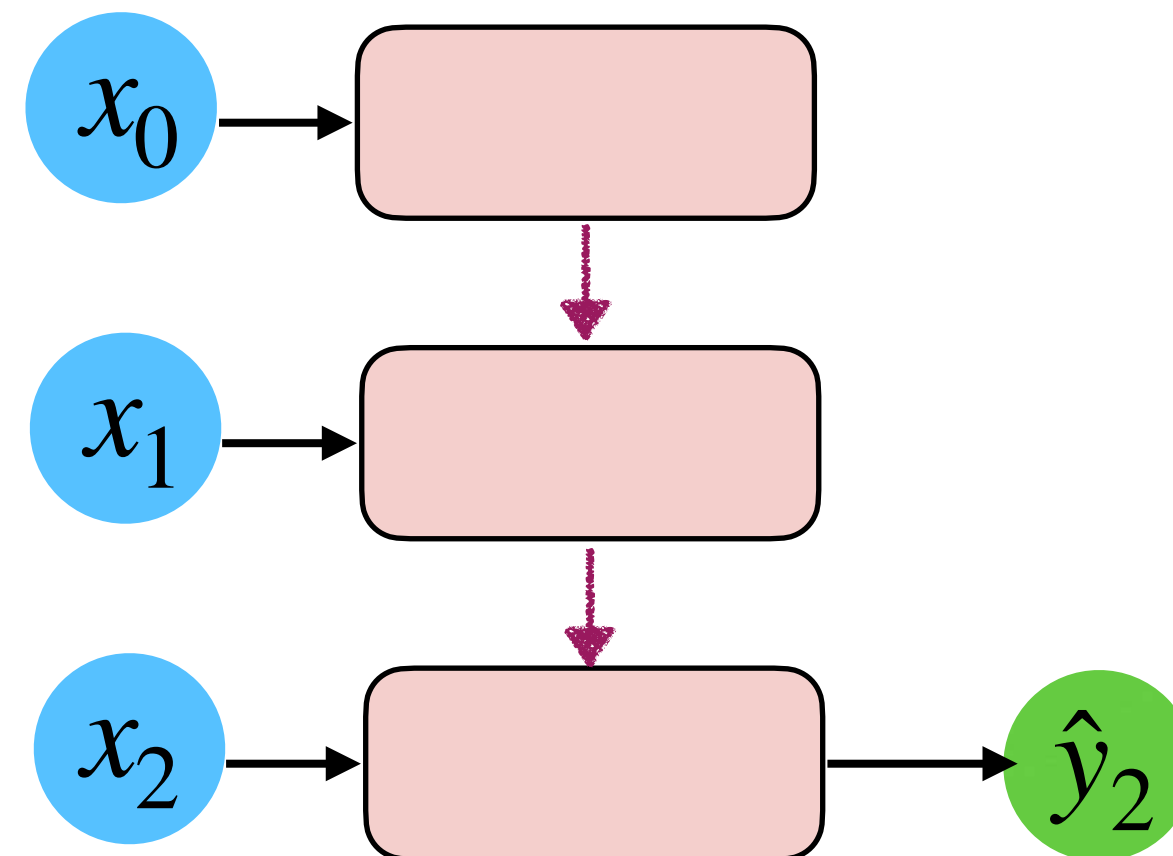
RNN Variants for Sequence Modeling



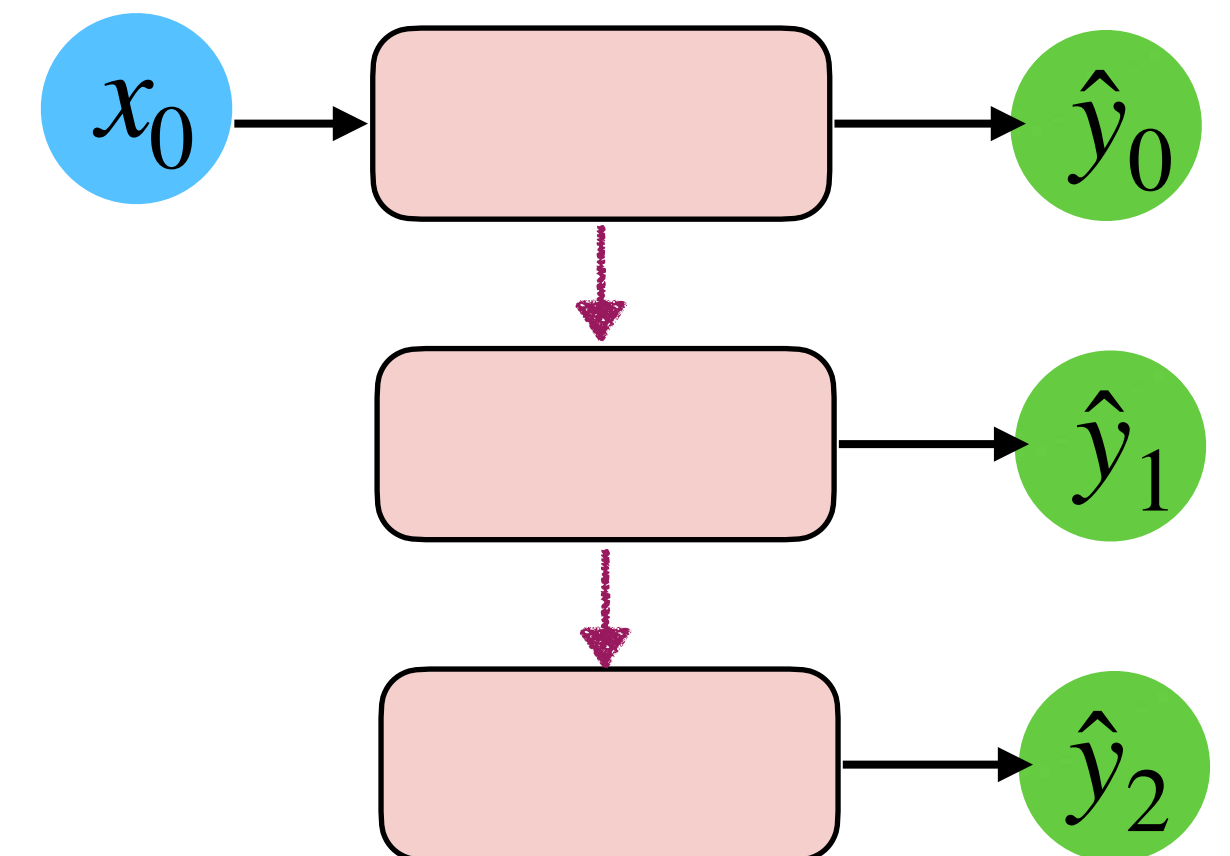
One to One “Vanilla” NN



Many to Many
Translation & Forecasting & Music
Generation



Many to One
Sentiment Classification



One to Many
Text Generation & Video Generation

RNNs from Scratch in PyTorch

T: sequence length, D: embedding size

H: hidden size, V: vocabulary size

tokens: (T,)

x = embedding(tokens) # (T, D)

h = zeros(H) # initial hidden state

outputs = []

for t in range(T):

 current_input = x[t] # (D,)

Update the hidden state.

 h = tanh(current_input @ W_x + h @ W_h + b_h) # (H,)

Predict the next token

 scores = h @ W_y + b_y # (V,)

 outputs.append(scores)

logits = stack(outputs, axis=0) # (T, V)

Language Models: Tokenization

Visual Data



What we see

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

What a computer sees

Numerical Values!

Text Data

I love this class!



?

Symbols

Values

Language Models: Tokenization

Visual Data



What we see

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

What a computer sees

Numerical Values!

Text Data

I love this class!

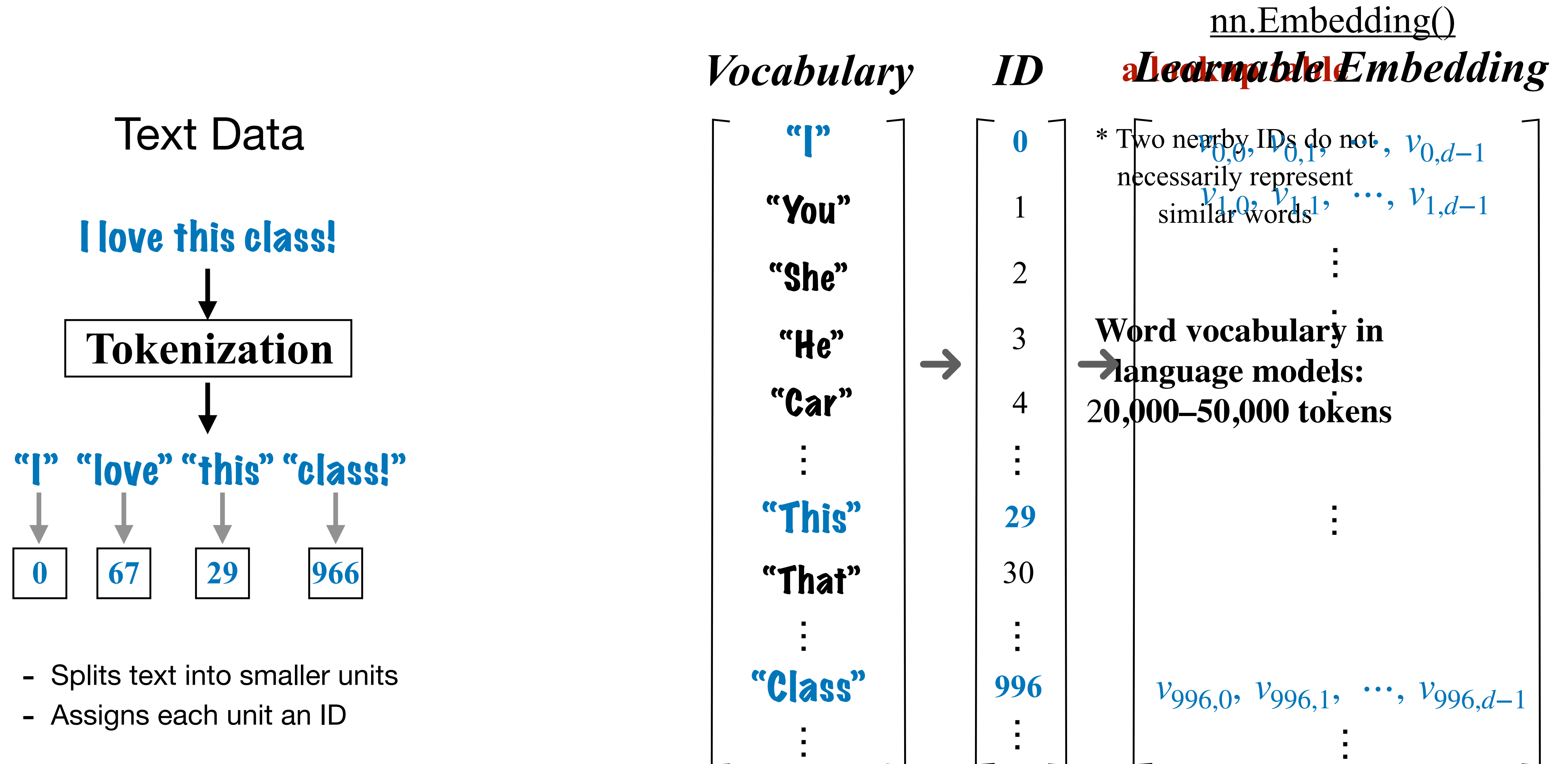


Numerical Values!

3 0 29 966

How? Tokenization!

Language Models: Tokenization



Language Models: Tokenization Types

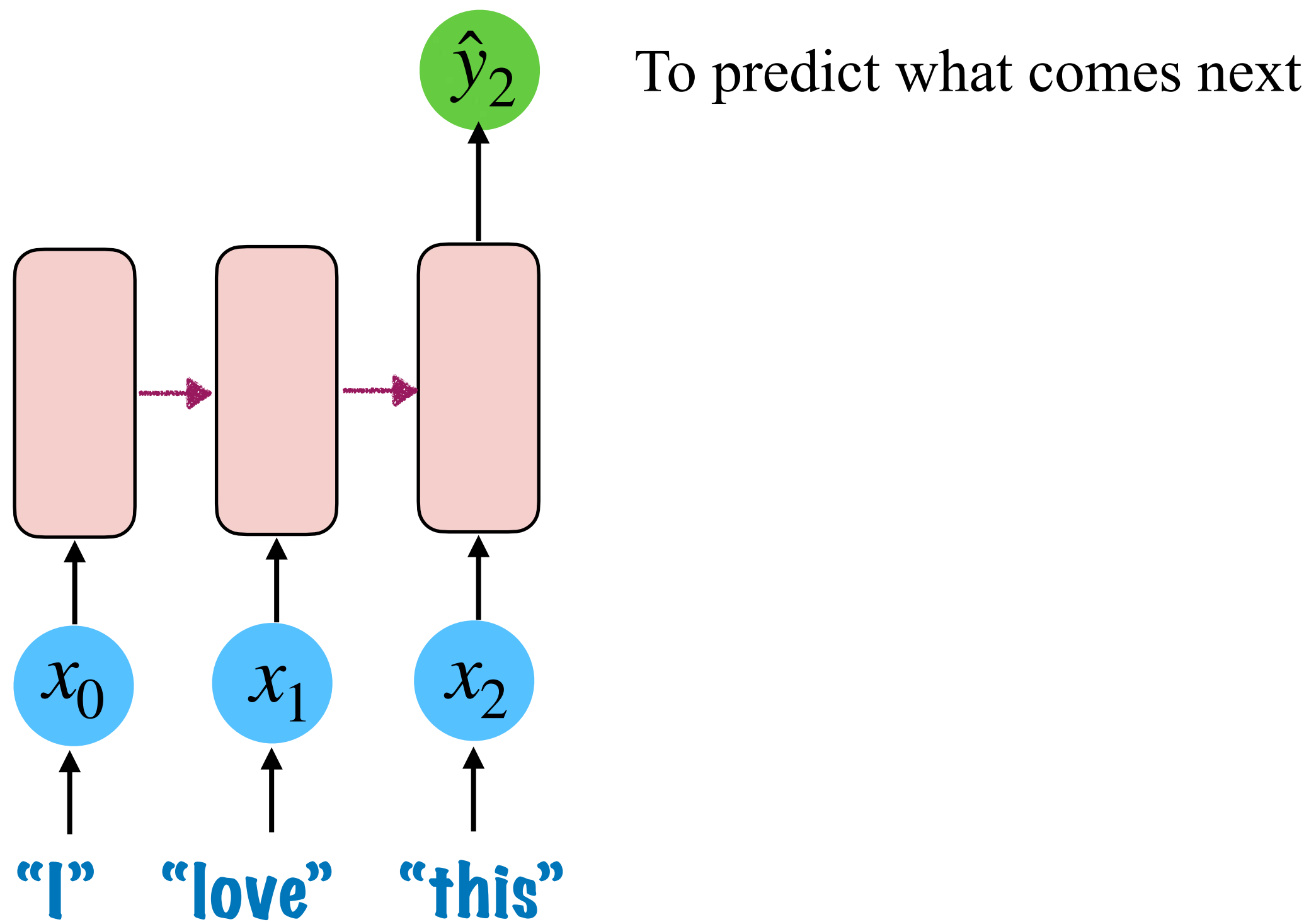
- Splits text into smaller units: trade-offs in vocabulary size and sequence length

Tokenization type	Resulting tokens
Sentence-level	<code>["I love this class!"]</code>
Whitespace/word-level	<code>["I", "love", "this", "class!"]</code>
Word + punctuation	<code>["I", "love", "this", "class", "!"]</code>
Character-level	<code>["I", " ", "l", "o", "v", "e", " ", "t", "h", "i", "s", " ", "c", "l", "a", "s", "s", "!"]</code>
Subword-level	<code>["I", "love", "the", "class", "!"]</code> or <code>["I", "lov", "e", "the", "class", "!"]</code>
 Byte-level	<code>[73, 32, 108, 111, 118, 101, 32, 116, 104, 101, 32, 99, 108, 97, 115, 115, 33]</code>

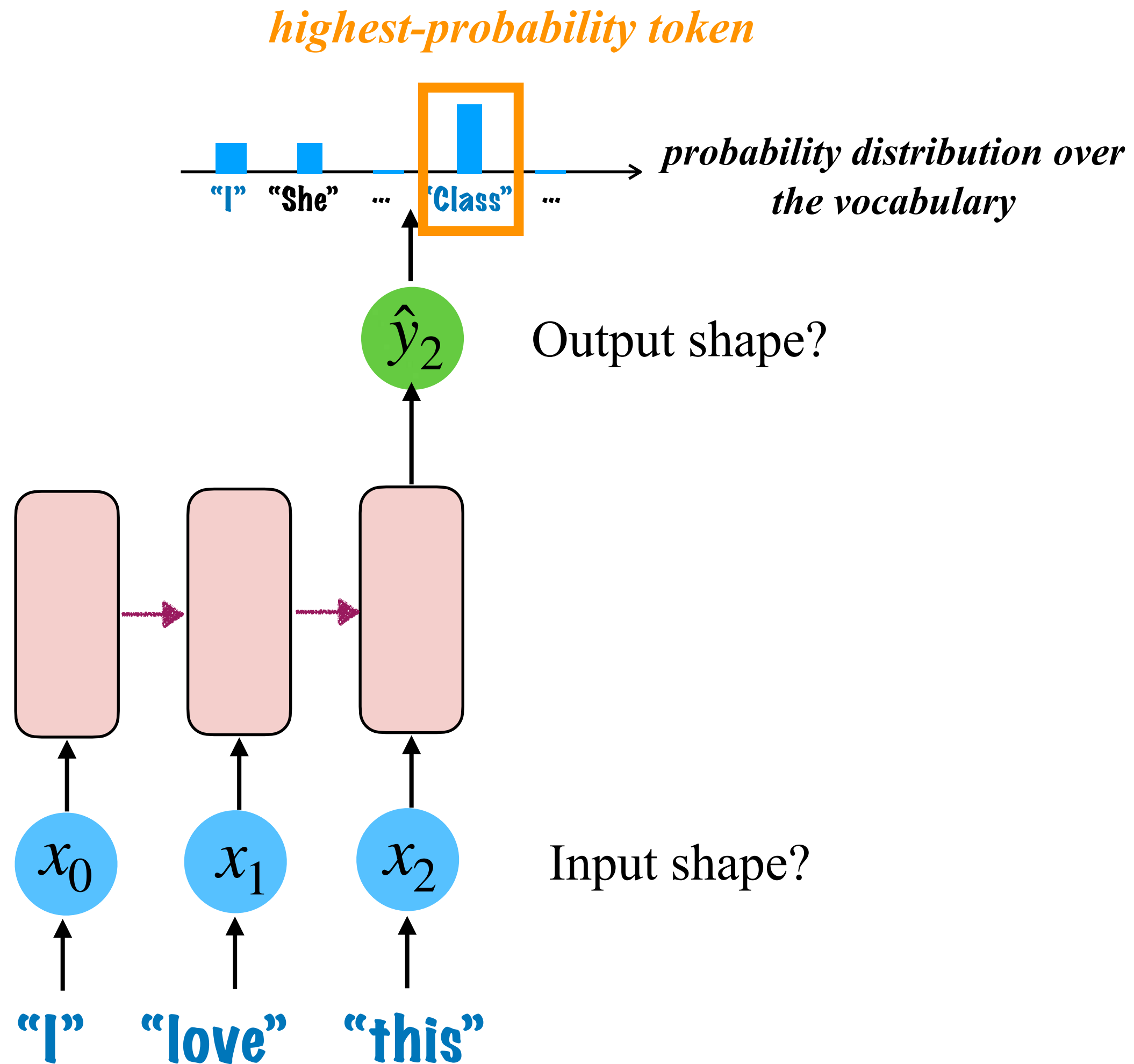
Byte Pair Encoding (BPE): Repeatedly merge the most frequent adjacent pair of tokens into a new token.

low → “low”
lower → “low” “er”
lowest → “low” “est”

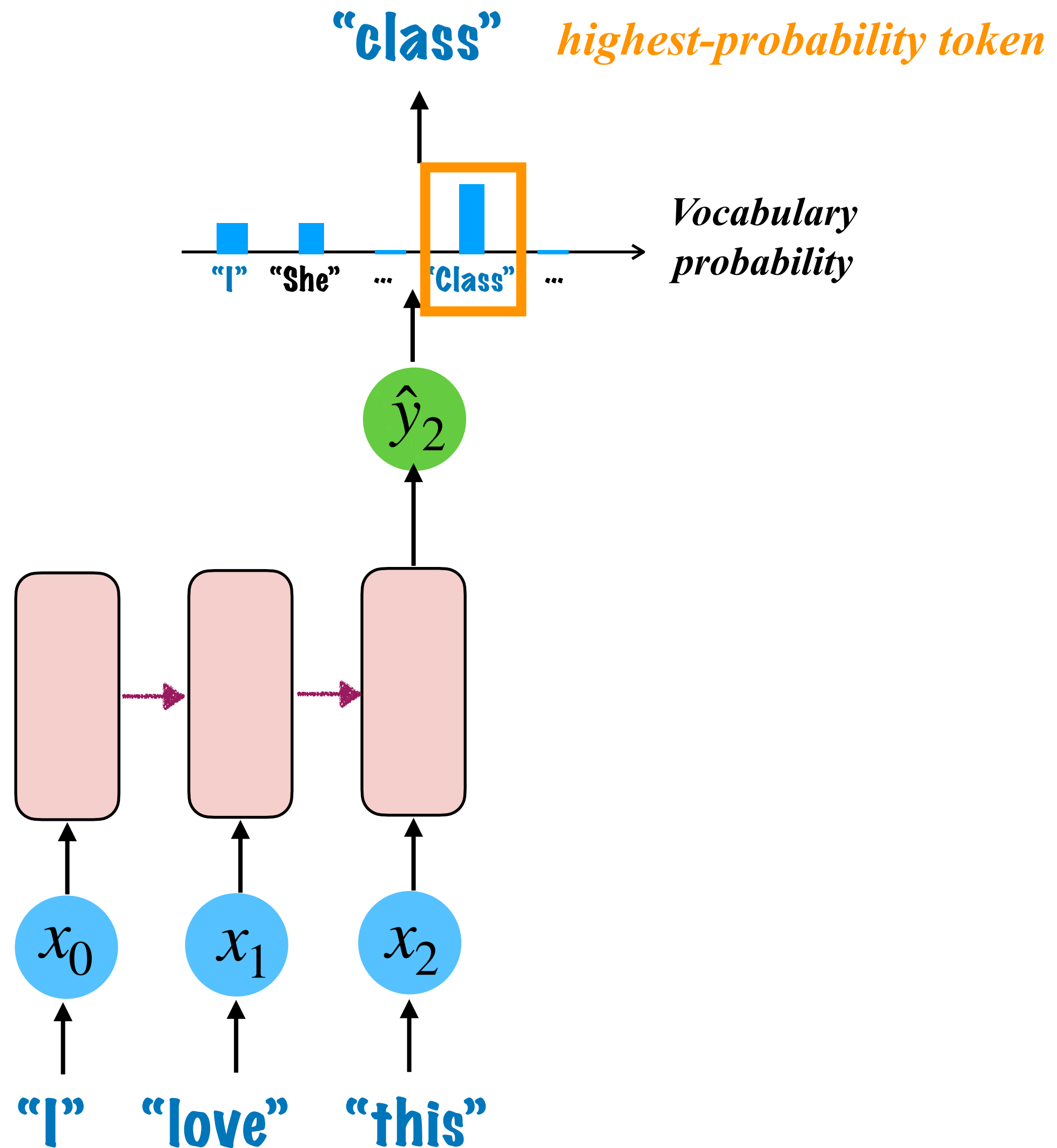
Language Models: Next token Prediction



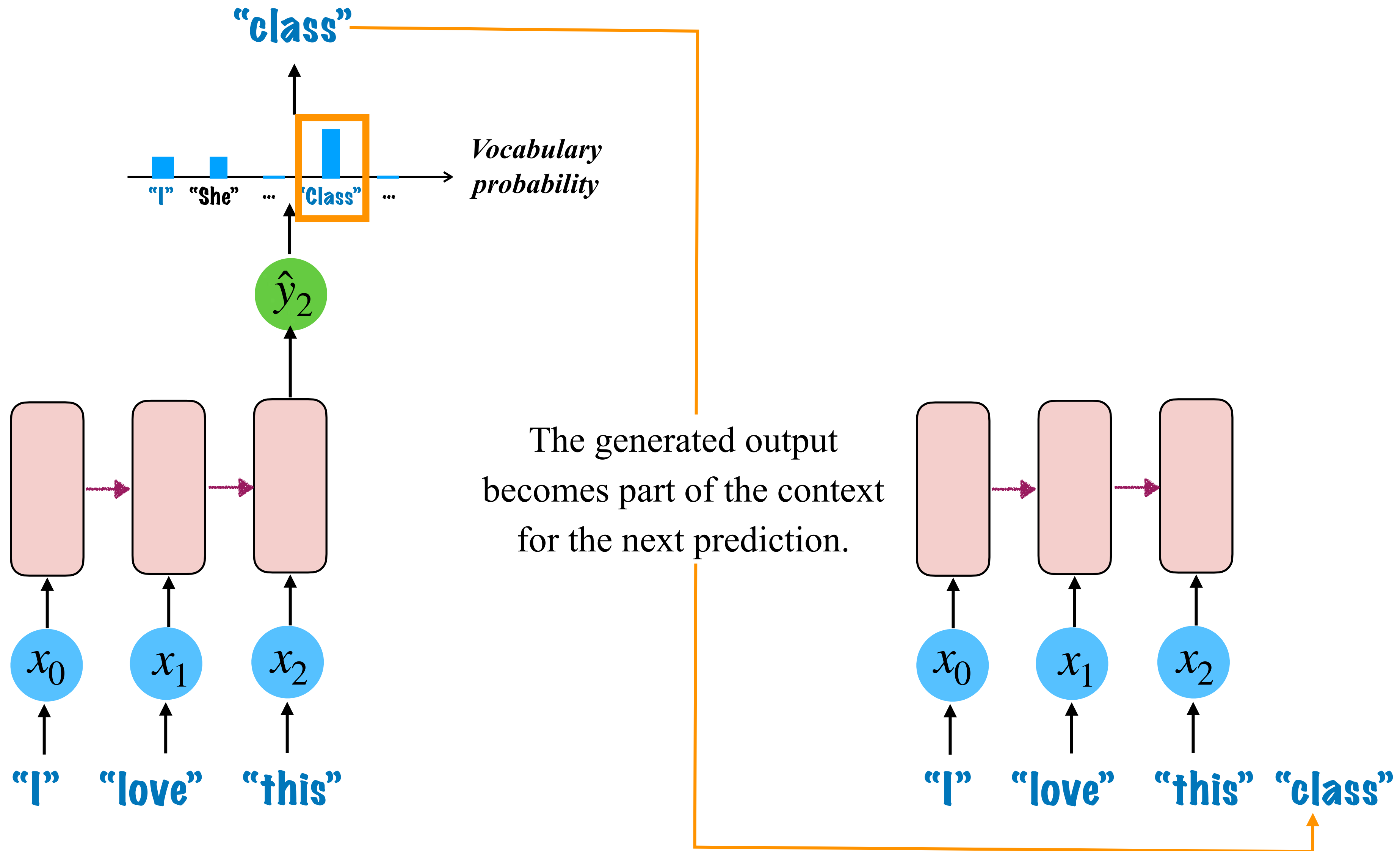
Language Models: Next token Prediction



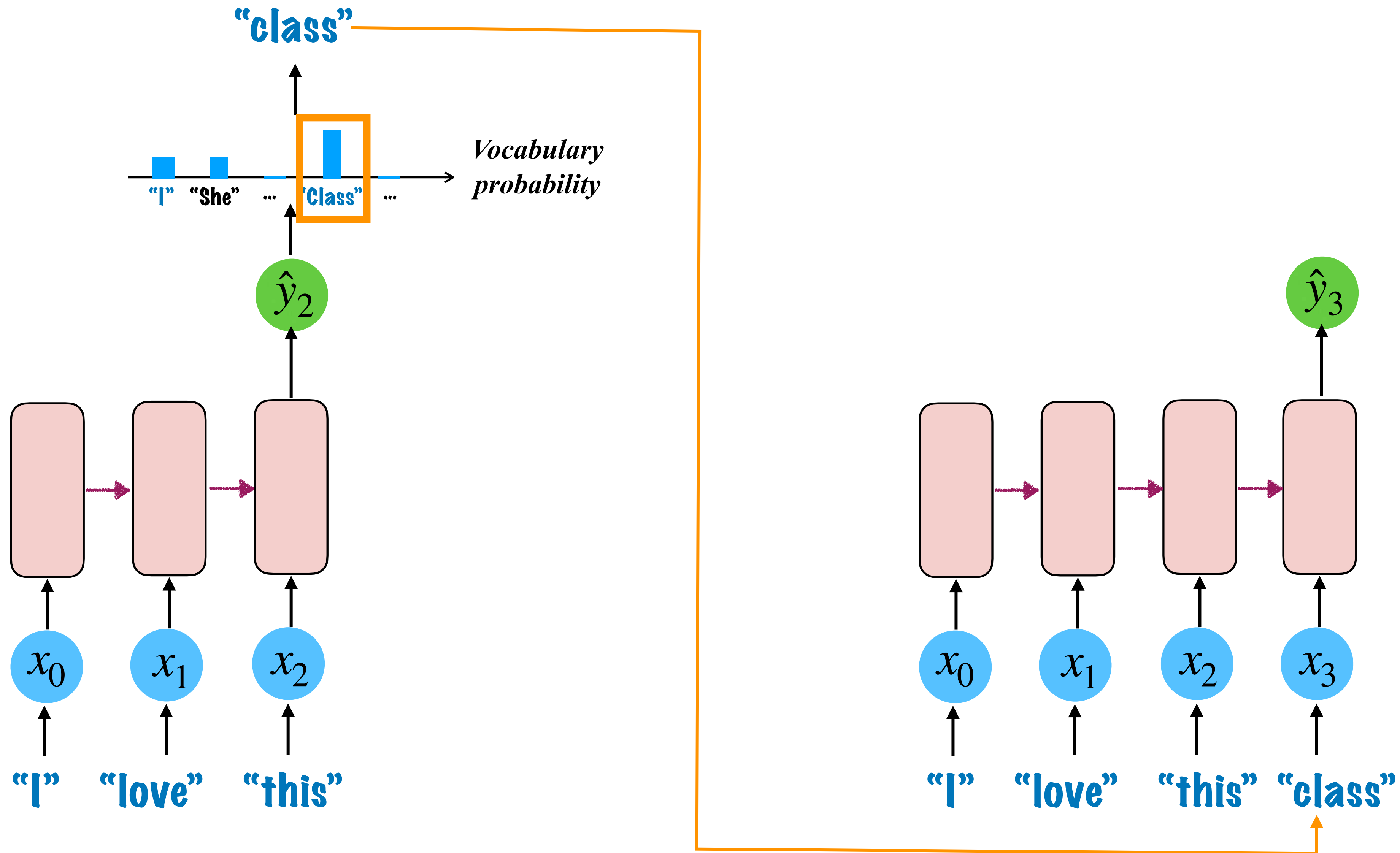
Language Models: Next token Prediction



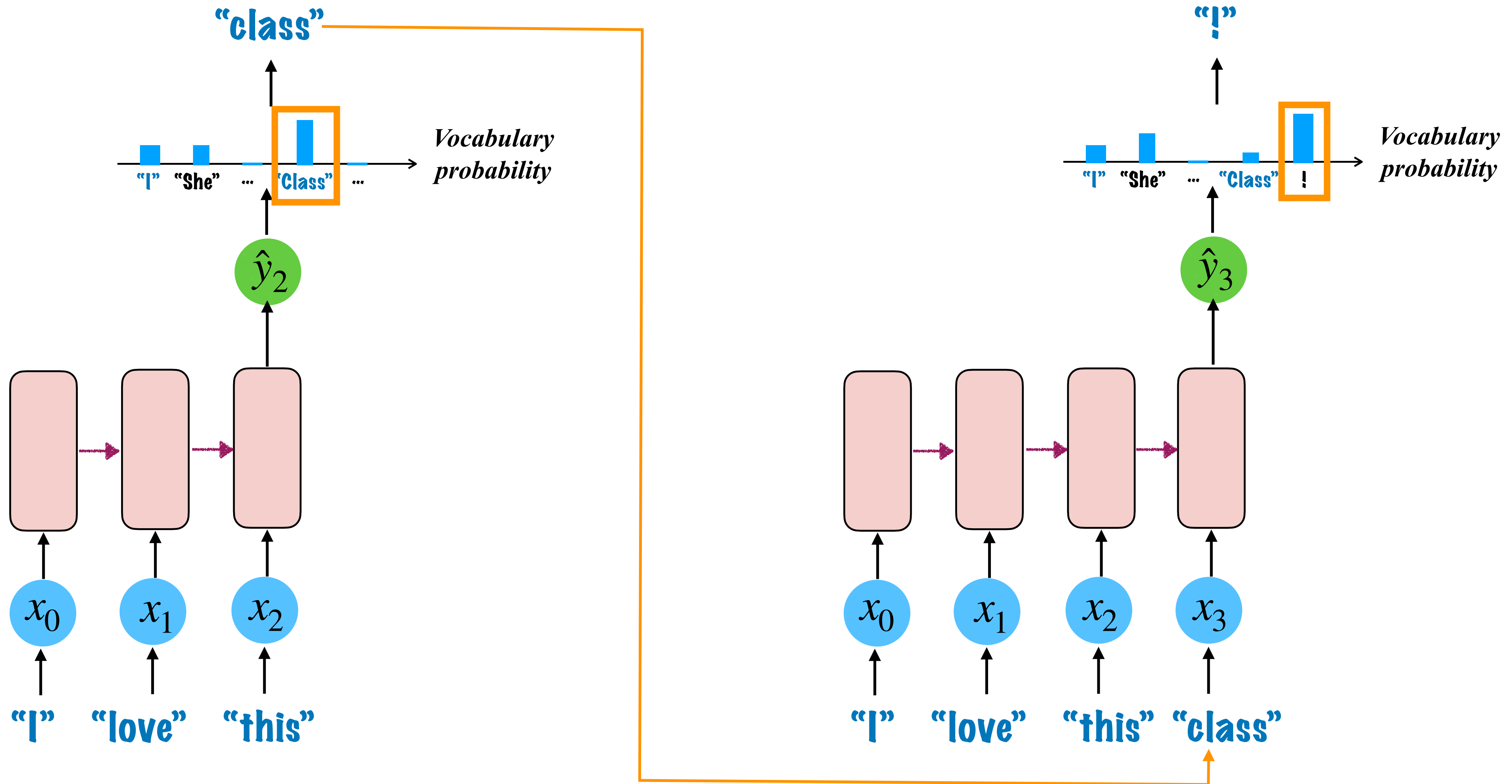
Language Models: Next token Prediction



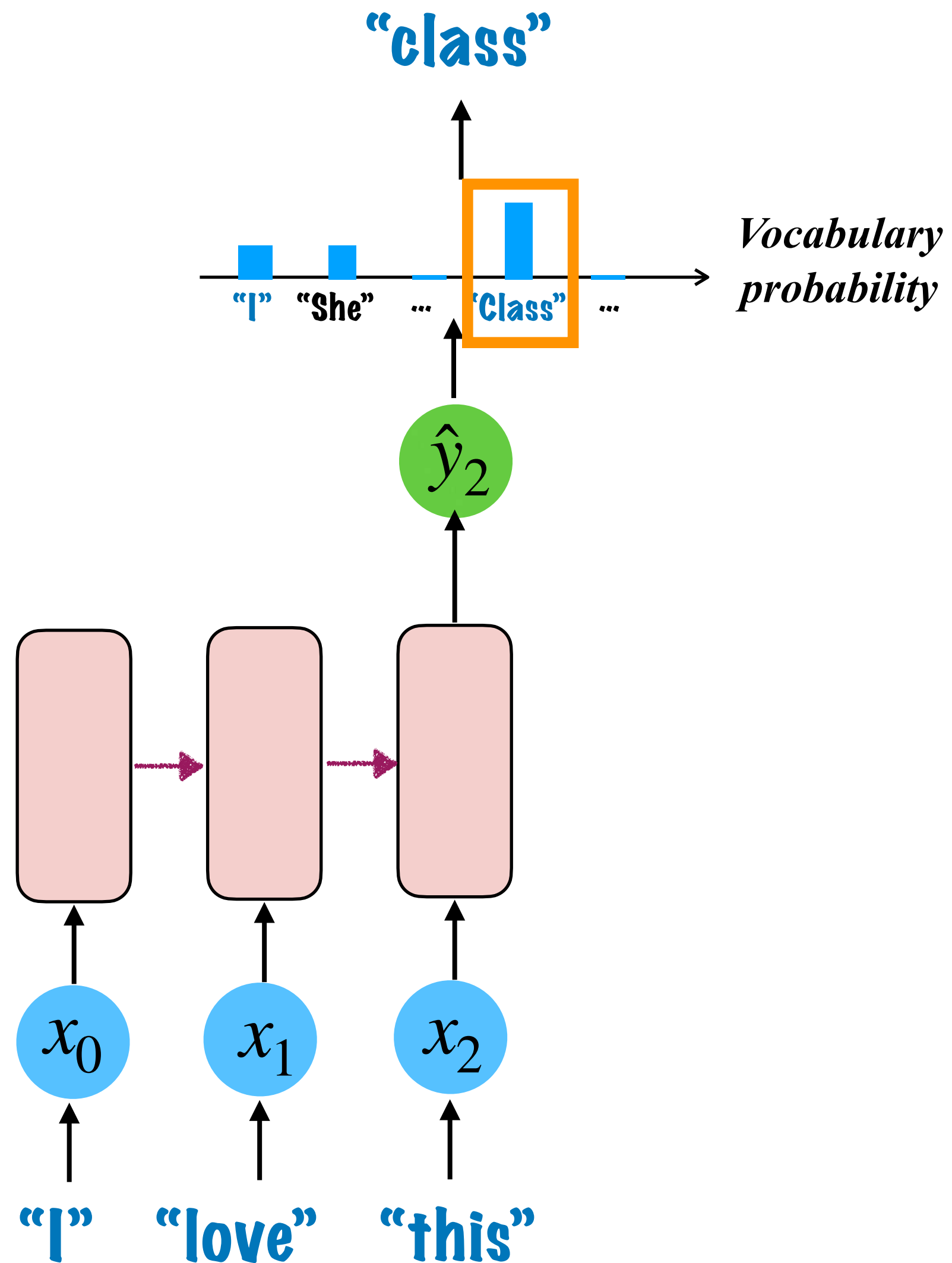
Language Models: Next token Prediction



Language Models: Next token Prediction



Language Models: Next token Prediction

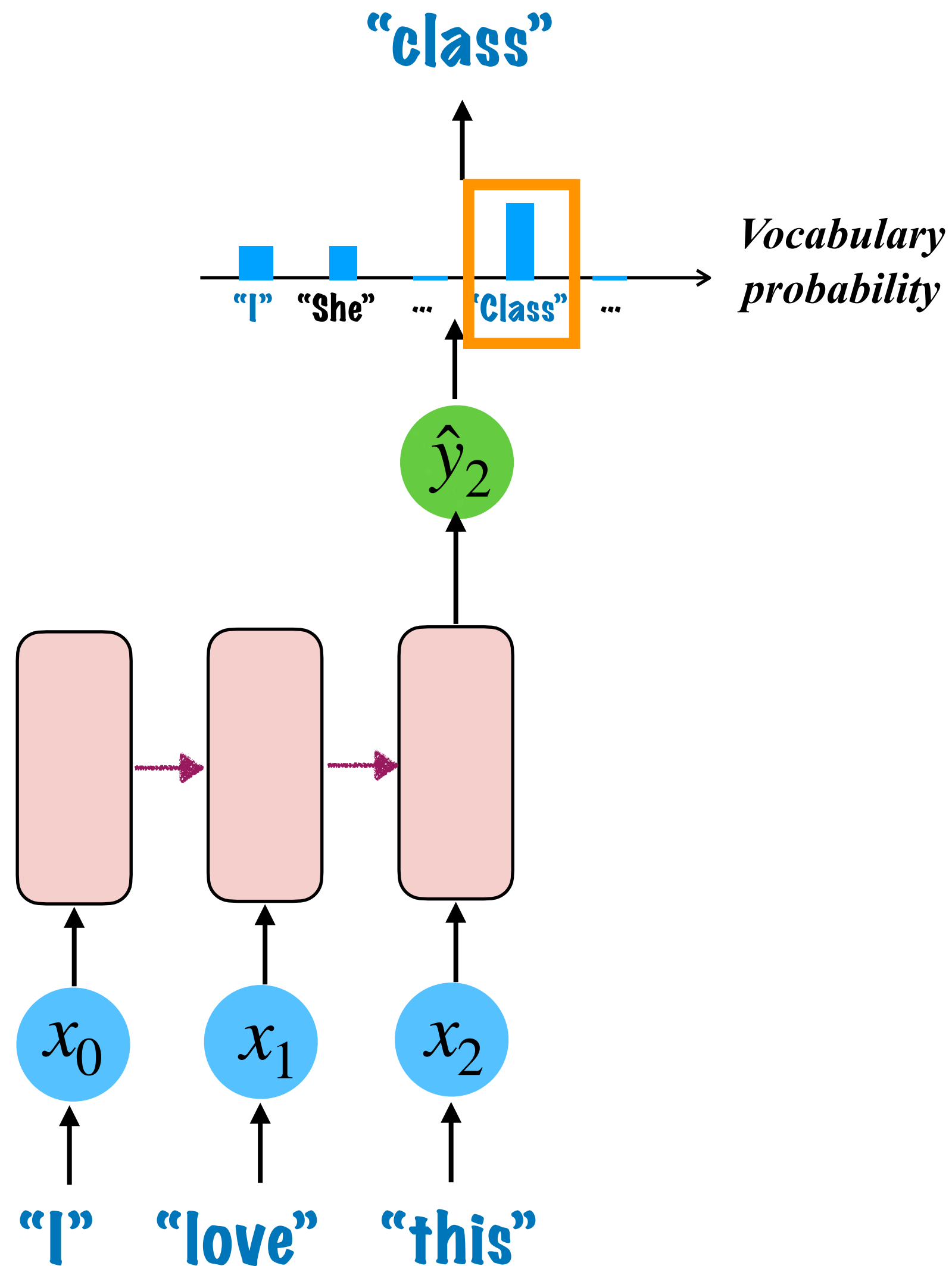


Basic probability chain rule:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$$

a product of next-token probabilities

Language Models: Next token Prediction



Basic probability chain rule:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$$

Example for understanding:

$$P(\text{I, love, this, class, !}) =$$

$$P(\text{I}) P(\text{love} | \text{I}) P(\text{this} | \text{I, love})$$

$$P(\text{class} | \text{I, love, this}) P(! | \text{I, love, this, class})$$

Sequence Modeling: Design Criteria

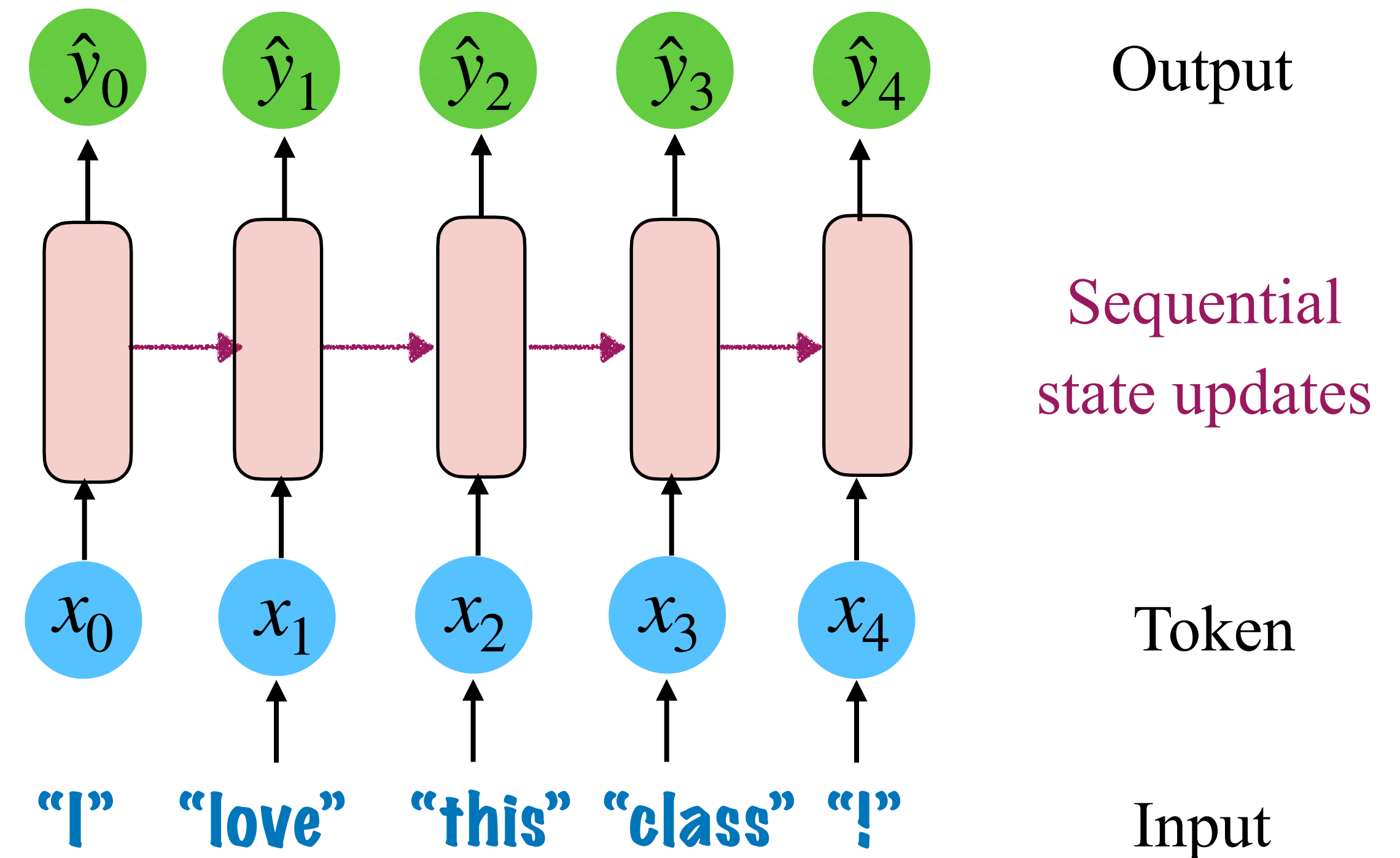
To model sequences, we need to:

1. Handle **variable-length** sequences
2. **Share parameters** across the sequence
3. Maintain information about **order** / **Continuous stream**
4. **Parallelization** preprocessing $\Rightarrow$ efficient training
5. Track **long-term** dependencies $\Rightarrow$ the ability to use information from far back in the sequence

Limitations of RNNs

To model sequences, we need to:

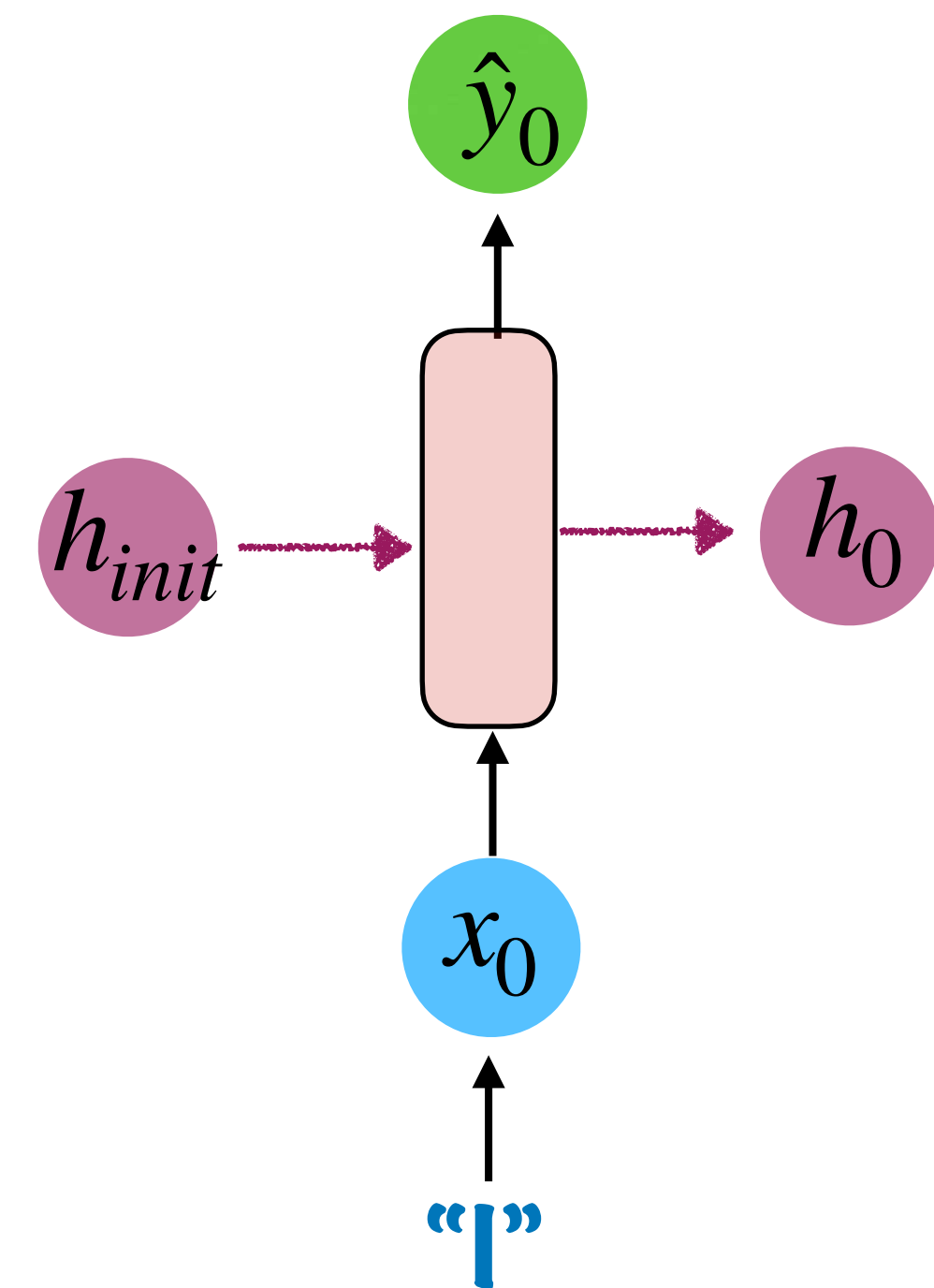
- ✓ 1. Handle **variable-length** sequences
- ✓ 2. **Share parameters** across the sequence
- ✓ 3. Maintain information about **order** / **Continuous stream**
- ✗ 4. ~~**Parallelization**~~ precessing
- ✗ 5. ~~Track **long-term**~~ dependencies



No Parallelization

Cannot parallelize across time steps (code!!!)

Sequential
state updates

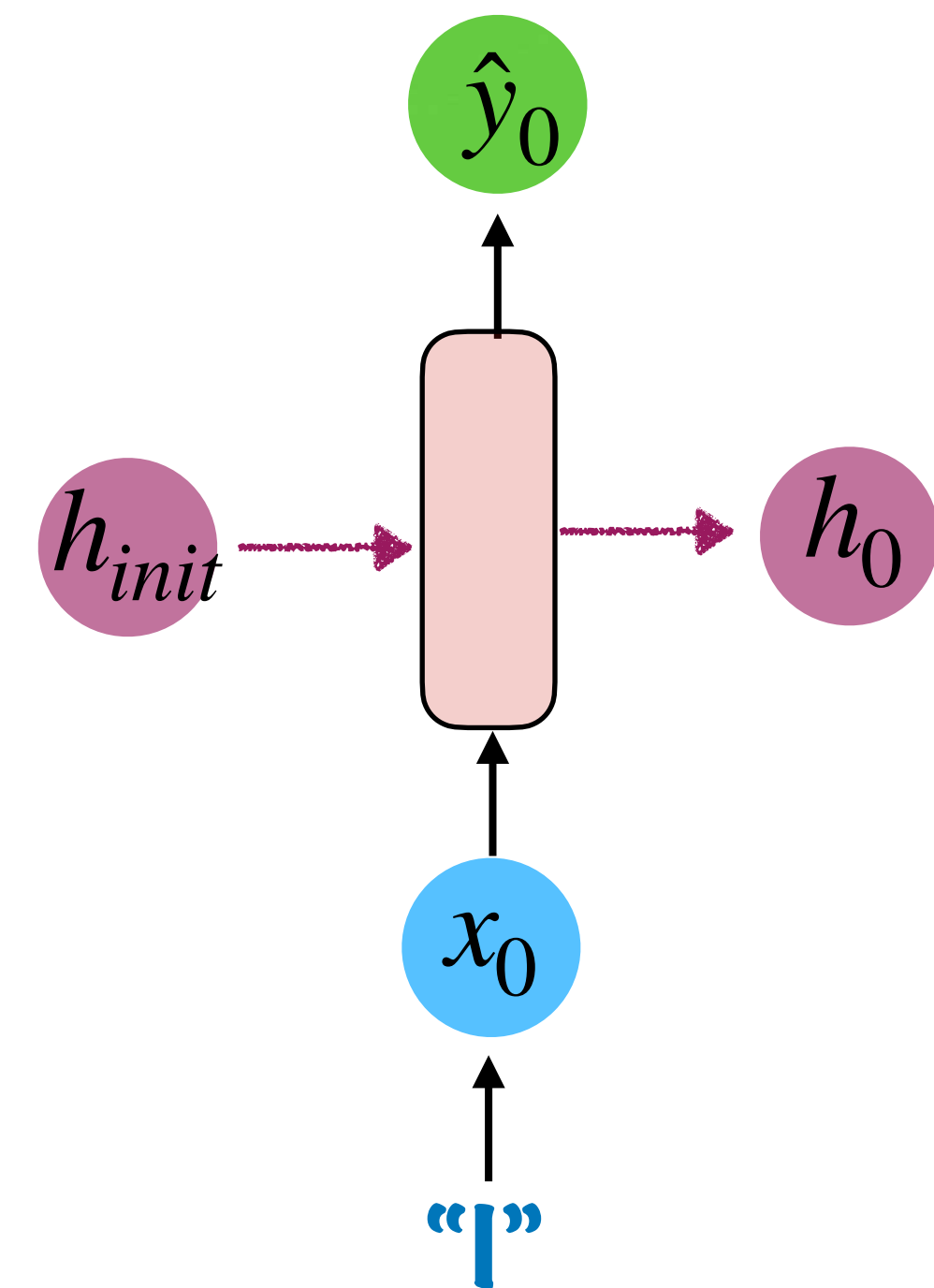


#1 feedforward
(0.01s)

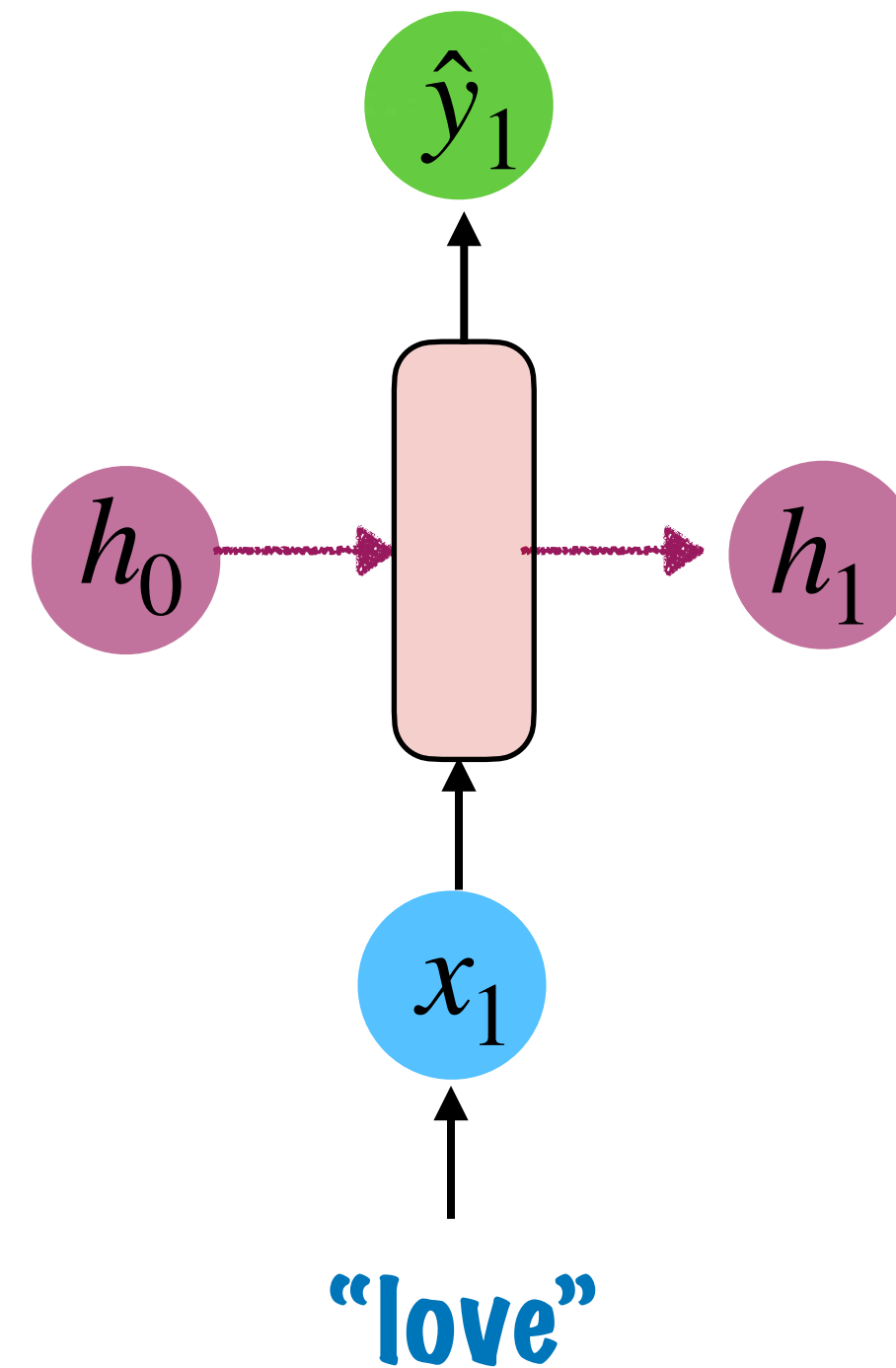
No Parallelization

Cannot parallelize across time steps (code!!!)

Sequential
state updates



#1 feedforward
(0.01s)



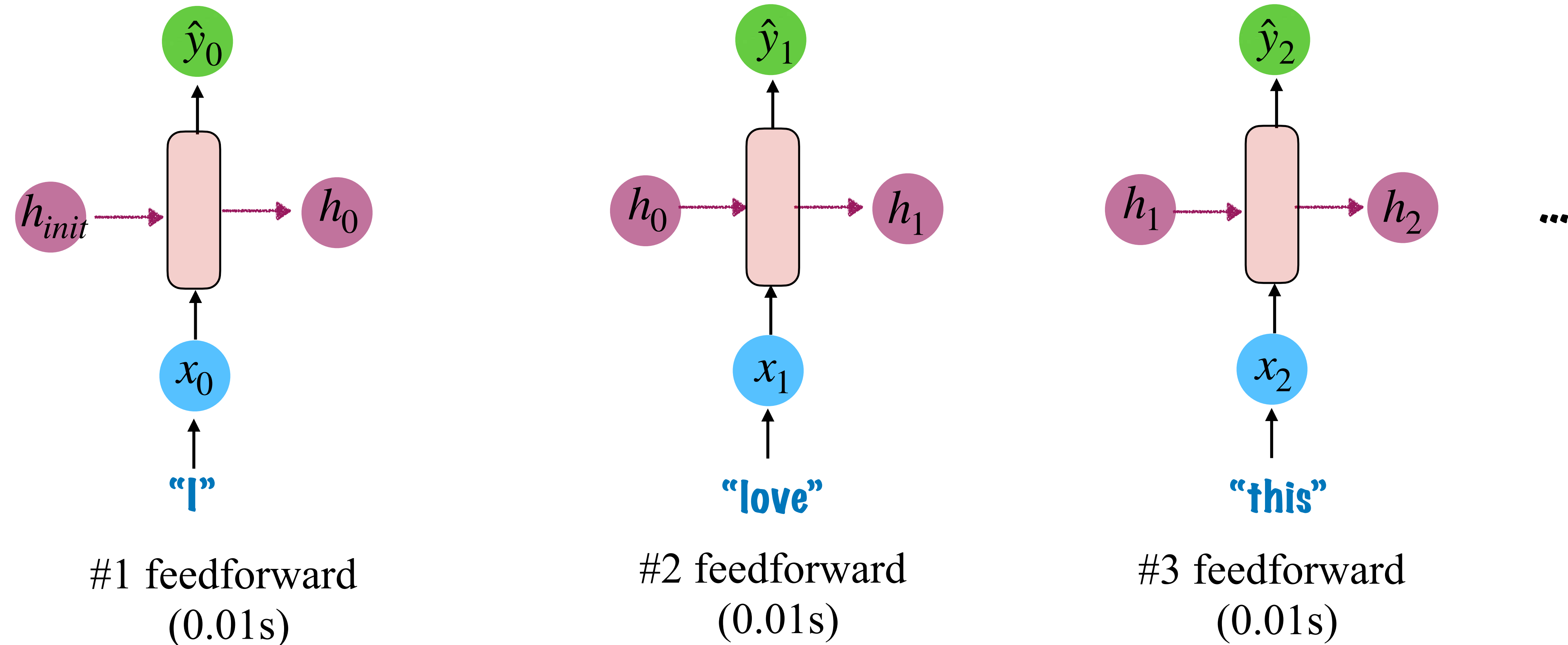
#2 feedforward
(0.01s)

This creates a sequential
dependency between time steps.

No Parallelization

Cannot parallelize across time steps (code!!!)

Sequential
state updates

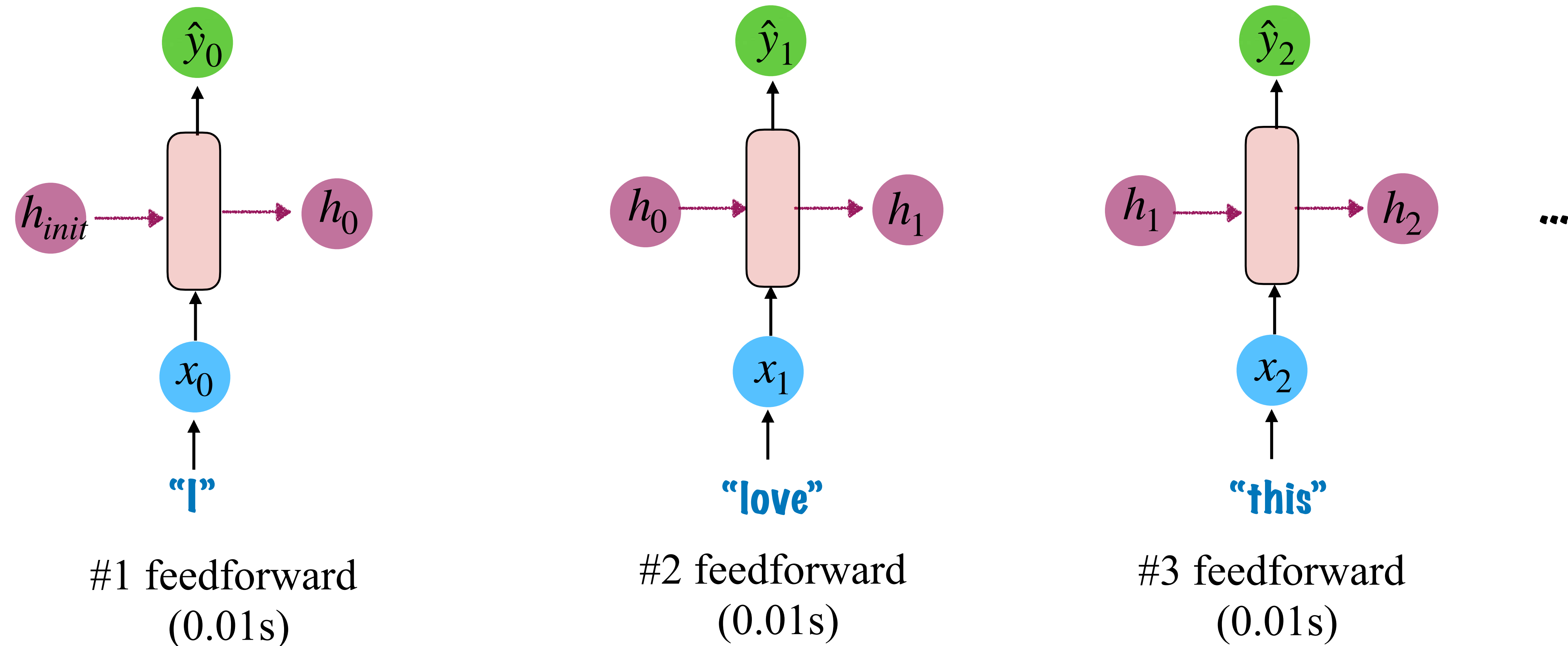


No Parallelization

Cannot parallelize across time steps (code!!!)

**Sequential
state updates**

**Slow
Training!**



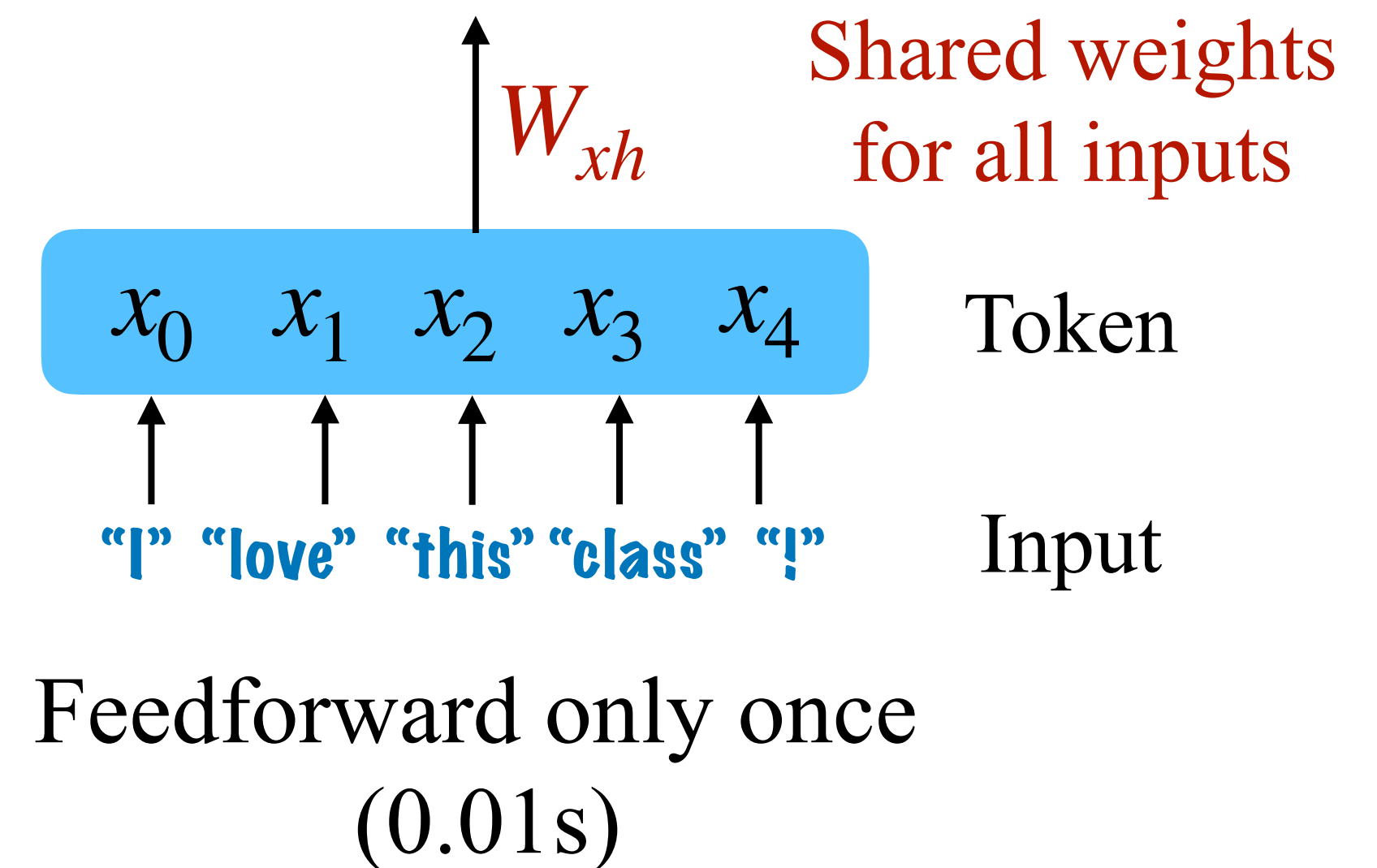
Parallelization $\Rightarrow$ Fast training

Core Idea: Remove step-by-step dependency?

feed everything into a dense network

Parallelization $\Rightarrow$ Fast training

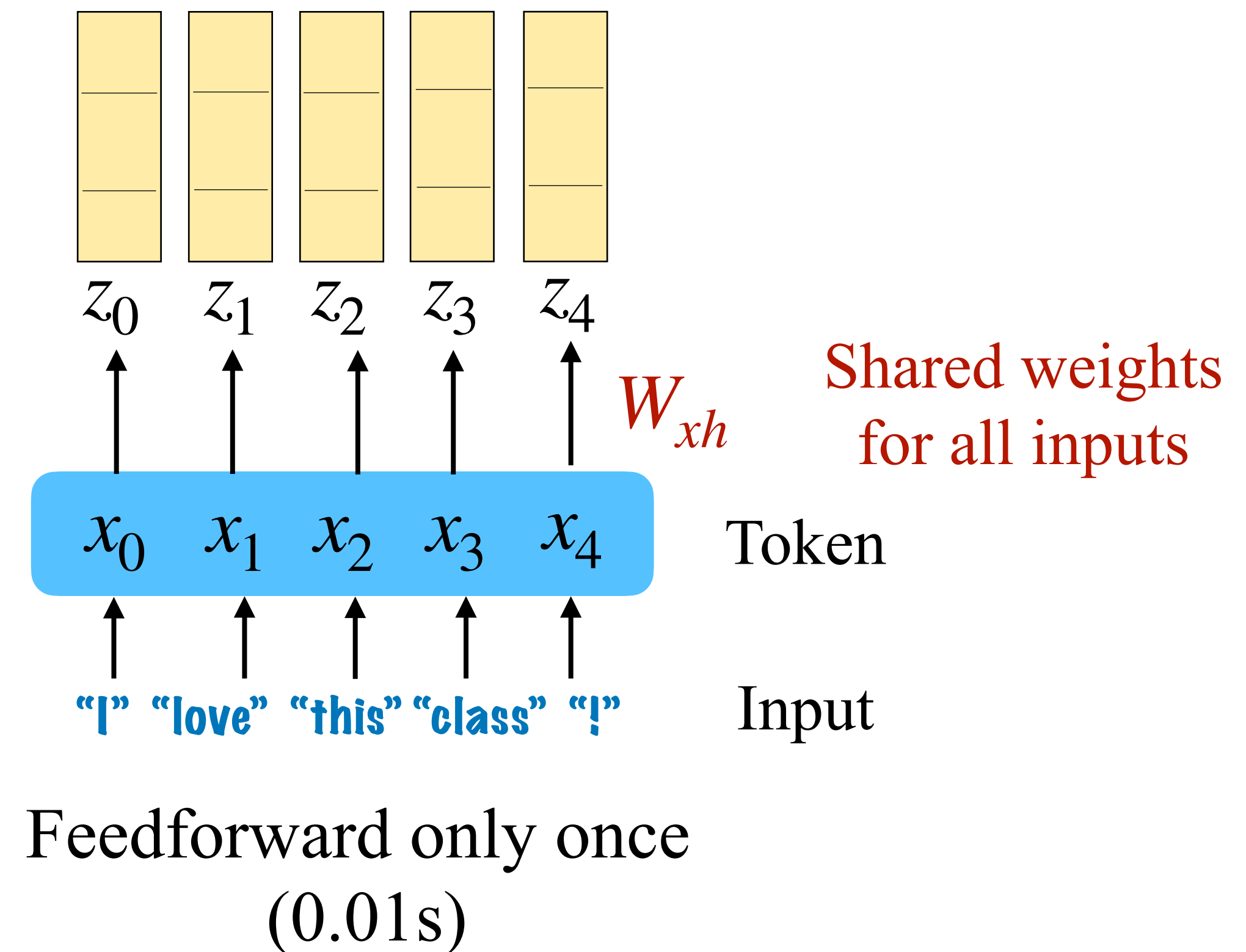
Core Idea: feed everything
into dense network



Parallelization $\Rightarrow$ Fast training

Core Idea: feed everything
into dense network

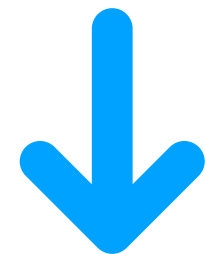
$$z_t = W_{xh} x_t, \quad \forall t \in \{0,1,2,3,4\}$$



Parallelization $\Rightarrow$ Fast training

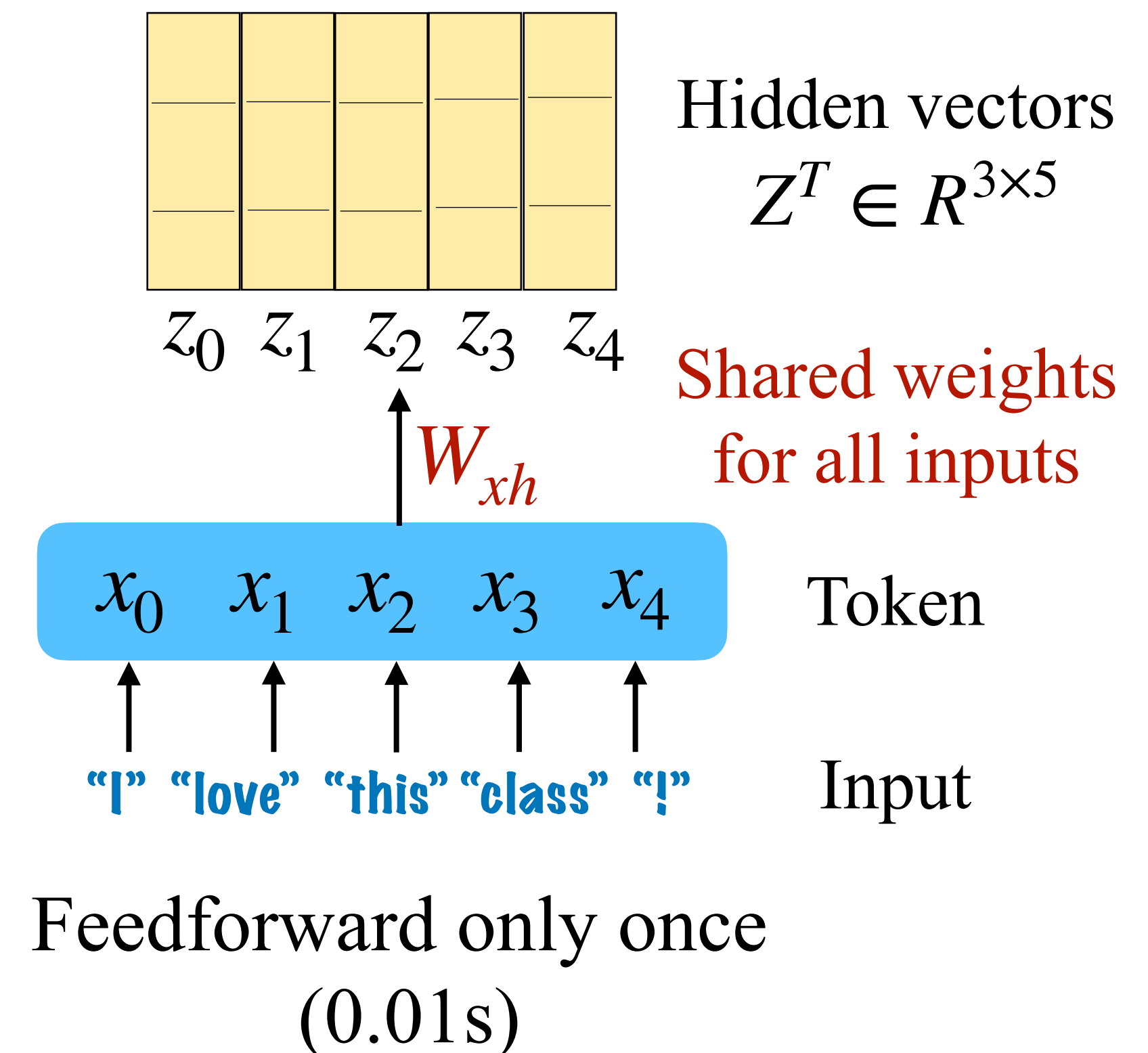
Core Idea: feed everything
into dense network

$$z_t = W_{xh} x_t, \quad \forall t \in \{0, 1, 2, 3, 4\}$$



$$[z_0, z_1, z_2, z_3, z_4] = W_{xh} [x_0, x_1, x_2, x_3, x_4]$$

Hidden vectors



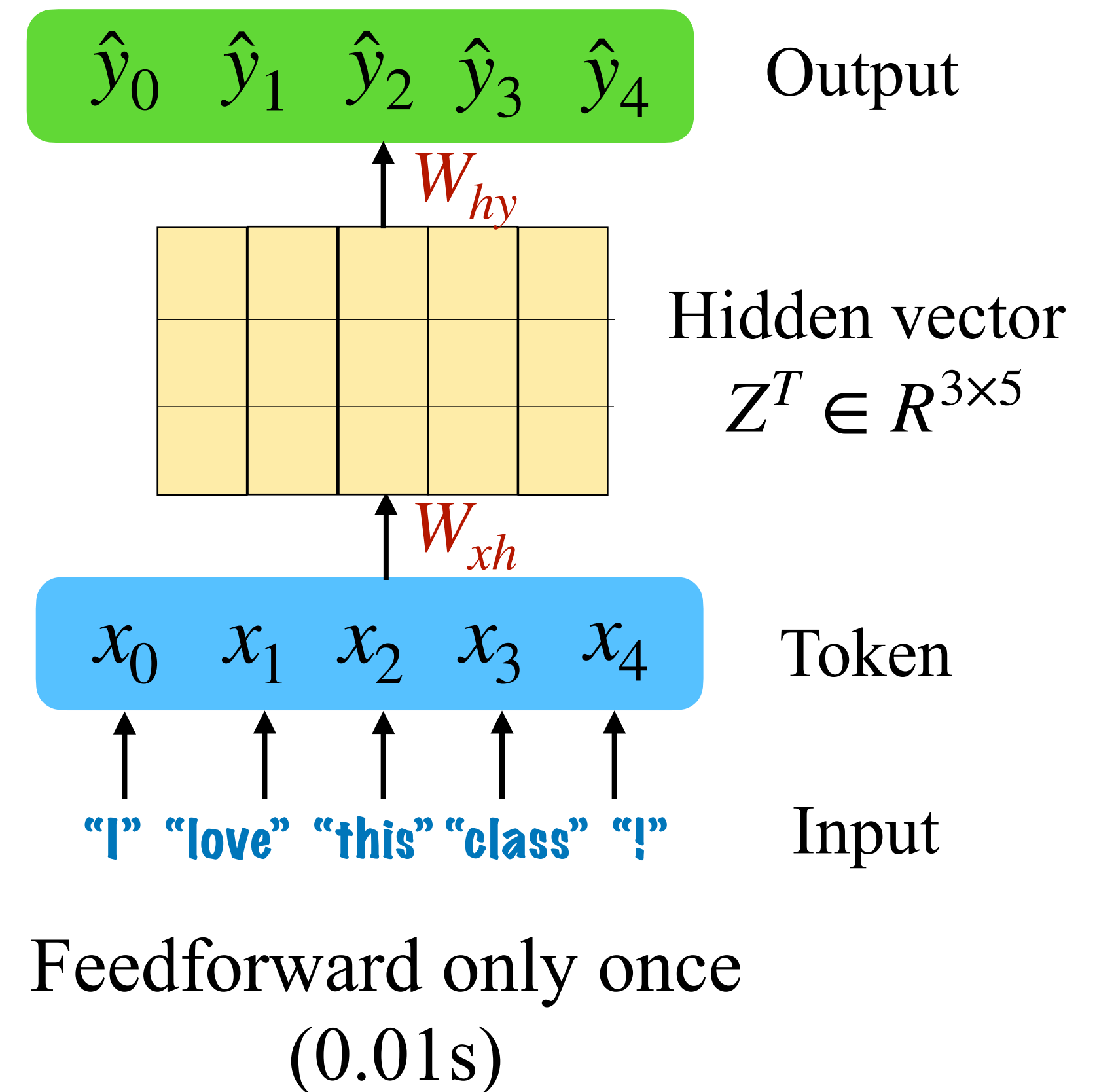
Parallelization $\Rightarrow$ fast training

Processing tokens separately
still leaves two questions

$$[z_0, z_1, z_2, z_3, z_4] = W_{xh}[x_0, x_1, x_2, x_3, x_4]$$

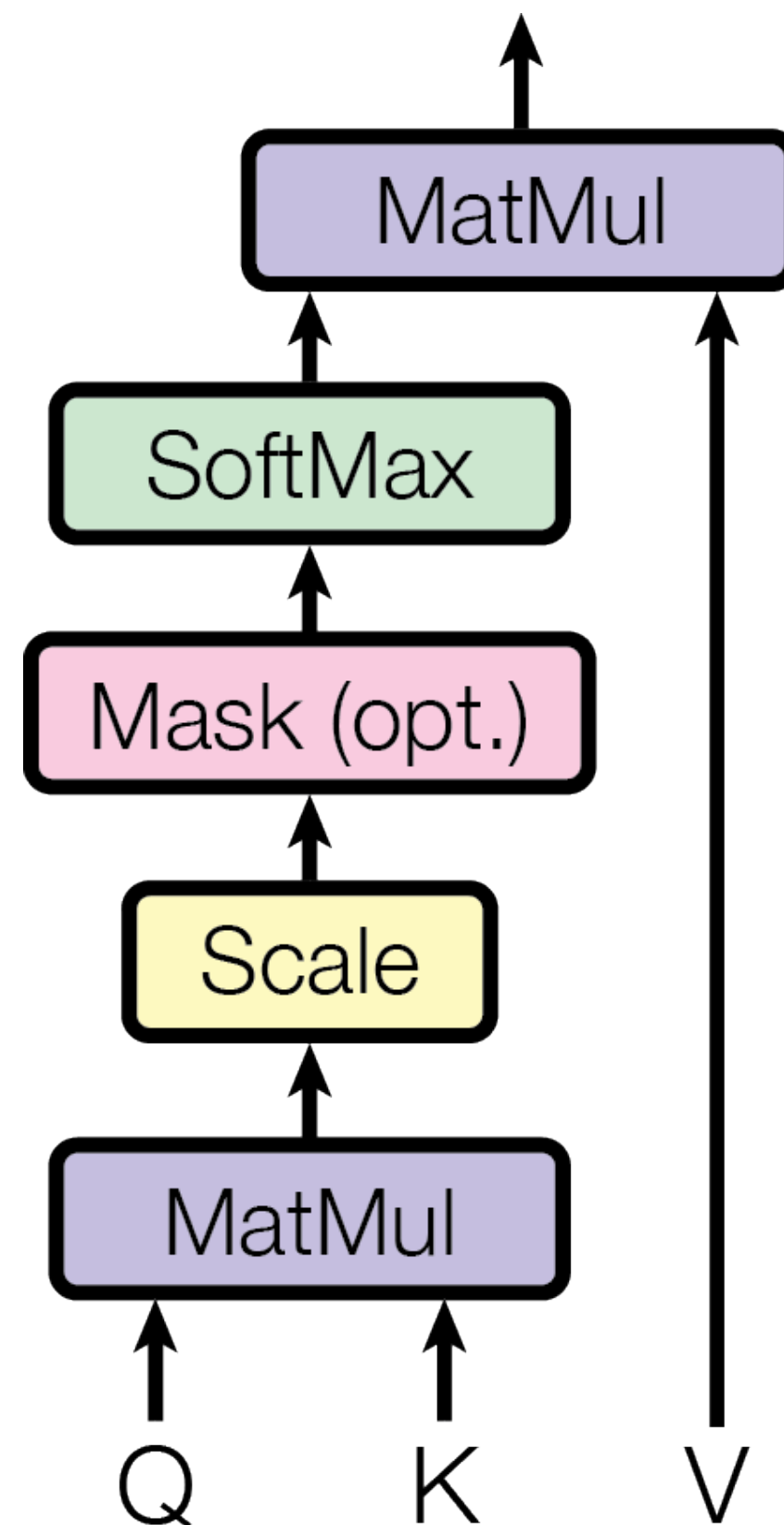
Q1: How to encode the relationship
between multiple tokens?

Q2: How to adapt to continuous
stream / preserve order?
(Only see previous tokens)



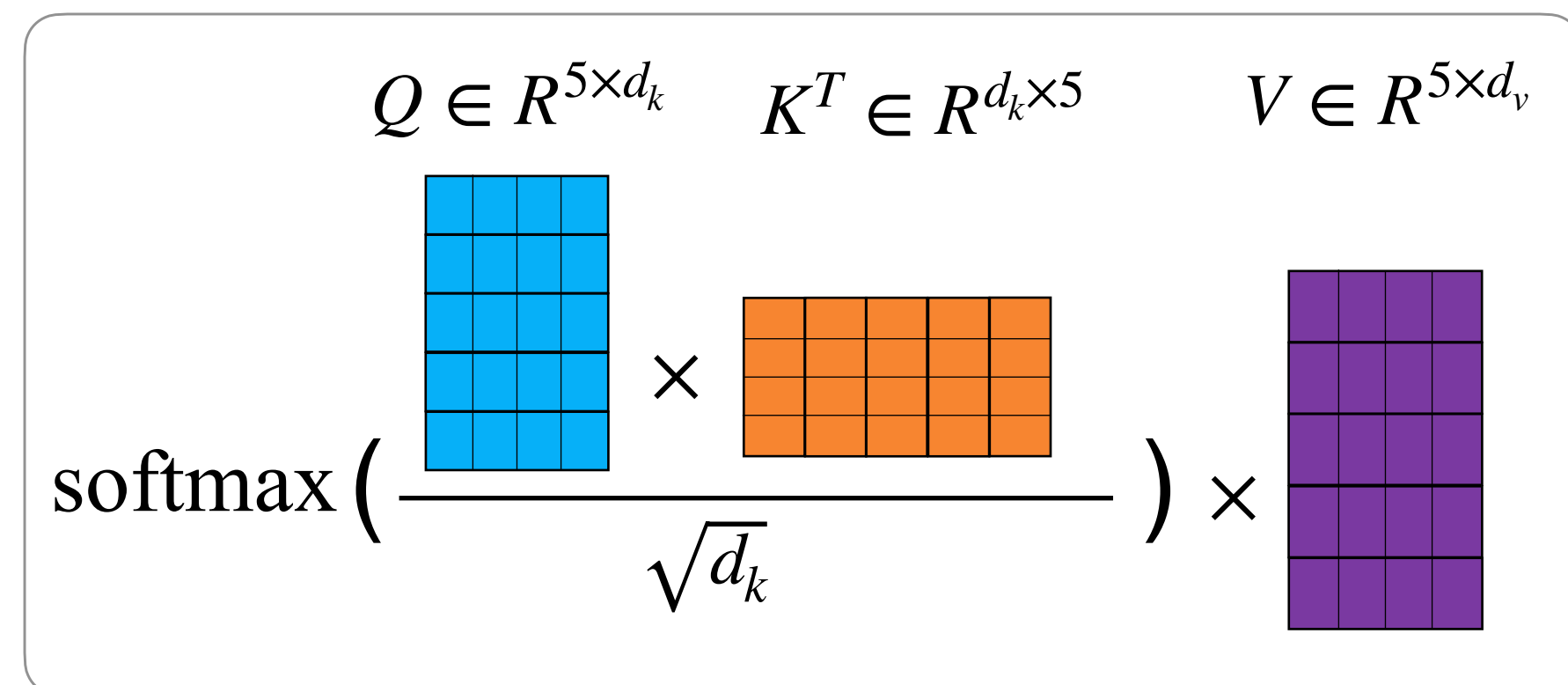
Attention & Transformers

Parallelization $\Rightarrow$ Fast training



Attention formulation:

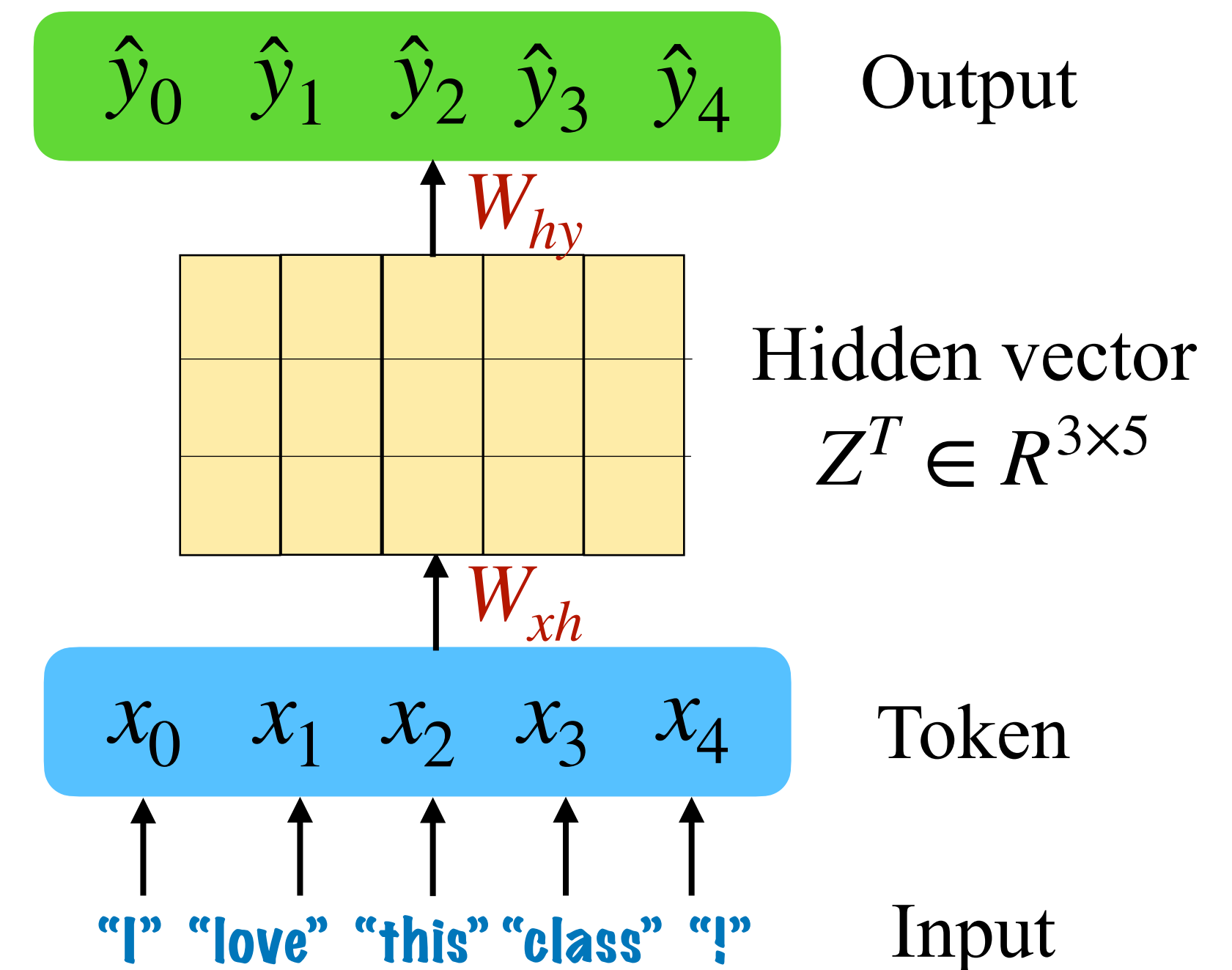
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Parallelization $\Rightarrow$ Fast training:

Core Idea: Feed everything into dense network

Q1: How to encode the relationship between multiple tokens?
(Exchange information)



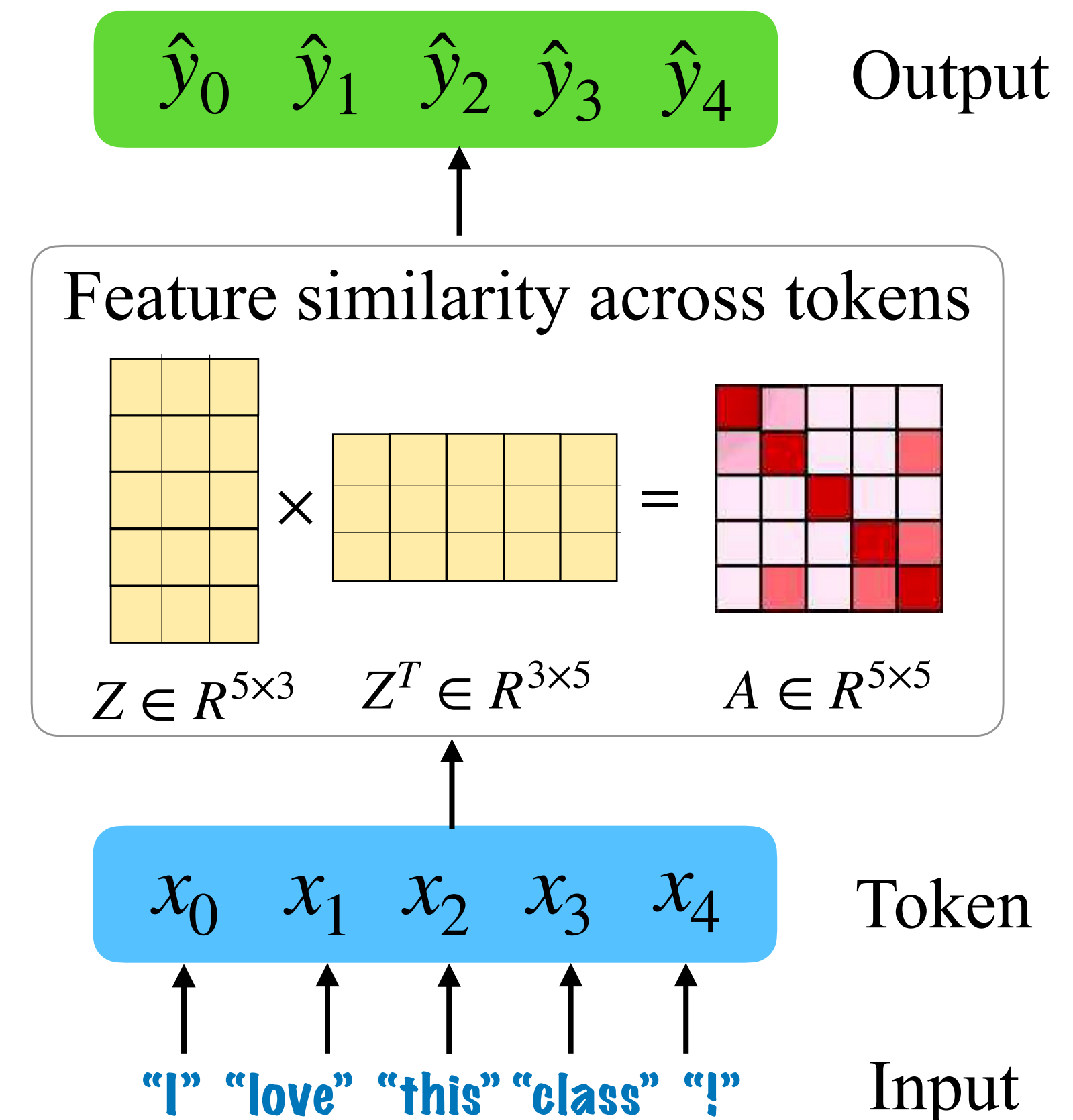
Parallelization $\Rightarrow$ Fast training:

Core Idea: Feed everything into dense network

Q1: How to encode the relationship between multiple tokens?
(Exchange information)

Feature similarity:
(dot-product)

$$A = ZZ^T \in R^{5 \times 5}$$



Parallelization $\Rightarrow$ Fast training:

Core Idea: Feed everything into dense network

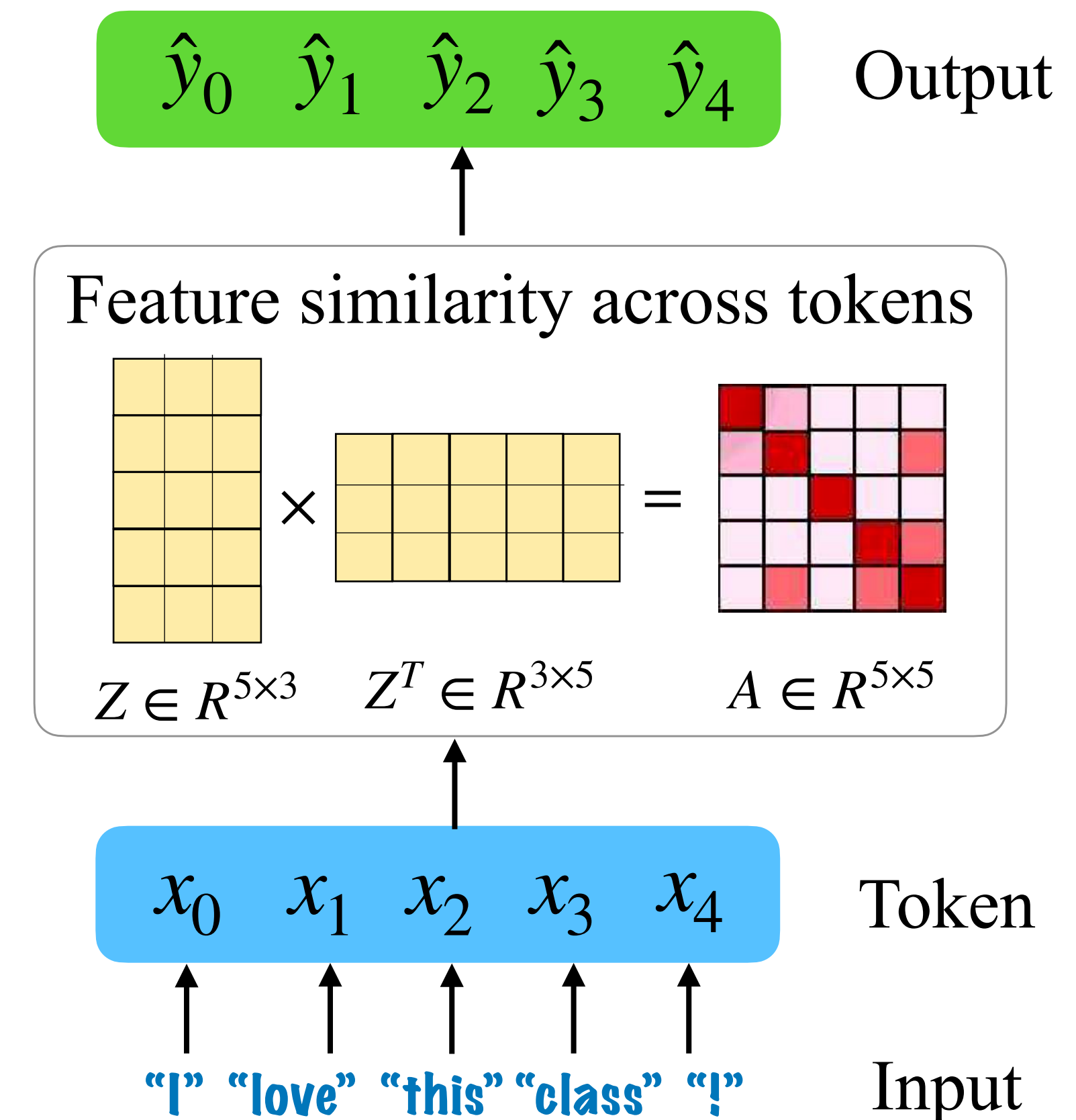
Q1: How to encode the relationship between multiple tokens?
(Exchange information)

Feature similarity:

$$A = ZZ^T \in R^{5 \times 5}$$

	z_0	z_1	z_2	z_3	z_4
z_0	150	89	-25	-36	56
z_1	69	163	-156	-56	89
z_2	49	81	452	26	54
z_3	-45	15	-278	562	362
z_4	-33	226	36	319	268

dot-product similarity between two feature vectors



Parallelization $\Rightarrow$ fast training:

Core Idea: Feed everything into dense network

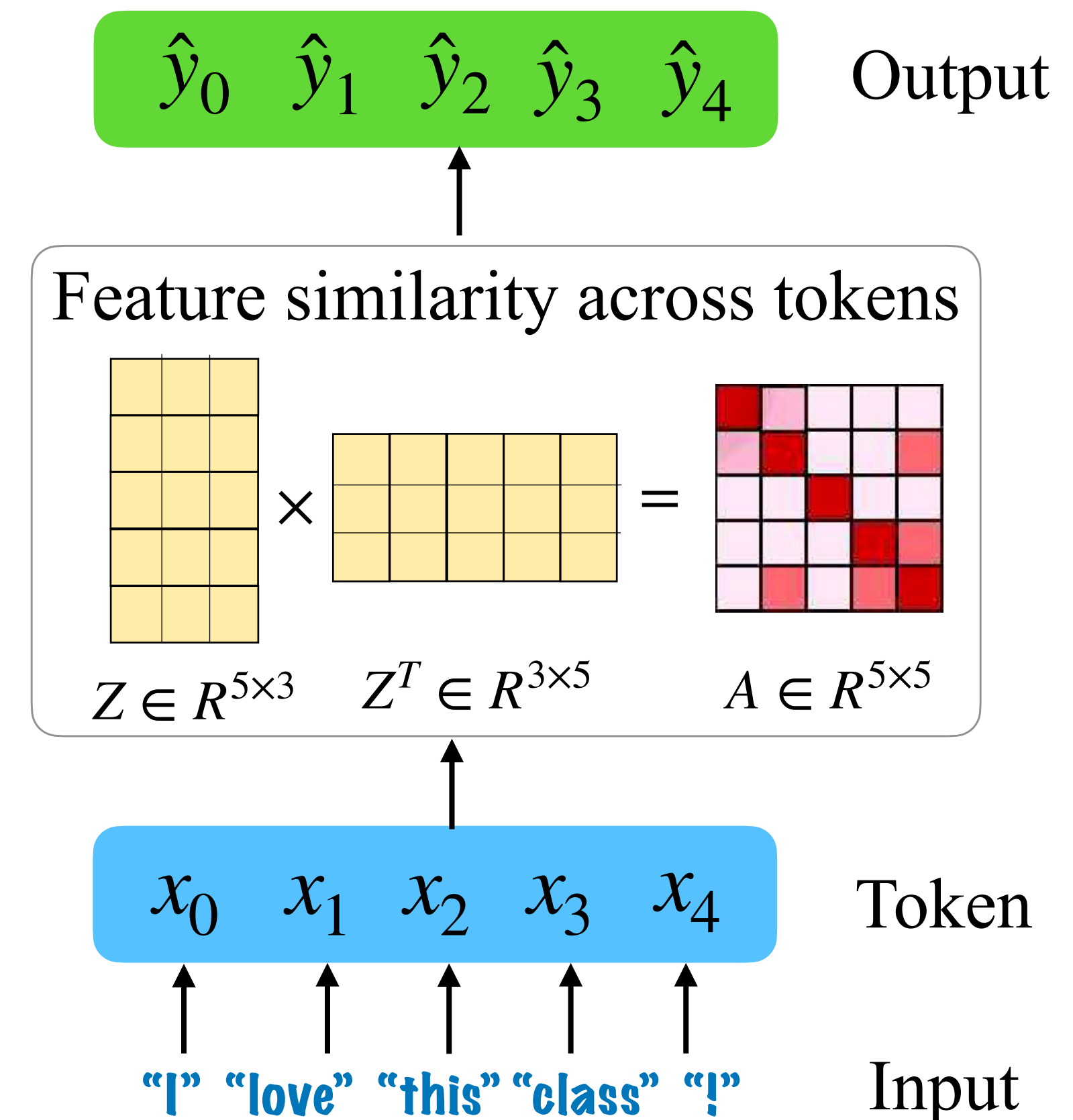
Q1: How to encode the relationship between multiple tokens?
(Exchange information)

Feature similarity: $A = \text{Softmax}(ZZ^T) \in R^{5 \times 5}$

Normalization with softmax separately to each row:

$$\sum_{j=1}^5 A_{ij} = 1$$

	z_0	z_1	z_2	z_3	z_4
z_0	0.7	0.18	0	0	0.12
z_1	0.1	0.75	0	0	0.15
z_2	0	0	1.0	0	0
z_3	0	0	0	0.6	0.4
z_4	0	0.25	0	0.25	0.5



Parallelization $\Rightarrow$ fast training:

Core Idea: Feed everything into dense network

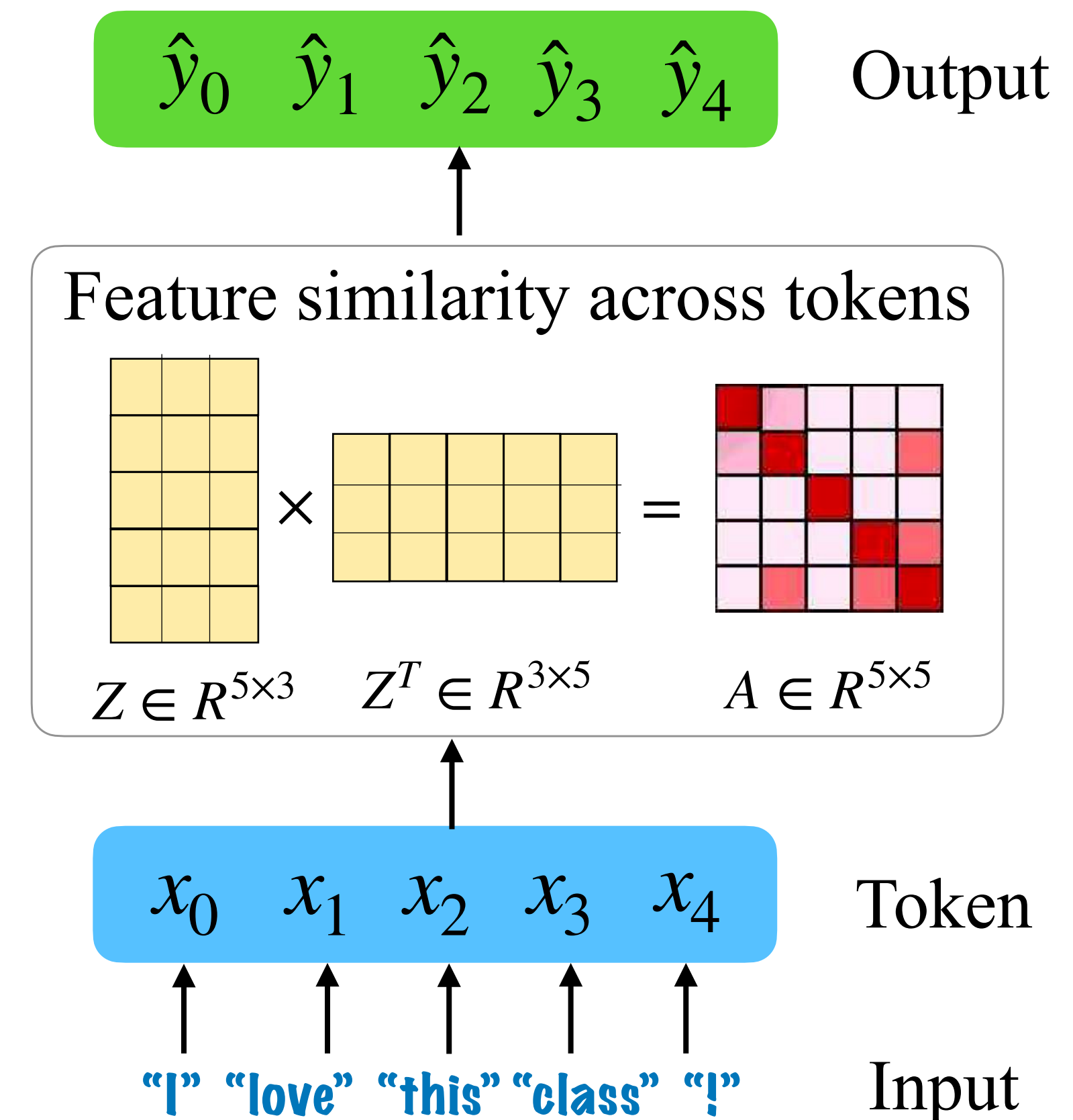
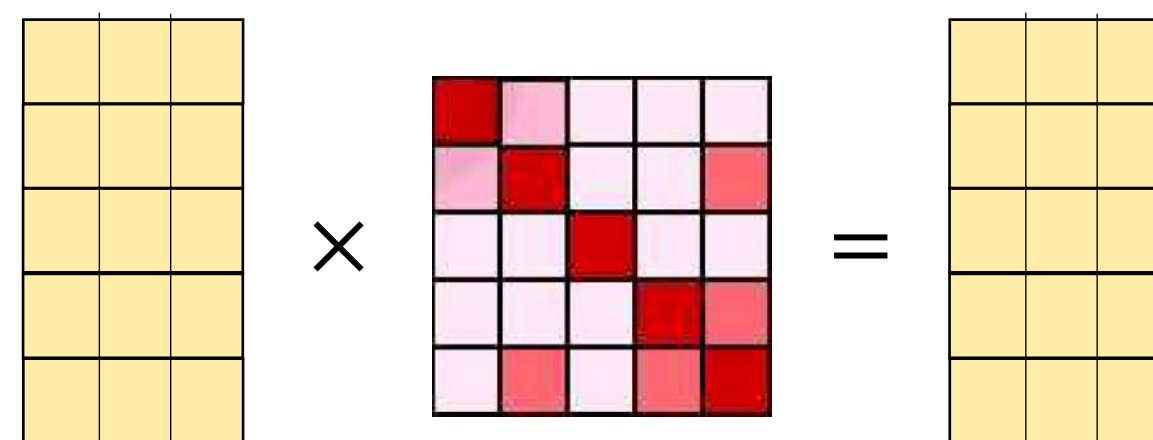
Q1: How to encode the relationship between multiple tokens?
(Exchange information)

Feature similarity: $A = \text{Softmax}(ZZ^T) \in R^{5 \times 5}$

Feature update with similarity

$$Y = AZ \in R^{5 \times 3}$$

A weighted sum of the token features

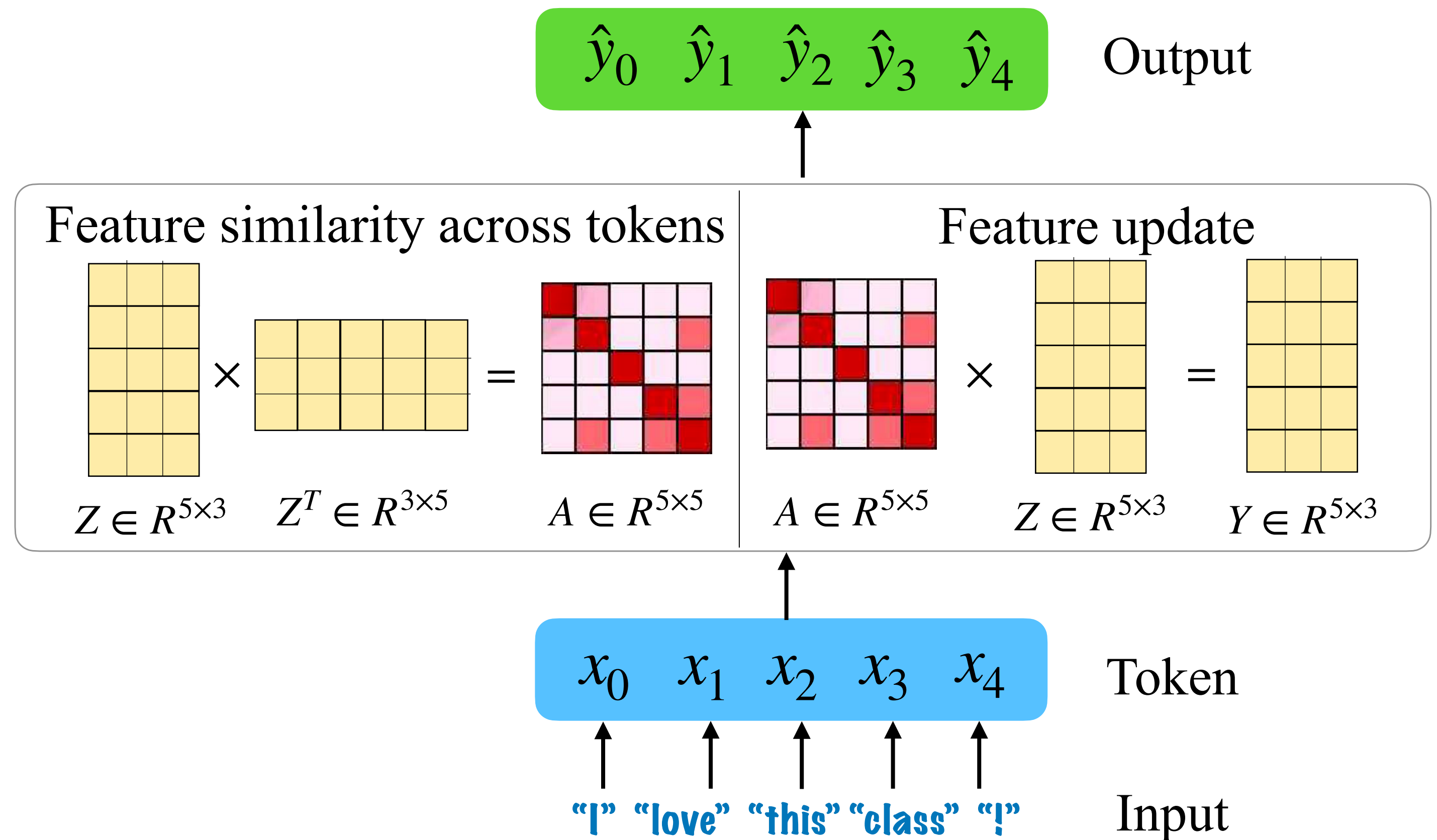


Parallelization $\Rightarrow$ fast training:

Core Idea: Feed everything into dense network

Feature update with similarity

$$Y = \text{softmax}(ZZ^T)Z$$



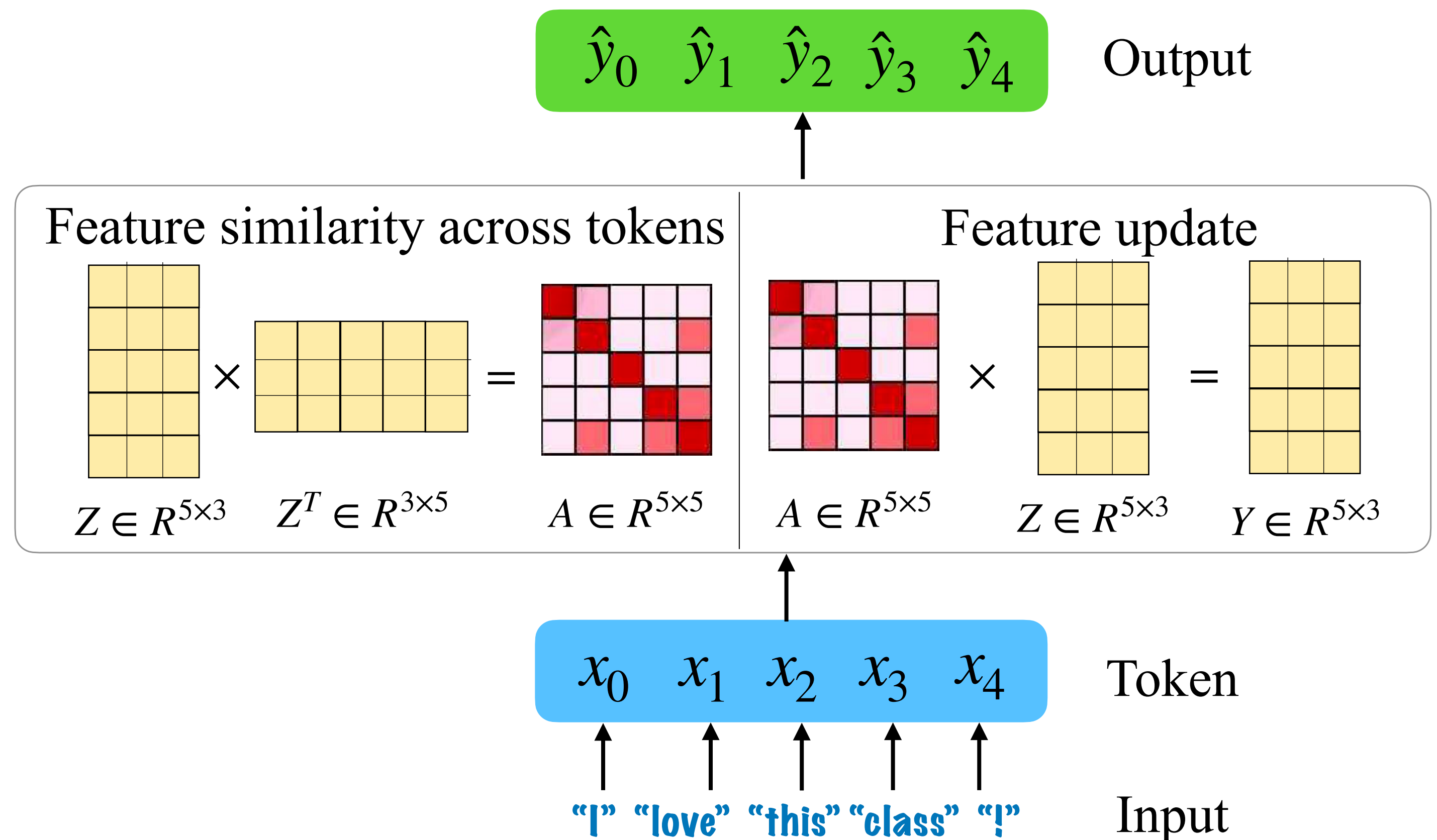
Parallelization but not continuous stream:

Core Idea: Feed everything into dense network

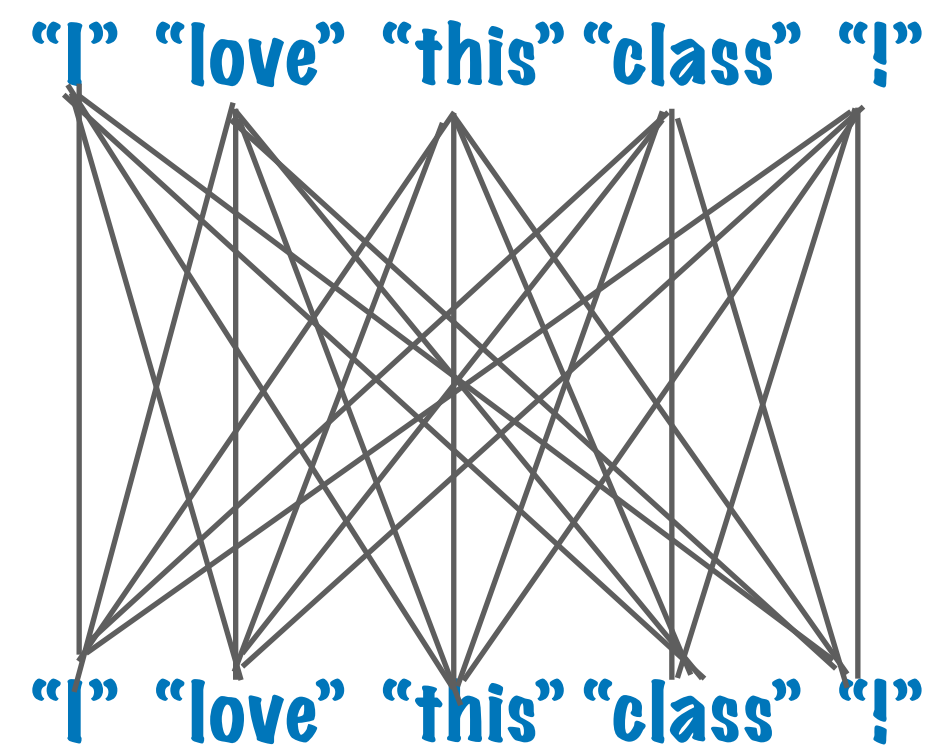
✗ Continuous stream
(online stream)

✓ Parallelization

✗ Long-term memory



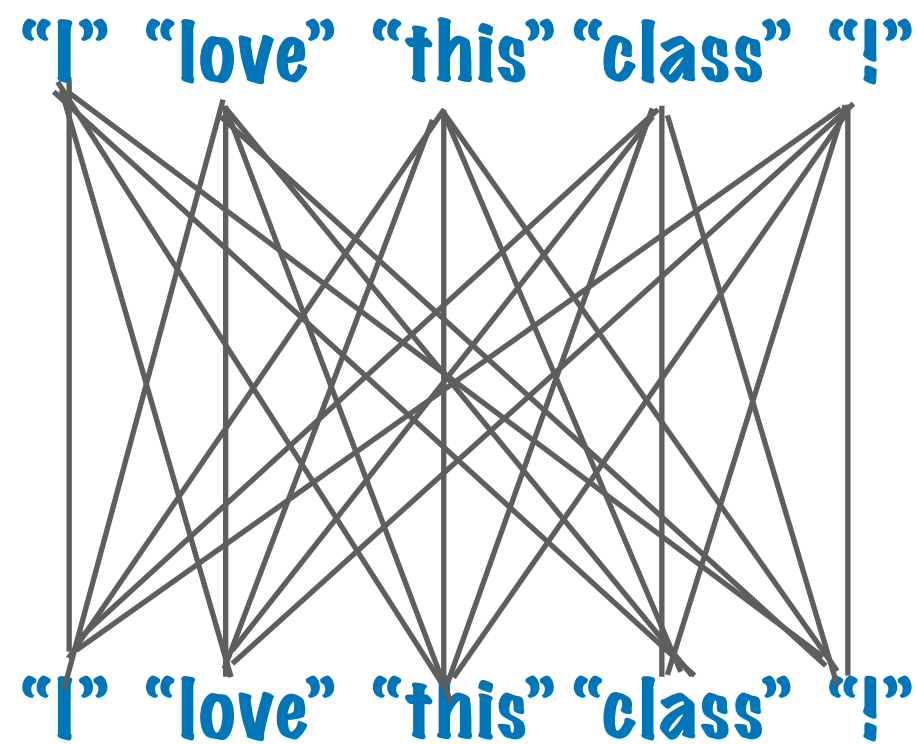
Continuous Streaming



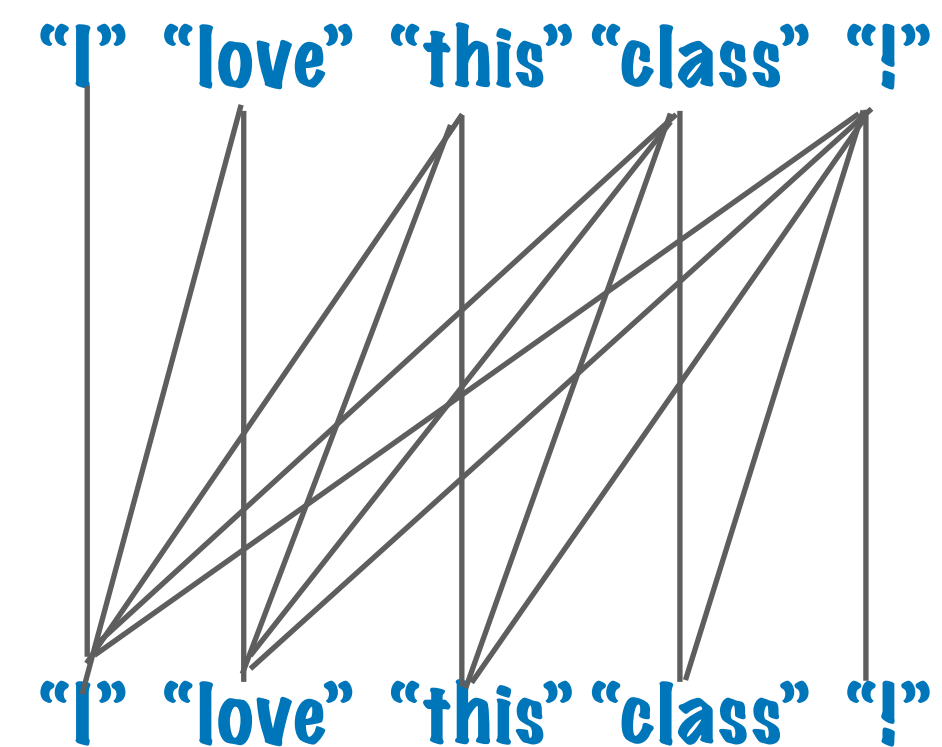
All tokens can see each other.
(Full attention)

This is useful **when the complete input is available**,
but not **when we must predict without seeing the future**.

Continuous Streaming



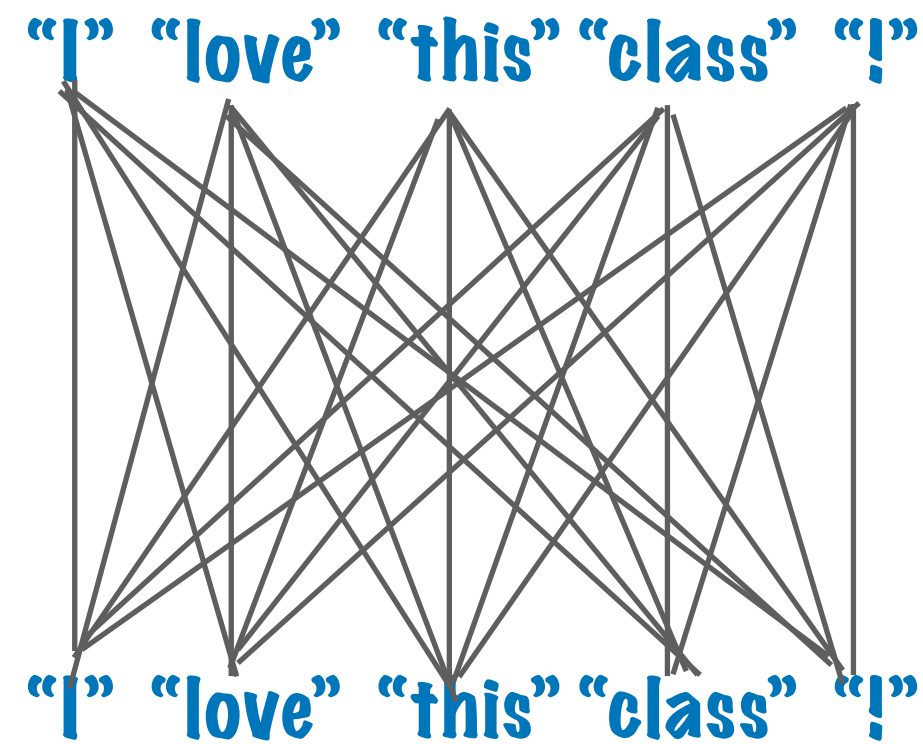
All tokens can see each other.
(Full attention)



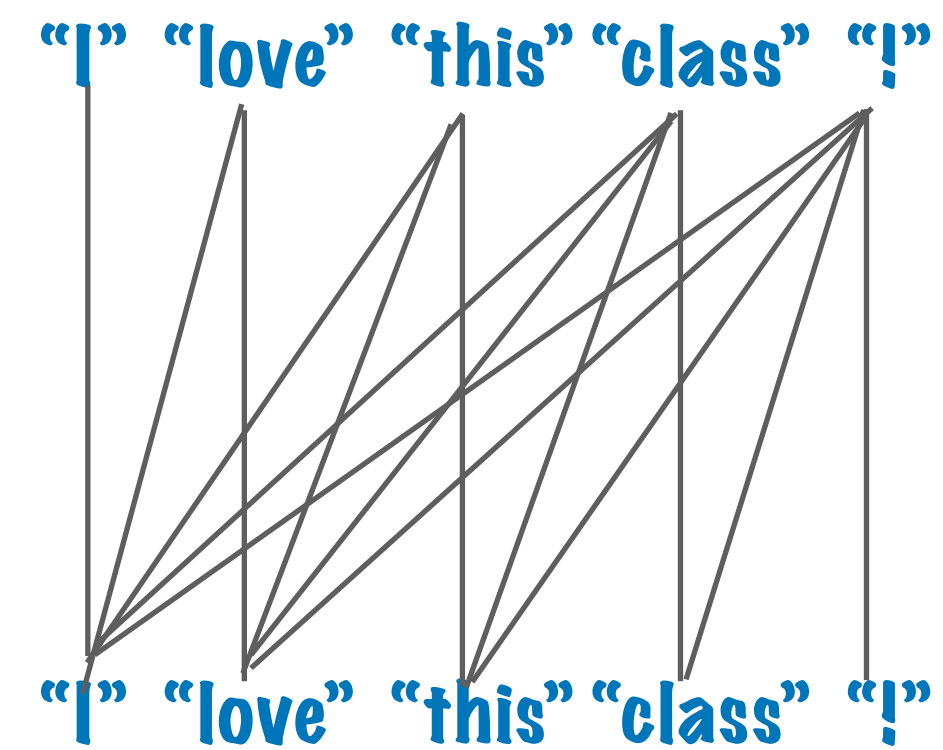
Only see previous tokens!
(Casual attention)

allow each position to use only its current and earlier tokens

Continuous Streaming



All tokens can see each other.
(Full attention)



Only see previous tokens!
(Causal attention)

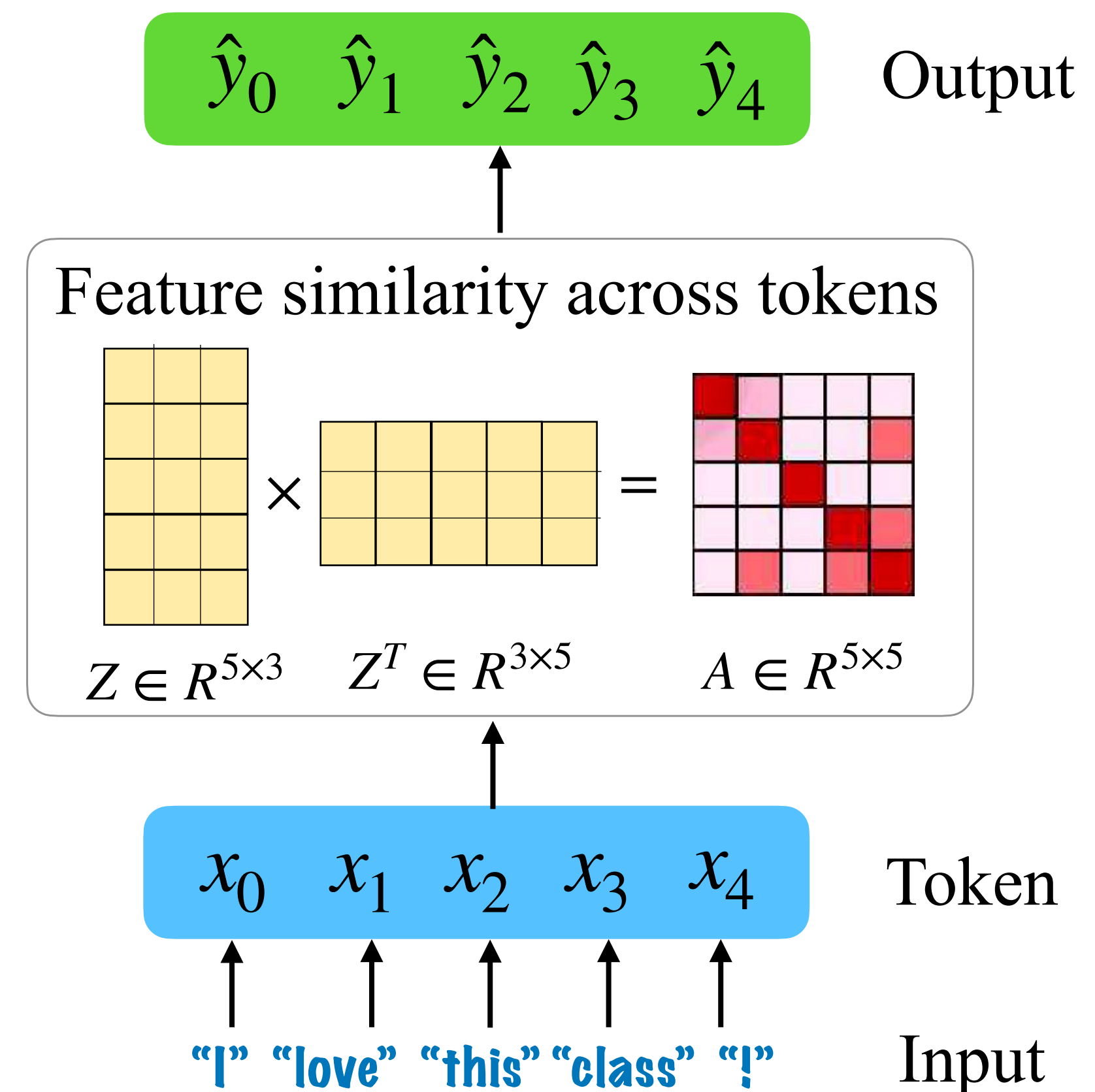
x_0
 x_0 x_1
 x_0 x_1 x_2
 x_0 x_1 x_2 x_3
 x_0 x_1 x_2 x_3 x_4 **Lower-triangular pattern**

Parallelization but not continuous stream:

Core Idea: Feed everything into dense network

Q2: How to adapt to continuous stream?

Only see previous tokens!



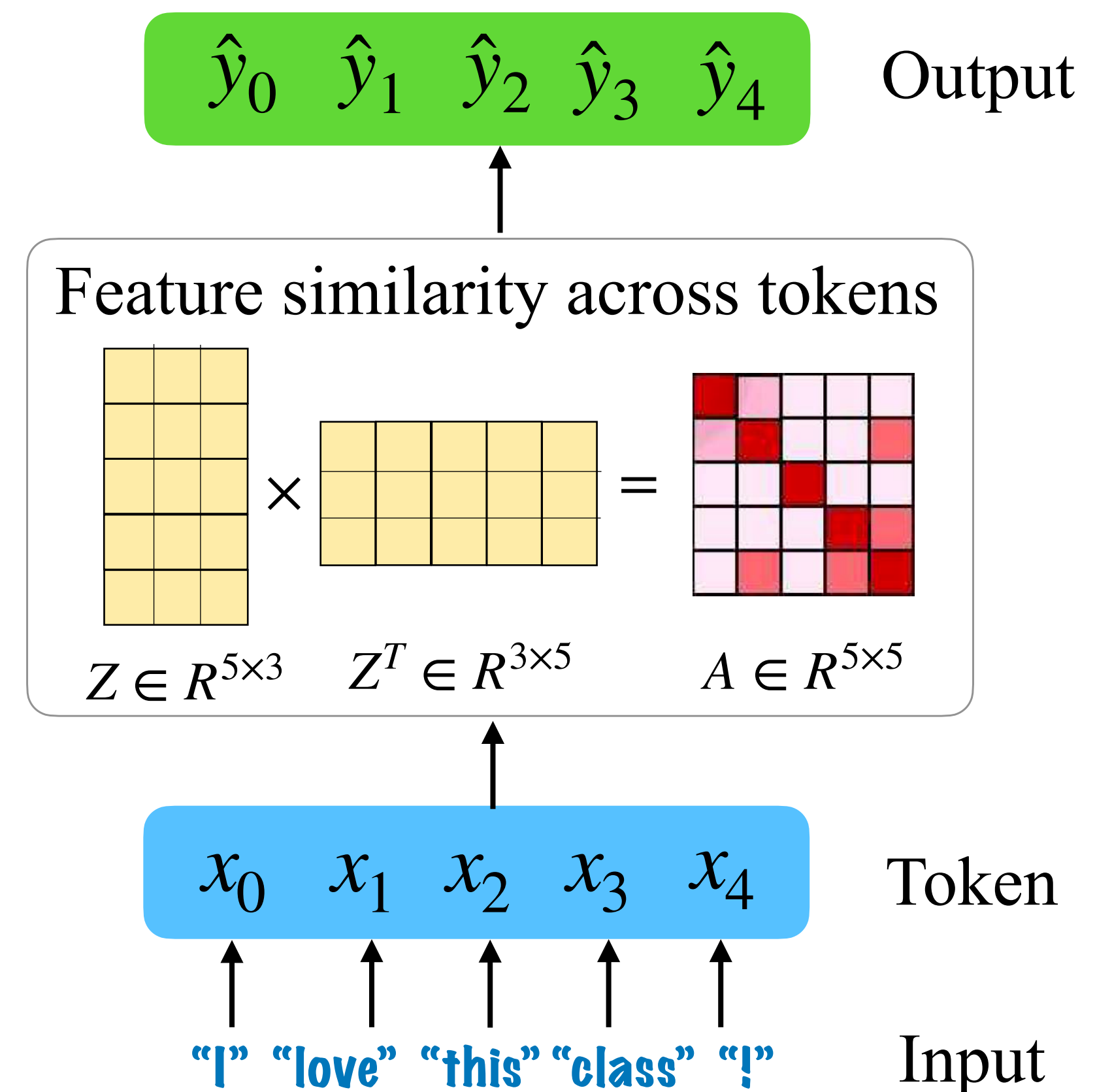
Parallelization but not continuous stream:

Core Idea: Feed everything into dense network

Q2: How to adapt to continuous stream?

Only see previous tokens!

x_0
 $x_0 \ x_1$
 $x_0 \ x_1 \ x_2$
 $x_0 \ x_1 \ x_2 \ x_3$
 $x_0 \ x_1 \ x_2 \ x_3 \ x_4$



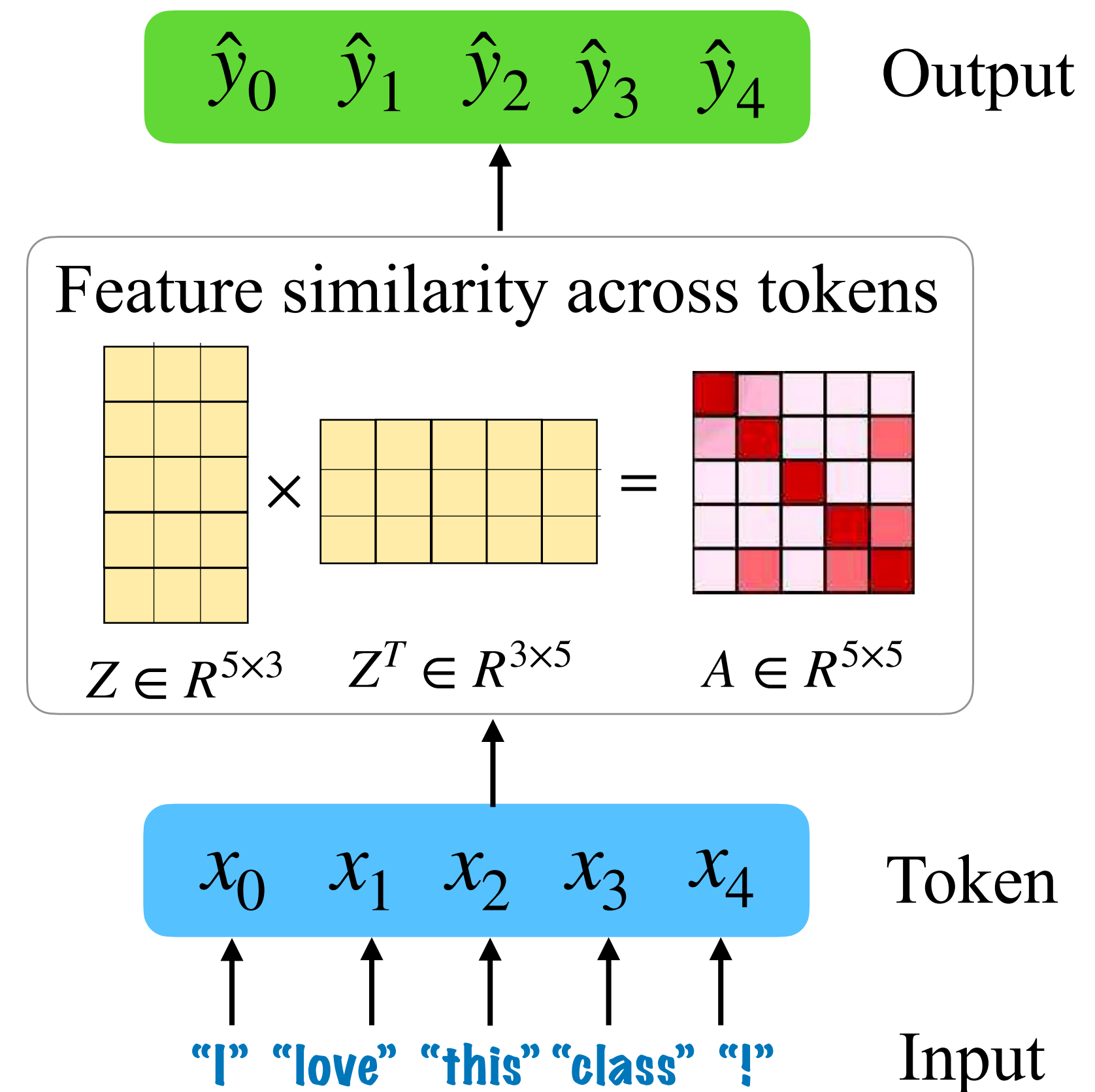
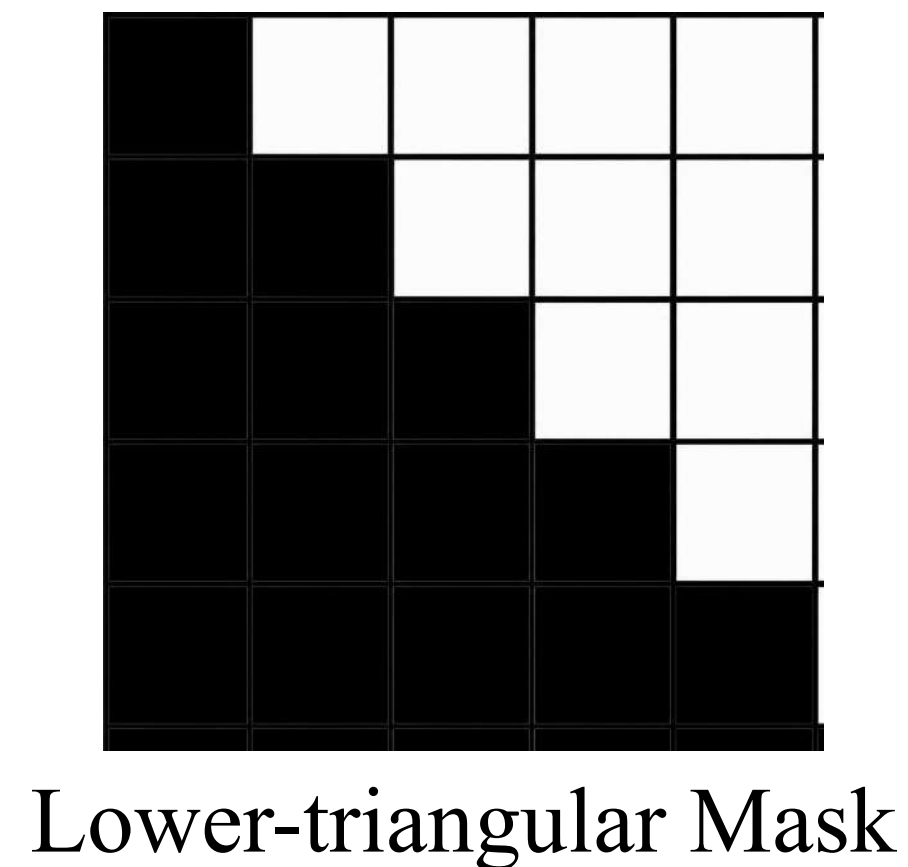
Parallelization but not continuous stream:

Core Idea: Feed everything into dense network

Q2: How to adapt to continuous stream?

Only see previous tokens!

x_0
 $x_0 \ x_1$
 $x_0 \ x_1 \ x_2$
 $x_0 \ x_1 \ x_2 \ x_3$
 $x_0 \ x_1 \ x_2 \ x_3 \ x_4$



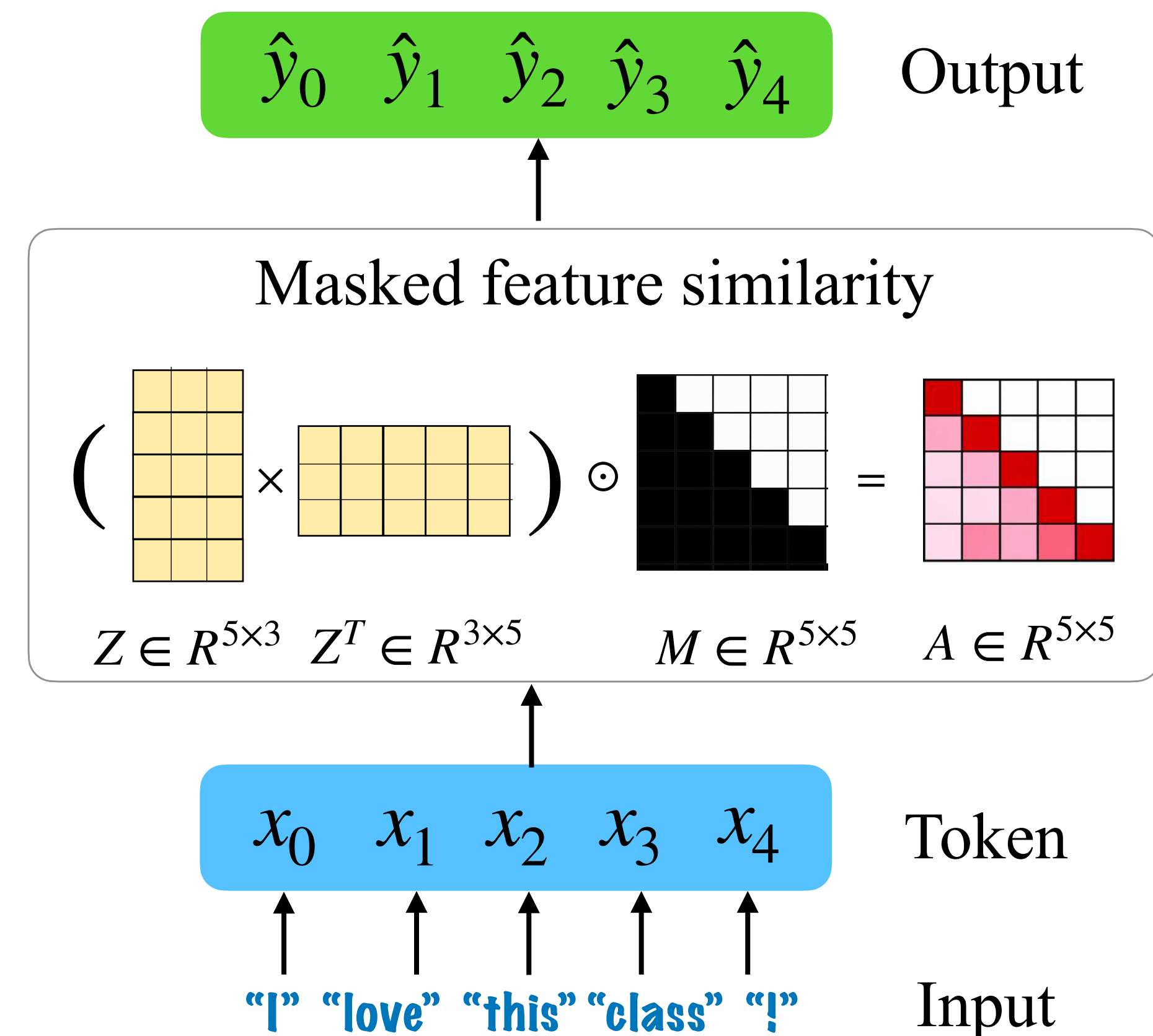
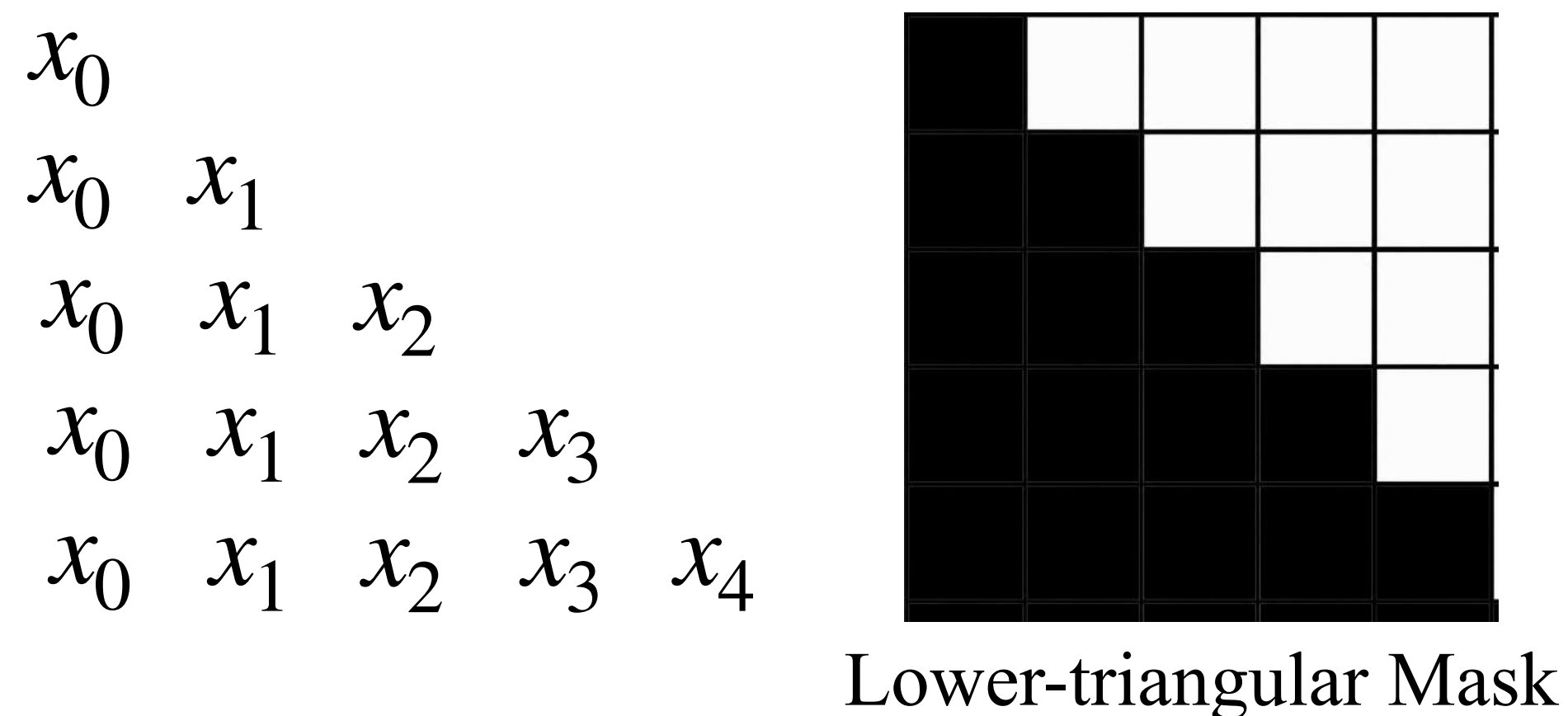
Parallelization but not continuous stream:

Core Idea: Feed everything into dense network

Q2: How to adapt to continuous stream?

Only see previous tokens!

$$A = (ZZ^T) \odot M$$



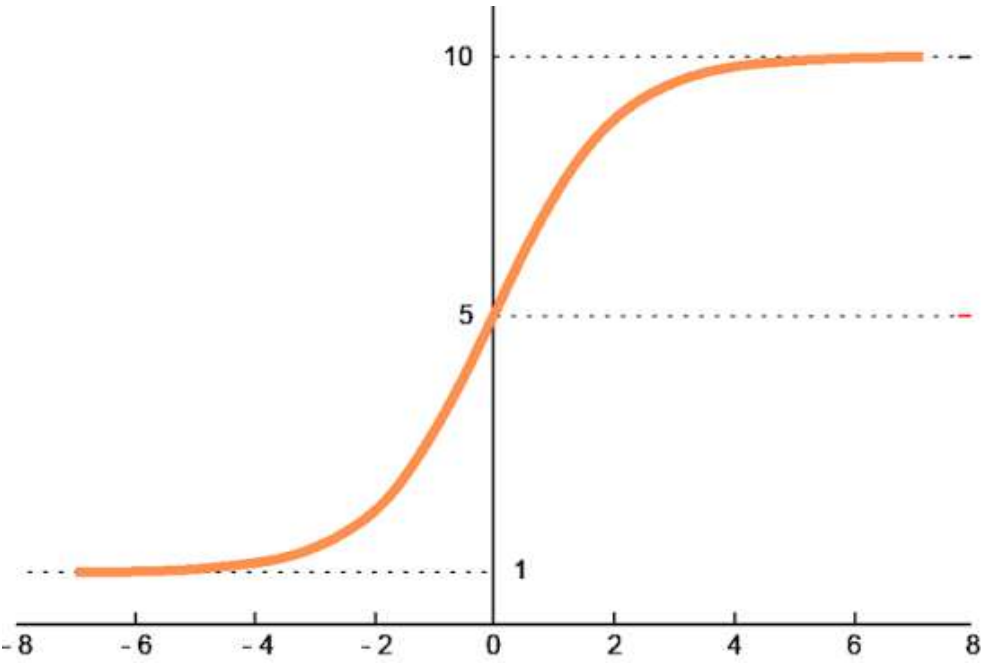
Parallelization but not continuous stream:

Core Idea: Feed everything into dense network

Q2: How to adapt to continuous stream?

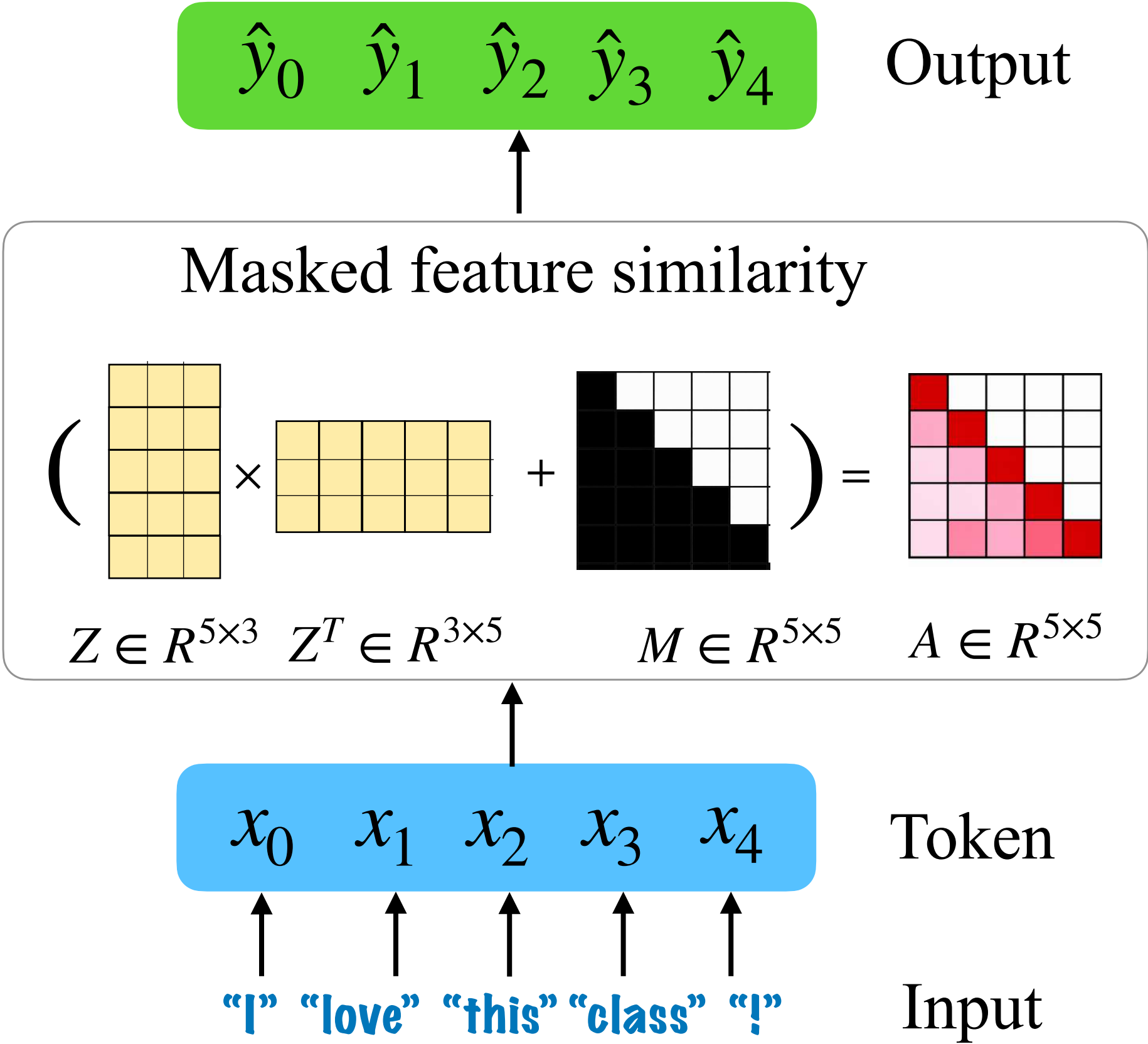
Only see previous tokens!

$$A = \text{softmax}(ZZ^T + M) \in R^{5 \times 5}$$



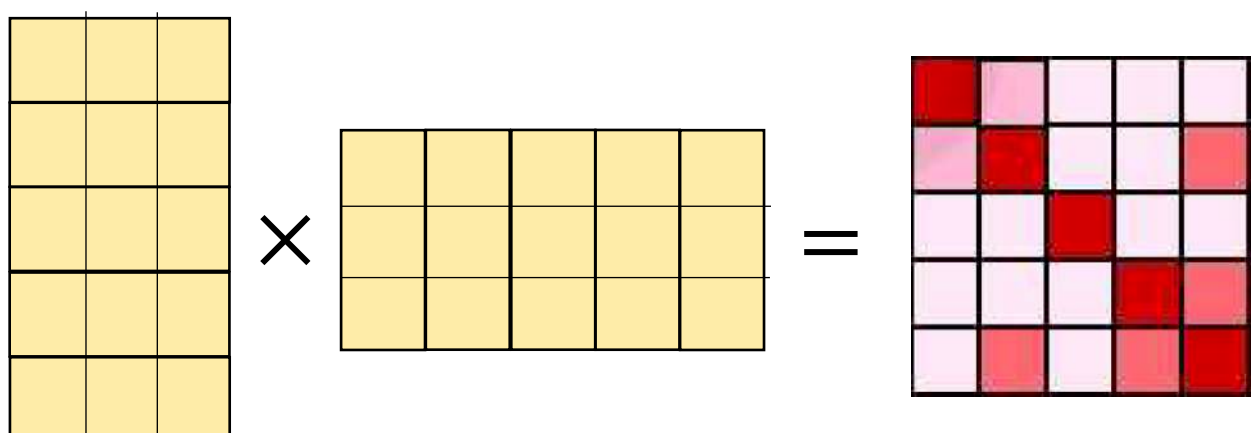
Softmax

0	-inf	-inf	-inf	-inf
0	0	-inf	-inf	-inf
0	0	0	-inf	-inf
0	0	0	0	-inf
0	0	0	0	0



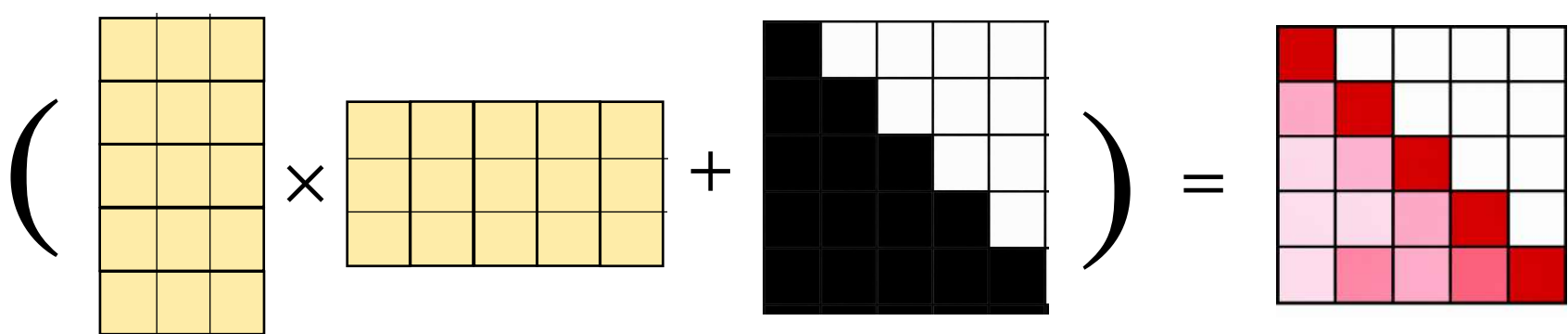
Comparison without and with mask

Feature similarity


$$\begin{matrix} Z \in R^{5 \times 3} & Z^T \in R^{3 \times 5} & A \in R^{5 \times 5} \end{matrix}$$

Not streaming

Masked feature similarity


$$\begin{matrix} Z \in R^{5 \times 3} & Z^T \in R^{3 \times 5} & M \in R^{5 \times 5} & A \in R^{5 \times 5} \end{matrix}$$

Streaming

Goal of Sequence Modeling

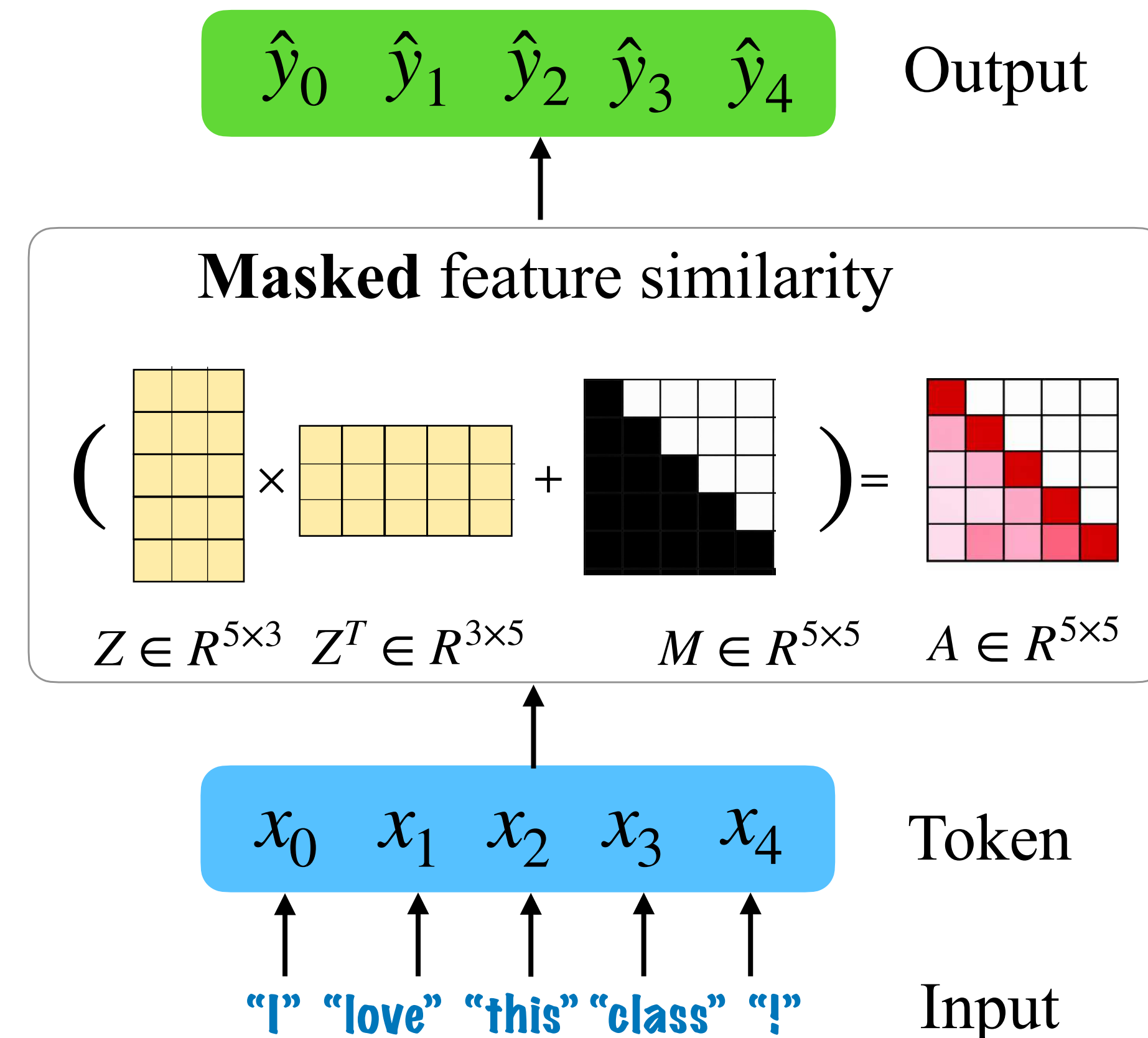
Core Idea: Feed everything into dense network

Desired Capabilities:

- ✓ No recurrence
- ✓ Continuous stream (online stream)
- ✓ Parallelization
- ✗ Long-term memory (Mamba Model)

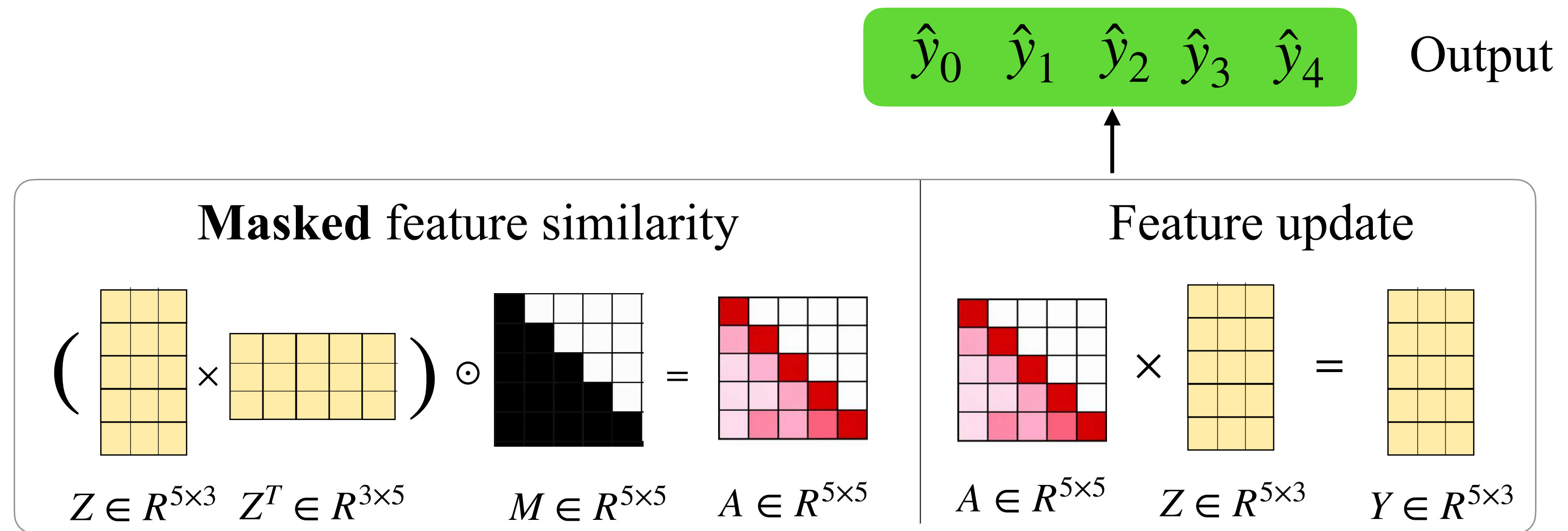
Causal attention

KV Cache



Goal of Sequence Modeling

Core Idea: Feed everything into dense network



Feature update with similarity and mask

$$Y = \text{softmax}(A \odot Z)$$

$$= \text{softmax}((ZZ^T) \odot M) Z$$

Next Lecture: Attention Is All You Need

- Scaled Dot-Product Attention
- Multi-head Attention
- Self-attention v.s. cross-attention
- Positional Embeddings (PE)
- Feed Forward Networks (FFNs)

Paper: [/https://arxiv.org/pdf/1706.03762](https://arxiv.org/pdf/1706.03762)

Visualization Github Repo: <https://github.com/jessevig/bertviz>